智能数字人 RAG 问答系统

软件著作权源代码清单

软件名称：智能数字人 RAG 问答系统
软件版本：V1.0.0
开发语言：Python
总代码行数：9,508 行
源代码文件数：38 个
申请日期：2025 年 10 月 29 日

目录

1. 1. 软件功能概述
2. 2. 技术架构说明
3. 3. 源代码组织原则
4. 4. 源代码清单
5. 4.1 第一部分：系统入口与配置
6. 4.2 第二部分：核心工具模块
7. 4.3 第三部分：数据处理核心
8. 4.4 第四部分：AI 模型集成
9. 4.5 第五部分：语音处理模块
10. 4.6 第六部分：情感分析模块
11. 4.7 第七部分：数字人控制
12. 4.8 第八部分：API 接口层
13. 4.9 第九部分：第三方集成
14. 4.10 第十部分：模块初始化
15. 5. 代码统计信息
16. 6. 技术特点说明

1. 软件功能概述

智能数字人 RAG 问答系统是一个基于检索增强生成（RAG）技术和数字人交互的智能问答系统。

系统集成了语音识别、自然语言处理、情感分析、语音合成、表情控制、唇形同步等多项

人工智能技术，实现了完整的数字人交互体验。

主要功能包括：

- 多格式文档知识库构建与智能检索
- 基于 Qwen 大模型的智能问答生成
- 语音识别与语音合成
- 情感分析与表情控制
- 唇形同步与数字人动画
- WebSocket 实时通信
- RESTful API 服务接口

2. 技术架构说明

系统采用模块化架构设计，主要包含以下技术组件：

- 核心层：文档处理、向量检索、对话记忆、会话管理
- AI 层：大语言模型集成、情感分析、智能回退
- 交互层：语音识别、语音合成、数字人控制
- 服务层：WebSocket 通信、HTTP API、第三方集成
- 工具层：日志系统、配置管理、通用工具

技术栈：Python + Flask + WebSocket + Qwen + ChromaDB + FunASR + Edge TTS

3. 源代码组织原则

源代码按照以下原则进行组织和排列：

1. 系统启动顺序：按照程序运行的逻辑顺序排列
2. 依赖关系：被依赖的模块优先放置
3. 功能重要性：核心功能模块优先
4. 模块完整性：同一模块的文件连续放置

共分为 10 个部分，38 个源代码文件，总计 9,508 行代码。

4. 源代码清单

4.2 第一部分：系统入口与配置

1. main.py

功能描述：系统主程序入口，负责应用启动和初始化

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""
智能问答系统 - 主程序入口
"""

import sys
import os
from pathlib import Path

# 添加项目路径
project_root = Path(__file__).parent
sys.path.insert(0, str(project_root))

from src.api.app import create_app
from src.utils.logger import setup_logger
from config.config_loader import load_config

def main():
    """主程序入口"""

    # 设置日志
    logger = setup_logger()
    logger.info("=" * 60)
    logger.info("智能问答系统启动中...")
    logger.info("=" * 60)

    try:
        # 加载配置
        config = load_config()
        logger.info(f"配置加载成功: {config.get('app_name', 'QA
System')}")

        # 创建必要的目录
        data_dirs = [
            "data/documents",
            "data/vectordb",
            "data/cache",
            "logs"
```

```
        ]
        for dir_path in data_dirs:
            Path(dir_path).mkdir(parents=True, exist_ok=True)
        logger.info("数据目录创建完成")

        # 创建并启动 Flask 应用
        app = create_app(config)

        host = config.get('host', '0.0.0.0')
        port = config.get('port', 5000)
        debug = config.get('debug', False)

        logger.info(f"服务地址: http://{host}:{port}")
        logger.info("系统启动成功，准备接收请求...")
        logger.info("=" * 60)

        # 启动 Web 服务
        app.run(host=host, port=port, debug=debug)

    except KeyboardInterrupt:
        logger.info("\n 接收到中断信号，正在关闭服务...")
    except Exception as e:
        logger.error(f"系统启动失败: {str(e)}", exc_info=True)
        sys.exit(1)
    finally:
        logger.info("服务已关闭")

if __name__ == "__main__":
    main()
```

代码行数：68 行

2. config/config_loader.py

功能描述：配置文件加载器，管理系统配置参数

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""
配置加载器
"""

import yaml
from pathlib import Path
from typing import Dict, Any

def load_config(config_path: str = None) -> Dict[str, Any]:
    """
    加载配置文件

    Args:
        config_path: 配置文件路径，默认为 config/config.yaml

    Returns:
        配置字典
    """
    if config_path is None:
        config_path = Path(__file__).parent / "config.yaml"
    else:
        config_path = Path(config_path)

    if not config_path.exists():
        raise FileNotFoundError(f"配置文件不存在: {config_path}")

    with open(config_path, 'r', encoding='utf-8') as f:
        config = yaml.safe_load(f)

    return config


def get_config_value(config: Dict[str, Any], key_path: str, default:
Any = None) -> Any:
    """
    获取嵌套配置值

    Args:
        config: 配置字典
        key_path: 键路径，用.分隔，如 "llm.local.model_name"
        default: 默认值

    Returns:
```

```
        配置值
    """
    keys = key_path.split('.')
    value = config

    for key in keys:
        if isinstance(value, dict) and key in value:
            value = value[key]
        else:
            return default

    return value
```

代码行数：58 行

3. config/__init__.py

功能描述：配置模块初始化文件

```
# -*- coding: utf-8 -*-
"""
配置模块
"""

from .config_loader import load_config, get_config_value

__all__ = ['load_config', 'get_config_value']
```

代码行数：12 行

4.2 第二部分：核心工具模块

4. src/utils/logger.py

功能描述：日志系统，提供统一的日志记录功能

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""
日志工具模块
"""

import sys
from pathlib import Path
from loguru import logger

def setup_logger(
    log_level: str = "INFO",
    log_dir: str = "./logs",
    log_file: str = "qa_system.log"
):
    """
    设置日志系统

    Args:
        log_level: 日志级别
        log_dir: 日志目录
        log_file: 日志文件名
    """
    # 创建日志目录
    log_path = Path(log_dir)
    log_path.mkdir(parents=True, exist_ok=True)

    # 移除默认处理器
    logger.remove()

    # 添加控制台输出
    logger.add(
        sys.stdout,
        format="<green>{time:YYYY-MM-DD HH:mm:ss}</green> |
<level>{level: <8}</level> |
<cyan>{name}</cyan>:<cyan>{function}</cyan>:<cyan>{line}</cyan> -
<level>{message}</level>",
        level=log_level,
        colorize=True
    )

    # 添加文件输出
```

```
    logger.add(
        log_path / log_file,
        format="{time:YYYY-MM-DD HH:mm:ss} | {level: <8} |
{name}:{function}:{line} - {message}",
        level=log_level,
        rotation="10 MB",  # 日志文件大小限制
        retention="7 days",  # 保留时间
        compression="zip",  # 压缩
        encoding="utf-8"
    )

    return logger


def get_logger(name: str = None):
    """
    获取 logger 实例

    Args:
        name: logger 名称

    Returns:
        logger 实例
    """
    if name:
        return logger.bind(name=name)
    return logger
```

代码行数：69 行

5. src/utils/helpers.py

功能描述：通用工具函数，提供系统级辅助功能

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""
辅助工具函数
"""

import os
import hashlib
from pathlib import Path
from typing import Any, Dict, List


def get_file_hash(file_path: str, algorithm: str = 'md5') -> str:
    """
    计算文件哈希值

    Args:
        file_path: 文件路径
        algorithm: 哈希算法 (md5/sha256)

    Returns:
        哈希值字符串
    """
    hash_func = hashlib.md5() if algorithm == 'md5' else
hashlib.sha256()

    with open(file_path, 'rb') as f:
        for chunk in iter(lambda: f.read(4096), b''):
            hash_func.update(chunk)

    return hash_func.hexdigest()


def format_file_size(size_bytes: int) -> str:
    """
    格式化文件大小

    Args:
        size_bytes: 字节数

    Returns:
        格式化后的字符串
    """
    for unit in ['B', 'KB', 'MB', 'GB', 'TB']:
        if size_bytes < 1024.0:
```

```
            return f"{size_bytes:.2f} {unit}"
        size_bytes /= 1024.0
    return f"{size_bytes:.2f} PB"


def ensure_directory(directory: str) -> Path:
    """
    确保目录存在，不存在则创建

    Args:
        directory: 目录路径

    Returns:
        Path 对象
    """
    path = Path(directory)
    path.mkdir(parents=True, exist_ok=True)
    return path


def clean_text(text: str) -> str:
    """
    清理文本

    Args:
        text: 原始文本

    Returns:
        清理后的文本
    """
    # 删除多余空白
    text = ' '.join(text.split())

    # 删除特殊字符（保留中文、英文、数字和标点）
    # text = re.sub(r'[^\w\s\u4e00-\u9fff,.!?;:，。！？；：]', '', text)

    return text.strip()


def truncate_text(text: str, max_length: int = 100, suffix: str = '...')
-> str:
    """
    截断文本

    Args:
        text: 原始文本
        max_length: 最大长度
```

```
        suffix: 后缀

    Returns:
        截断后的文本
    """
    if len(text) <= max_length:
        return text
    return text[:max_length] + suffix


def merge_dicts(*dicts: Dict) -> Dict:
    """
    合并多个字典

    Args:
        *dicts: 字典列表

    Returns:
        合并后的字典
    """
    result = {}
    for d in dicts:
        result.update(d)
    return result


def chunk_list(lst: List[Any], chunk_size: int) -> List[List[Any]]:
    """
    将列表分块

    Args:
        lst: 原始列表
        chunk_size: 块大小

    Returns:
        分块后的列表
    """
    return [lst[i:i + chunk_size] for i in range(0, len(lst),
chunk_size)]
```

代码行数：132 行

6. src/utils/__init__.py

功能描述：工具模块初始化文件

```
# -*- coding: utf-8 -*-
"""
工具模块
"""

from .logger import setup_logger, get_logger
from .helpers import (
    get_file_hash,
    format_file_size,
    ensure_directory,
    clean_text,
    truncate_text
)

__all__ = [
    'setup_logger',
    'get_logger',
    'get_file_hash',
    'format_file_size',
    'ensure_directory',
    'clean_text',
    'truncate_text'
]
```

代码行数：27 行

4.2 第三部分：数据处理核心

7. src/core/document_loader.py

功能描述：文档加载器，支持多种格式文档解析

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""
文档加载模块
支持多种格式的文档加载和解析
"""

from pathlib import Path
from typing import List, Dict, Any
from langchain_community.document_loaders import (
    TextLoader,
    PyPDFLoader,
    Docx2txtLoader,
    UnstructuredMarkdownLoader,
    CSVLoader
)
from langchain.schema import Document
from src.utils.logger import get_logger

logger = get_logger(__name__)


class DocumentLoader:
    """文档加载器"""

    # 支持的文件格式映射
    LOADER_MAPPING = {
        '.txt': TextLoader,
        '.pdf': PyPDFLoader,
        '.docx': Docx2txtLoader,
        '.md': UnstructuredMarkdownLoader,
        '.csv': CSVLoader,
    }

    def __init__(self, encoding: str = 'utf-8'):
        """
        初始化文档加载器

        Args:
            encoding: 文本编码格式
        """
        self.encoding = encoding
```

```
    def load_file(self, file_path: str) -> List[Document]:
        """
        加载单个文件

        Args:
            file_path: 文件路径

        Returns:
            文档列表
        """
        file_path = Path(file_path)

        if not file_path.exists():
            raise FileNotFoundError(f"文件不存在: {file_path}")

        # 获取文件扩展名
        file_ext = file_path.suffix.lower()

        if file_ext not in self.LOADER_MAPPING:
            raise ValueError(f"不支持的文件格式: {file_ext}")

        # 选择对应的加载器
        loader_class = self.LOADER_MAPPING[file_ext]

        try:
            # 对于文本文件，需要指定编码
            if file_ext == '.txt':
                loader = loader_class(str(file_path),
encoding=self.encoding)
            else:
                loader = loader_class(str(file_path))

            documents = loader.load()

            # 添加元数据
            for doc in documents:
                doc.metadata['source_file'] = file_path.name
                doc.metadata['file_type'] = file_ext

            logger.info(f"成功加载文档: {file_path.name}, 共
{len(documents)} 个文档片段")
            return documents

        except Exception as e:
            logger.error(f"加载文档失败 {file_path.name}: {str(e)}")
            raise
```

```
    def load_directory(self, directory: str, recursive: bool = True) ->
List[Document]:
        """
        加载目录中的所有支持的文件

        Args:
            directory: 目录路径
            recursive: 是否递归加载子目录

        Returns:
            文档列表
        """
        directory = Path(directory)

        if not directory.exists():
            raise FileNotFoundError(f"目录不存在: {directory}")

        all_documents = []

        # 遍历目录
        pattern = "**/*" if recursive else "*"
        for file_path in directory.glob(pattern):
            if file_path.is_file() and file_path.suffix.lower() in
self.LOADER_MAPPING:
                try:
                    documents = self.load_file(str(file_path))
                    all_documents.extend(documents)
                except Exception as e:
                    logger.warning(f"跳过文件 {file_path.name}: {str(e)}")
                    continue

        logger.info(f"从目录 {directory} 加载了 {len(all_documents)} 个文
档片段")
        return all_documents

    @staticmethod
    def get_supported_formats() -> List[str]:
        """
        获取支持的文件格式列表

        Returns:
            文件格式列表
        """
        return list(DocumentLoader.LOADER_MAPPING.keys())
```

代码行数：133 行

8. src/core/text_splitter.py

功能描述：文本分割器，将长文档切分为合适的片段

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""
文本分割模块
将长文档分割成适合向量化的小块
"""

from typing import List, Optional
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.schema import Document
from src.utils.logger import get_logger

logger = get_logger(__name__)


class SmartTextSplitter:
    """智能文本分割器"""

    def __init__(
        self,
        chunk_size: int = 500,
        chunk_overlap: int = 50,
        separators: Optional[List[str]] = None
    ):
        """
        初始化文本分割器

        Args:
            chunk_size: 每个文档块的字符数
            chunk_overlap: 文档块之间的重叠字符数
            separators: 分隔符列表，按优先级排列
        """
        self.chunk_size = chunk_size
        self.chunk_overlap = chunk_overlap

        # 默认中文分隔符
        if separators is None:
            separators = [
                "\n\n",   # 段落
                "\n",     # 换行
                "。",     # 句号
                "！",     # 感叹号
                "？",     # 问号
                "；",     # 分号
```

```
                ", ",     # 逗号
                " ",      # 空格
                ""        # 字符
            ]

        self.splitter = RecursiveCharacterTextSplitter(
            chunk_size=chunk_size,
            chunk_overlap=chunk_overlap,
            separators=separators,
            length_function=len,
        )

        logger.info(f"初始化文本分割器: chunk_size={chunk_size},
overlap={chunk_overlap}")

    def split_documents(self, documents: List[Document]) ->
List[Document]:
        """
        分割文档列表

        Args:
            documents: 文档列表

        Returns:
            分割后的文档列表
        """
        try:
            chunks = self.splitter.split_documents(documents)

            # 为每个 chunk 添加序号
            for i, chunk in enumerate(chunks):
                chunk.metadata['chunk_id'] = i
                chunk.metadata['chunk_size'] = len(chunk.page_content)

            logger.info(f"文档分割完成: {len(documents)} 个文档 ->
{len(chunks)} 个片段")
            return chunks

        except Exception as e:
            logger.error(f"文档分割失败: {str(e)}")
            raise

    def split_text(self, text: str, metadata: Optional[dict] = None) ->
List[Document]:
        """
        分割单个文本

        Args:
```

```
            text: 文本内容
            metadata: 元数据

        Returns:
            分割后的文档列表
        """
        if metadata is None:
            metadata = {}

        try:
            chunks = self.splitter.split_text(text)

            # 转换为 Document 对象
            documents = []
            for i, chunk in enumerate(chunks):
                doc_metadata = metadata.copy()
                doc_metadata['chunk_id'] = i
                doc_metadata['chunk_size'] = len(chunk)

                documents.append(Document(
                    page_content=chunk,
                    metadata=doc_metadata
                ))

            logger.info(f"文本分割完成: {len(text)} 字符 ->
{len(documents)} 个片段")
            return documents

        except Exception as e:
            logger.error(f"文本分割失败: {str(e)}")
            raise
```

代码行数：122 行

9. src/core/vector_store.py

功能描述：向量数据库管理，负责向量存储和检索

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""
向量存储模块
管理文档向量的存储和检索
"""

from typing import List, Optional, Dict, Any
from pathlib import Path
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_community.vectorstores import Chroma
from langchain.schema import Document
from src.utils.logger import get_logger

logger = get_logger(__name__)


class VectorStoreManager:
    """向量存储管理器"""

    def __init__(
        self,
        embedding_model_name: str = "BAAI/bge-large-zh",
        persist_directory: str = "./data/vectordb",
        collection_name: str = "qa_documents",
        device: str = "cuda"
    ):
        """
        初始化向量存储管理器

        Args:
            embedding_model_name: Embedding 模型名称
            persist_directory: 持久化目录
            collection_name: 集合名称
            device: 设备 (cuda/cpu)
        """
        self.embedding_model_name = embedding_model_name
        self.persist_directory = persist_directory
        self.collection_name = collection_name
        self.device = device

        # 创建持久化目录
        Path(persist_directory).mkdir(parents=True, exist_ok=True)
```

```
        # 初始化 Embedding 模型
        logger.info(f"加载 Embedding 模型: {embedding_model_name}")
        self.embeddings = HuggingFaceEmbeddings(
            model_name=embedding_model_name,
            model_kwargs={'device': device},
            encode_kwargs={'normalize_embeddings': True}
        )

        # 向量存储
        self.vectorstore = None

        logger.info("向量存储管理器初始化完成")

    def create_vectorstore(self, documents: List[Document]) -> Chroma:
        """
        创建向量存储（会覆盖已存在的）

        Args:
            documents: 文档列表

        Returns:
            向量存储实例
        """
        try:
            logger.info(f"创建向量存储，文档数量: {len(documents)}")

            self.vectorstore = Chroma.from_documents(
                documents=documents,
                embedding=self.embeddings,
                persist_directory=self.persist_directory,
                collection_name=self.collection_name
            )

            # 持久化
            self.vectorstore.persist()

            logger.info("向量存储创建成功并已持久化")
            return self.vectorstore

        except Exception as e:
            logger.error(f"创建向量存储失败: {str(e)}")
            raise

    def load_vectorstore(self) -> Chroma:
        """
        加载已存在的向量存储

        Returns:
```

```
        向量存储实例
    """
    try:
        logger.info("加载已存在的向量存储")

        self.vectorstore = Chroma(
            persist_directory=self.persist_directory,
            embedding_function=self.embeddings,
            collection_name=self.collection_name
        )

        # 获取文档数量
        collection = self.vectorstore._collection
        count = collection.count()

        logger.info(f"向量存储加载成功，包含 {count} 个文档")
        return self.vectorstore

    except Exception as e:
        logger.error(f"加载向量存储失败: {str(e)}")
        raise

def add_documents(self, documents: List[Document]) -> None:
    """
    向已存在的向量存储添加文档

    Args:
        documents: 文档列表
    """
    try:
        if self.vectorstore is None:
            self.load_vectorstore()

        logger.info(f"添加 {len(documents)} 个文档到向量存储")

        self.vectorstore.add_documents(documents)
        self.vectorstore.persist()

        logger.info("文档添加成功")

    except Exception as e:
        logger.error(f"添加文档失败: {str(e)}")
        raise

def similarity_search(
    self,
    query: str,
    k: int = 3,
```

```
        score_threshold: Optional[float] = None
    ) -> List[Document]:
        """
        相似度搜索

        Args:
            query: 查询文本
            k: 返回结果数量
            score_threshold: 相似度阈值

        Returns:
            相关文档列表
        """
        try:
            if self.vectorstore is None:
                self.load_vectorstore()

            logger.info(f"执行相似度搜索: query='{query[:50]}...', k={k}")

            if score_threshold is not None:
                # 带分数的搜索
                docs_and_scores =
self.vectorstore.similarity_search_with_score(
                    query, k=k
                )

                # 过滤分数
                results = [
                    doc for doc, score in docs_and_scores
                    if score >= score_threshold
                ]
            else:
                results = self.vectorstore.similarity_search(query, k=k)

            logger.info(f"搜索完成，返回 {len(results)} 个结果")
            return results

        except Exception as e:
            logger.error(f"相似度搜索失败: {str(e)}")
            raise

    def delete_collection(self) -> None:
        """删除当前集合"""
        try:
            if self.vectorstore is not None:
                self.vectorstore.delete_collection()
                self.vectorstore = None
                logger.info("向量存储集合已删除")
```

```
        except Exception as e:
            logger.error(f"删除集合失败: {str(e)}")
            raise

    def get_stats(self) -> Dict[str, Any]:
        """
        获取向量存储统计信息

        Returns:
            统计信息字典
        """
        try:
            if self.vectorstore is None:
                self.load_vectorstore()

            collection = self.vectorstore._collection
            count = collection.count()

            return {
                'collection_name': self.collection_name,
                'document_count': count,
                'embedding_model': self.embedding_model_name,
                'persist_directory': self.persist_directory
            }

        except Exception as e:
            logger.error(f"获取统计信息失败: {str(e)}")
            return {}
```

代码行数：219 行

10. src/core/memory_manager.py

功能描述：对话记忆管理器，智能管理用户对话历史

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""
对话记忆管理器
实现基于 SQLite 的对话历史存储和智能记忆功能
"""

import sqlite3
import json
import uuid
from datetime import datetime, timedelta
from typing import List, Dict, Any, Optional, Tuple
from pathlib import Path
import threading
from src.utils.logger import get_logger

logger = get_logger(__name__)


class MemoryManager:
    """对话记忆管理器"""

    def __init__(
        self,
        db_path: str = "data/memory/conversations.db",
        max_history_length: int = 20,
        max_context_tokens: int = 4000,
        summary_threshold: int = 10
    ):
        """
        初始化记忆管理器

        Args:
            db_path: 数据库文件路径
            max_history_length: 最大历史消息数量
            max_context_tokens: 最大上下文 token 数（估算）
            summary_threshold: 触发摘要的消息数量阈值
        """
        self.db_path = Path(db_path)
        self.max_history_length = max_history_length
        self.max_context_tokens = max_context_tokens
        self.summary_threshold = summary_threshold
        self._lock = threading.Lock()

        # 确保数据库目录存在
```

```
        self.db_path.parent.mkdir(parents=True, exist_ok=True)

        # 初始化数据库
        self._init_database()

        logger.info(f"记忆管理器初始化完成: {self.db_path}")

    def _init_database(self):
        """初始化数据库表结构"""
        with sqlite3.connect(self.db_path) as conn:
            cursor = conn.cursor()

            # 会话表
            cursor.execute('''
                CREATE TABLE IF NOT EXISTS sessions (
                    id TEXT PRIMARY KEY,
                    user_id TEXT NOT NULL,
                    title TEXT,
                    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                    metadata TEXT DEFAULT '{}',
                    is_active INTEGER DEFAULT 1
                )
            ''')

            # 消息表
            cursor.execute('''
                CREATE TABLE IF NOT EXISTS messages (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    session_id TEXT NOT NULL,
                    role TEXT NOT NULL,
                    content TEXT NOT NULL,
                    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                    metadata TEXT DEFAULT '{}',
                    token_count INTEGER DEFAULT 0,
                    FOREIGN KEY (session_id) REFERENCES sessions (id)
                )
            ''')

            # 记忆摘要表
            cursor.execute('''
                CREATE TABLE IF NOT EXISTS memory_summaries (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    session_id TEXT NOT NULL,
                    summary_text TEXT NOT NULL,
                    message_range_start INTEGER NOT NULL,
                    message_range_end INTEGER NOT NULL,
                    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
                    FOREIGN KEY (session_id) REFERENCES sessions (id)
                )
            ''')

            # 创建索引
            cursor.execute('CREATE INDEX IF NOT EXISTS
idx_sessions_user_id ON sessions (user_id)')
            cursor.execute('CREATE INDEX IF NOT EXISTS
idx_messages_session_id ON messages (session_id)')
            cursor.execute('CREATE INDEX IF NOT EXISTS
idx_messages_timestamp ON messages (timestamp)')
            cursor.execute('CREATE INDEX IF NOT EXISTS
idx_summaries_session_id ON memory_summaries (session_id)')

            conn.commit()

    def create_session(
        self,
        user_id: str = "default_user",
        title: Optional[str] = None,
        metadata: Optional[Dict] = None
    ) -> str:
        """
        创建新会话

        Args:
            user_id: 用户 ID
            title: 会话标题
            metadata: 会话元数据

        Returns:
            会话 ID
        """
        session_id = str(uuid.uuid4())

        if not title:
            title = f"对话 {datetime.now().strftime('%Y-%m-%d %H:%M')}"

        metadata_json = json.dumps(metadata or {}, ensure_ascii=False)

        with self._lock:
            with sqlite3.connect(self.db_path) as conn:
                cursor = conn.cursor()
                cursor.execute('''
                    INSERT INTO sessions (id, user_id, title, metadata)
                    VALUES (?, ?, ?, ?)
                ''', (session_id, user_id, title, metadata_json))
                conn.commit()
```

```
        logger.info(f"创建新会话: {session_id} - {title}")
        return session_id

    def add_message(
        self,
        session_id: str,
        role: str,
        content: str,
        metadata: Optional[Dict] = None
    ) -> int:
        """
        添加消息到会话

        Args:
            session_id: 会话 ID
            role: 角色 (user/assistant/system)
            content: 消息内容
            metadata: 消息元数据

        Returns:
            消息 ID
        """
        metadata_json = json.dumps(metadata or {}, ensure_ascii=False)
        token_count = self._estimate_tokens(content)

        with self._lock:
            with sqlite3.connect(self.db_path) as conn:
                cursor = conn.cursor()

                # 添加消息
                cursor.execute('''
                    INSERT INTO messages (session_id, role, content,
metadata, token_count)
                    VALUES (?, ?, ?, ?, ?)
                ''', (session_id, role, content, metadata_json,
token_count))

                message_id = cursor.lastrowid

                # 更新会话的最后更新时间
                cursor.execute('''
                    UPDATE sessions SET updated_at = CURRENT_TIMESTAMP
                    WHERE id = ?
                ''', (session_id,))

                conn.commit()
```

```
        # 检查是否需要创建摘要
        self._check_and_create_summary(session_id)

        logger.debug(f"添加消息到会话 {session_id}: {role} - {len(content)}
字符")
        return message_id

    def get_conversation_history(
        self,
        session_id: str,
        limit: Optional[int] = None,
        include_summary: bool = True
    ) -> List[Dict[str, Any]]:
        """
        获取会话历史

        Args:
            session_id: 会话 ID
            limit: 限制消息数量
            include_summary: 是否包含历史摘要

        Returns:
            消息列表
        """
        with sqlite3.connect(self.db_path) as conn:
            cursor = conn.cursor()

            messages = []

            # 获取摘要（如果需要）
            if include_summary:
                cursor.execute('''
                    SELECT summary_text FROM memory_summaries
                    WHERE session_id = ?
                    ORDER BY created_at DESC
                    LIMIT 1
                ''', (session_id,))

                summary_row = cursor.fetchone()
                if summary_row:
                    messages.append({
                        'role': 'system',
                        'content': f"[历史对话摘要] {summary_row[0]}",
                        'is_summary': True
                    })

            # 获取最近的消息
            limit_clause = f"LIMIT {limit}" if limit else ""
```

```
            cursor.execute(f'''
                SELECT role, content, timestamp, metadata, token_count
                FROM messages
                WHERE session_id = ?
                ORDER BY timestamp DESC
                {limit_clause}
            ''', (session_id,))

            rows = cursor.fetchall()

            # 反转顺序（最新的在后面）
            for row in reversed(rows):
                role, content, timestamp, metadata_json, token_count =
row
                metadata = json.loads(metadata_json) if metadata_json
else {}

                messages.append({
                    'role': role,
                    'content': content,
                    'timestamp': timestamp,
                    'metadata': metadata,
                    'token_count': token_count,
                    'is_summary': False
                })

        return messages

    def get_contextual_memory(
        self,
        session_id: str,
        current_question: str,
        max_tokens: Optional[int] = None
    ) -> Tuple[List[Dict], str]:
        """
        获取上下文相关的记忆

        Args:
            session_id: 会话 ID
            current_question: 当前问题
            max_tokens: 最大 token 数限制

        Returns:
            (消息列表, 格式化的历史字符串)
        """
        max_tokens = max_tokens or self.max_context_tokens

        # 获取完整历史
```

```
        full_history = self.get_conversation_history(
            session_id,
            limit=self.max_history_length,
            include_summary=True
        )

        # 计算 token 并截断
        selected_messages = []
        total_tokens = 0

        # 从最新的消息开始选择
        for message in reversed(full_history):
            message_tokens = message.get('token_count',
self._estimate_tokens(message['content']))

            if total_tokens + message_tokens > max_tokens:
                break

            selected_messages.insert(0, message)
            total_tokens += message_tokens

        # 格式化历史字符串
        history_text =
self._format_history_for_prompt(selected_messages)

        return selected_messages, history_text

    def _format_history_for_prompt(self, messages: List[Dict]) -> str:
        """格式化历史消息为提示词"""
        if not messages:
            return "无对话历史"

        formatted_parts = []

        for msg in messages:
            role = msg['role']
            content = msg['content']

            if msg.get('is_summary', False):
                formatted_parts.append(f"[历史摘要] {content}")
            elif role == 'user':
                formatted_parts.append(f"用户: {content}")
            elif role == 'assistant':
                formatted_parts.append(f"助手: {content}")
            elif role == 'system':
                formatted_parts.append(f"[系统] {content}")

        return "\n".join(formatted_parts)
```

```
    def _check_and_create_summary(self, session_id: str):
        """检查并创建记忆摘要"""
        with sqlite3.connect(self.db_path) as conn:
            cursor = conn.cursor()

            # 获取消息数量
            cursor.execute('''
                SELECT COUNT(*) FROM messages WHERE session_id = ?
            ''', (session_id,))

            message_count = cursor.fetchone()[0]

            # 检查是否需要摘要
            if message_count >= self.summary_threshold:
                # 检查是否已有摘要
                cursor.execute('''
                    SELECT COUNT(*) FROM memory_summaries WHERE
session_id = ?
                ''', (session_id,))

                summary_count = cursor.fetchone()[0]

                # 如果没有摘要，或者新消息足够多，创建摘要
                if summary_count == 0 or (message_count - summary_count
* self.summary_threshold) >= self.summary_threshold:
                    self._create_summary(session_id)

    def _create_summary(self, session_id: str):
        """创建对话摘要"""
        try:
            # 获取需要摘要的消息
            with sqlite3.connect(self.db_path) as conn:
                cursor = conn.cursor()

                # 获取最后一个摘要的结束位置
                cursor.execute('''
                    SELECT MAX(message_range_end) FROM memory_summaries
                    WHERE session_id = ?
                ''', (session_id,))

                last_summary_end = cursor.fetchone()[0] or 0

                # 获取需要摘要的消息
                cursor.execute('''
                    SELECT id, role, content FROM messages
                    WHERE session_id = ? AND id > ?
                    ORDER BY timestamp
```

```
                    LIMIT ?
                ''', (session_id, last_summary_end,
self.summary_threshold))

                messages_to_summarize = cursor.fetchall()

                if len(messages_to_summarize) < 3:  # 至少需要 3 条消息才摘
要
                    return

                # 构建摘要内容
                conversation_text = []
                for msg_id, role, content in messages_to_summarize:
                    if role == 'user':
                        conversation_text.append(f"用户: {content}")
                    elif role == 'assistant':
                        conversation_text.append(f"助手: {content}")

                # 创建简单摘要（后续可以用 LLM 生成更智能的摘要）
                summary =
self._generate_simple_summary(conversation_text)

                # 保存摘要
                start_id = messages_to_summarize[0][0]
                end_id = messages_to_summarize[-1][0]

                cursor.execute('''
                    INSERT INTO memory_summaries (session_id,
summary_text, message_range_start, message_range_end)
                    VALUES (?, ?, ?, ?)
                ''', (session_id, summary, start_id, end_id))

                conn.commit()

                logger.info(f"为会话 {session_id} 创建摘要:
{len(messages_to_summarize)} 条消息")

        except Exception as e:
            logger.error(f"创建摘要失败: {str(e)}")

    def _generate_simple_summary(self, conversation_text: List[str]) ->
str:
        """生成简单摘要（后续可以用 LLM 优化）"""
        if not conversation_text:
            return "空对话"

        # 简单的关键词提取和摘要
```

```
        topics = []
        questions = []

        for line in conversation_text:
            if line.startswith("用户:"):
                question = line[3:].strip()
                if "什么" in question or "如何" in question or "?" in
question or "？" in question:
                    questions.append(question[:50])
            elif line.startswith("助手:"):
                # 提取关键概念
                content = line[3:].strip()
                if len(content) > 20:
                    topics.append(content[:100])

        summary_parts = []
        if questions:
            summary_parts.append(f"主要问题: {', '.join(questions[:2])}")
        if topics:
            summary_parts.append(f"讨论内容: {topics[0]}")

        return "; ".join(summary_parts) if summary_parts else "一般性对话
"

    def _estimate_tokens(self, text: str) -> int:
        """估算文本的 token 数量（简单估算）"""
        # 中文大约 1.5 个字符=1 个 token，英文大约 4 个字符=1 个 token
        chinese_chars = len([c for c in text if '\u4e00' <= c <=
'\u9fff'])
        other_chars = len(text) - chinese_chars

        return int(chinese_chars / 1.5 + other_chars / 4)

    def get_session_info(self, session_id: str) -> Optional[Dict[str,
Any]]:
        """获取会话信息"""
        with sqlite3.connect(self.db_path) as conn:
            cursor = conn.cursor()
            cursor.execute('''
                SELECT id, user_id, title, created_at, updated_at,
metadata
                FROM sessions WHERE id = ?
            ''', (session_id,))

            row = cursor.fetchone()
            if not row:
                return None
```

```
            # 获取消息统计
            cursor.execute('''
                SELECT COUNT(*), MAX(timestamp) FROM messages WHERE
session_id = ?
            ''', (session_id,))

            msg_count, last_message_time = cursor.fetchone()

            return {
                'id': row[0],
                'user_id': row[1],
                'title': row[2],
                'created_at': row[3],
                'updated_at': row[4],
                'metadata': json.loads(row[5]) if row[5] else {},
                'message_count': msg_count or 0,
                'last_message_time': last_message_time
            }

    def list_user_sessions(
        self,
        user_id: str = "default_user",
        limit: int = 50,
        active_only: bool = True
    ) -> List[Dict[str, Any]]:
        """获取用户的会话列表"""
        with sqlite3.connect(self.db_path) as conn:
            cursor = conn.cursor()

            where_clause = "WHERE user_id = ?"
            params = [user_id]

            if active_only:
                where_clause += " AND is_active = 1"

            cursor.execute(f'''
                SELECT s.id, s.title, s.created_at, s.updated_at,
s.metadata,
                       COUNT(m.id) as message_count,
                       MAX(m.timestamp) as last_message_time
                FROM sessions s
                LEFT JOIN messages m ON s.id = m.session_id
                {where_clause}
                GROUP BY s.id
                ORDER BY s.updated_at DESC
                LIMIT ?
            ''', params + [limit])

            sessions = []
```

```
            for row in cursor.fetchall():
                sessions.append({
                    'id': row[0],
                    'title': row[1],
                    'created_at': row[2],
                    'updated_at': row[3],
                    'metadata': json.loads(row[4]) if row[4] else {},
                    'message_count': row[5] or 0,
                    'last_message_time': row[6]
                })

            return sessions

    def delete_session(self, session_id: str) -> bool:
        """删除会话（软删除）"""
        with self._lock:
            with sqlite3.connect(self.db_path) as conn:
                cursor = conn.cursor()
                cursor.execute('''
                    UPDATE sessions SET is_active = 0 WHERE id = ?
                ''', (session_id,))

                affected = cursor.rowcount
                conn.commit()

                if affected > 0:
                    logger.info(f"删除会话: {session_id}")
                    return True
                return False

    def update_session_title(self, session_id: str, title: str) -> bool:
        """更新会话标题"""
        with self._lock:
            with sqlite3.connect(self.db_path) as conn:
                cursor = conn.cursor()
                cursor.execute('''
                    UPDATE sessions SET title = ?, updated_at =
CURRENT_TIMESTAMP
                    WHERE id = ?
                ''', (title, session_id))

                affected = cursor.rowcount
                conn.commit()

                if affected > 0:
                    logger.info(f"更新会话标题: {session_id} -> {title}")
                    return True
                return False
```

```
    def cleanup_old_sessions(self, days_old: int = 30):
        """清理旧会话"""
        cutoff_date = datetime.now() - timedelta(days=days_old)

        with self._lock:
            with sqlite3.connect(self.db_path) as conn:
                cursor = conn.cursor()

                # 标记旧会话为非活跃
                cursor.execute('''
                    UPDATE sessions SET is_active = 0
                    WHERE updated_at < ? AND is_active = 1
                ''', (cutoff_date,))

                affected = cursor.rowcount
                conn.commit()

                if affected > 0:
                    logger.info(f"清理了 {affected} 个旧会话")

                return affected


# 全局记忆管理器实例
_memory_manager = None

def get_memory_manager() -> MemoryManager:
    """获取全局记忆管理器实例"""
    global _memory_manager
    if _memory_manager is None:
        _memory_manager = MemoryManager()
    return _memory_manager
```

代码行数：579 行

11. src/core/session_manager.py

功能描述：会话管理器，管理用户会话状态

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""
会话管理模块
用于管理用户的问答会话历史
"""

import json
import time
from pathlib import Path
from typing import List, Dict, Optional
from datetime import datetime
from src.utils.logger import get_logger

logger = get_logger(__name__)


class SessionManager:
    """会话管理器"""

    def __init__(self, data_dir: str = "data/sessions"):
        """
        初始化会话管理器

        Args:
            data_dir: 会话数据存储目录
        """
        self.data_dir = Path(data_dir)
        self.data_dir.mkdir(parents=True, exist_ok=True)
        self.sessions_file = self.data_dir / "sessions.json"

        # 确保会话索引文件存在
        if not self.sessions_file.exists():
            self._save_sessions_index([])
            logger.info("创建会话索引文件")

    def _save_sessions_index(self, sessions: List[Dict]):
        """保存会话索引"""
        with open(self.sessions_file, 'w', encoding='utf-8') as f:
            json.dump(sessions, f, ensure_ascii=False, indent=2)

    def _load_sessions_index(self) -> List[Dict]:
        """加载会话索引"""
        try:
            with open(self.sessions_file, 'r', encoding='utf-8') as f:
```

```
                return json.load(f)
        except Exception as e:
            logger.error(f"加载会话索引失败: {e}")
            return []

    def create_session(self, title: str = None) -> Dict:
        """
        创建新会话

        Args:
            title: 会话标题，如果为空则自动生成

        Returns:
            会话信息字典
        """
        session_id = f"session_{int(time.time() * 1000)}"

        if not title:
            title = f"新对话 - {datetime.now().strftime('%m月%d
日 %H:%M')}"

        session = {
            'id': session_id,
            'title': title,
            'created_at': datetime.now().isoformat(),
            'updated_at': datetime.now().isoformat(),
            'message_count': 0
        }

        # 创建会话文件
        session_file = self.data_dir / f"{session_id}.json"
        session_data = {
            'info': session,
            'messages': []
        }

        with open(session_file, 'w', encoding='utf-8') as f:
            json.dump(session_data, f, ensure_ascii=False, indent=2)

        # 更新会话索引
        sessions = self._load_sessions_index()
        sessions.insert(0, session)  # 新会话放在最前面
        self._save_sessions_index(sessions)

        logger.info(f"创建新会话: {session_id} - {title}")
        return session

    def get_session(self, session_id: str) -> Optional[Dict]:
```

```
    """
    获取会话详情

    Args:
        session_id: 会话 ID

    Returns:
        会话数据（包含消息列表），如果不存在返回 None
    """
    session_file = self.data_dir / f"{session_id}.json"

    if not session_file.exists():
        logger.warning(f"会话不存在: {session_id}")
        return None

    try:
        with open(session_file, 'r', encoding='utf-8') as f:
            return json.load(f)
    except Exception as e:
        logger.error(f"加载会话失败: {e}")
        return None

def list_sessions(self, limit: int = 50) -> List[Dict]:
    """
    获取会话列表

    Args:
        limit: 返回会话数量限制

    Returns:
        会话列表
    """
    sessions = self._load_sessions_index()
    return sessions[:limit]

def add_message(self, session_id: str, role: str, content: str,
                sources: List[Dict] = None) -> bool:
    """
    向会话添加消息

    Args:
        session_id: 会话 ID
        role: 消息角色 (user/assistant/system)
        content: 消息内容
        sources: 参考来源（仅对 assistant 消息）

    Returns:
```

```
            是否成功
        """
        session_data = self.get_session(session_id)
        if not session_data:
            return False

        message = {
            'role': role,
            'content': content,
            'timestamp': datetime.now().isoformat(),
            'time': datetime.now().strftime('%H:%M')
        }

        if sources:
            message['sources'] = sources

        # 添加消息
        session_data['messages'].append(message)

        # 更新会话信息
        session_data['info']['updated_at'] = datetime.now().isoformat()
        session_data['info']['message_count'] =
len(session_data['messages'])

        # 如果是第一条用户消息，用它作为标题
        if role == 'user' and session_data['info']['message_count'] ==
1:
            # 截取前 30 个字符作为标题
            session_data['info']['title'] = content[:30] + ('...' if
len(content) > 30 else '')

        # 保存会话文件
        session_file = self.data_dir / f"{session_id}.json"
        with open(session_file, 'w', encoding='utf-8') as f:
            json.dump(session_data, f, ensure_ascii=False, indent=2)

        # 更新索引中的会话信息
        sessions = self._load_sessions_index()
        for i, sess in enumerate(sessions):
            if sess['id'] == session_id:
                sessions[i] = session_data['info']
                # 将更新的会话移到最前面
                sessions.insert(0, sessions.pop(i))
                break
        self._save_sessions_index(sessions)

        return True
```

```
def delete_session(self, session_id: str) -> bool:
    """
    删除会话

    Args:
        session_id: 会话 ID

    Returns:
        是否成功
    """
    session_file = self.data_dir / f"{session_id}.json"

    if not session_file.exists():
        return False

    try:
        # 删除会话文件
        session_file.unlink()

        # 从索引中移除
        sessions = self._load_sessions_index()
        sessions = [s for s in sessions if s['id'] != session_id]
        self._save_sessions_index(sessions)

        logger.info(f"删除会话: {session_id}")
        return True
    except Exception as e:
        logger.error(f"删除会话失败: {e}")
        return False

def update_session_title(self, session_id: str, title: str) -> bool:
    """
    更新会话标题

    Args:
        session_id: 会话 ID
        title: 新标题

    Returns:
        是否成功
    """
    session_data = self.get_session(session_id)
    if not session_data:
        return False

    session_data['info']['title'] = title
    session_data['info']['updated_at'] = datetime.now().isoformat()
```

```
        # 保存会话文件
        session_file = self.data_dir / f"{session_id}.json"
        with open(session_file, 'w', encoding='utf-8') as f:
            json.dump(session_data, f, ensure_ascii=False, indent=2)

        # 更新索引
        sessions = self._load_sessions_index()
        for i, sess in enumerate(sessions):
            if sess['id'] == session_id:
                sessions[i]['title'] = title
                sessions[i]['updated_at'] =
session_data['info']['updated_at']
                break
        self._save_sessions_index(sessions)

        logger.info(f"更新会话标题: {session_id} -> {title}")
        return True

    def clear_old_sessions(self, keep_days: int = 30) -> int:
        """
        清理旧会话

        Args:
            keep_days: 保留最近几天的会话

        Returns:
            删除的会话数量
        """
        import datetime as dt

        cutoff_time = dt.datetime.now() - dt.timedelta(days=keep_days)
        sessions = self._load_sessions_index()

        deleted_count = 0
        remaining_sessions = []

        for session in sessions:
            try:
                updated_at =
dt.datetime.fromisoformat(session['updated_at'])
                if updated_at < cutoff_time:
                    # 删除旧会话
                    self.delete_session(session['id'])
                    deleted_count += 1
                else:
                    remaining_sessions.append(session)
            except Exception as e:
                logger.error(f"处理会话时出错: {e}")
```

```
                remaining_sessions.append(session)

        if deleted_count > 0:
            self._save_sessions_index(remaining_sessions)
            logger.info(f"清理了 {deleted_count} 个旧会话")

        return deleted_count
```

代码行数：285 行

12. src/core/fallback_manager.py

功能描述：智能回退管理器，处理知识库不足的情况

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""
回退搜索管理器
当文档库中没有相关答案时，使用大模型进行通用问答
"""

import re
from typing import List, Dict, Any, Optional, Tuple
from langchain.schema import Document
from src.utils.logger import get_logger

logger = get_logger(__name__)


class FallbackManager:
    """回退搜索管理器"""

    def __init__(
        self,
        relevance_threshold: float = 0.3,
        min_content_length: int = 50,
        max_fallback_ratio: float = 0.8
    ):
        """
        初始化回退管理器

        Args:
            relevance_threshold: 相关性阈值，低于此值触发回退
            min_content_length: 最小内容长度，低于此长度可能触发回退
            max_fallback_ratio: 最大回退比例，超过此比例的问题会使用回退
        """
        self.relevance_threshold = relevance_threshold
        self.min_content_length = min_content_length
        self.max_fallback_ratio = max_fallback_ratio

        # 常见的需要回退的问题类型
        self.fallback_patterns = [
            r'什么是.*?',
            r'如何.*?',
            r'怎么.*?',
            r'为什么.*?',
            r'.*?的定义',
            r'.*?的概念',
```

```
            r'.*?的原理',
            r'.*?的历史',
            r'.*?的发展',
            r'解释.*?',
            r'介绍.*?',
            r'.*?有什么.*?',
            r'.*?和.*?的区别',
            r'.*?的优缺点',
            r'.*?的应用',
            r'.*?的作用'
        ]

        logger.info("回退搜索管理器初始化完成")

    def should_use_fallback(
        self,
        question: str,
        retrieved_docs: List[Document],
        similarity_scores: Optional[List[float]] = None
    ) -> Tuple[bool, str]:
        """
        判断是否应该使用回退机制

        Args:
            question: 用户问题
            retrieved_docs: 检索到的文档
            similarity_scores: 相似度分数列表

        Returns:
            (是否使用回退, 回退原因)
        """
        try:
            # 1. 检查是否有检索结果
            if not retrieved_docs:
                return True, "未检索到任何相关文档"

            # 2. 检查相似度分数
            if similarity_scores:
                max_score = max(similarity_scores)
                if max_score < self.relevance_threshold:
                    return True, f"最高相似度 {max_score:.3f} 低于阈值
{self.relevance_threshold}"

            # 3. 检查文档内容质量
            total_content_length = sum(len(doc.page_content) for doc in
retrieved_docs)
```

```
            if total_content_length < self.min_content_length:
                return True, f"文档内容过少 ({total_content_length} 字符)"

            # 4. 检查问题类型是否适合回退
            if self._is_general_knowledge_question(question):
                # 对于通用知识问题，检查文档是否真正相关
                if not self._docs_contain_relevant_info(question,
retrieved_docs):
                    return True, "通用知识问题且文档不够相关"

            # 5. 检查文档内容是否包含问题关键词
            question_keywords = self._extract_keywords(question)
            if not self._docs_contain_keywords(retrieved_docs,
question_keywords):
                return True, "文档不包含问题关键词"

            return False, "文档检索结果充分"

        except Exception as e:
            logger.error(f"判断回退失败: {str(e)}")
            return False, "判断过程出错，使用文档结果"

    def _is_general_knowledge_question(self, question: str) -> bool:
        """判断是否为通用知识问题"""
        question_lower = question.lower()

        # 检查是否匹配通用问题模式
        for pattern in self.fallback_patterns:
            if re.search(pattern, question, re.IGNORECASE):
                return True

        # 检查是否包含通用知识关键词
        general_keywords = [
            '定义', '概念', '原理', '历史', '发展', '起源',
            '什么是', '如何', '怎么', '为什么', '解释', '介绍',
            '区别', '差异', '优缺点', '优势', '劣势', '应用', '作用'
        ]

        return any(keyword in question for keyword in general_keywords)

    def _docs_contain_relevant_info(self, question: str, docs:
List[Document]) -> bool:
        """检查文档是否包含相关信息"""
        question_keywords = self._extract_keywords(question)

        # 计算关键词覆盖率
        total_keywords = len(question_keywords)
```

```
        if total_keywords == 0:
            return True

        covered_keywords = 0
        for doc in docs:
            doc_content = doc.page_content.lower()
            for keyword in question_keywords:
                if keyword.lower() in doc_content:
                    covered_keywords += 1
                    break

        coverage_ratio = covered_keywords / total_keywords
        return coverage_ratio >= 0.3  # 至少30%的关键词被覆盖

    def _docs_contain_keywords(self, docs: List[Document], keywords:
List[str]) -> bool:
        """检查文档是否包含关键词"""
        if not keywords:
            return True

        doc_text = ' '.join(doc.page_content for doc in docs).lower()

        # 至少包含一个关键词
        return any(keyword.lower() in doc_text for keyword in keywords)

    def _extract_keywords(self, question: str) -> List[str]:
        """提取问题关键词"""
        # 移除常见的停用词
        stop_words = {
            '什么', '如何', '怎么', '为什么', '哪个', '哪些', '怎样',
            '是', '的', '了', '在', '有', '和', '与', '或', '但',
            '这', '那', '这个', '那个', '一个', '一些', '所有',
            '请', '帮', '告诉', '说明', '解释', '介绍', '描述'
        }

        # 简单的关键词提取
        words = re.findall(r'[\u4e00-\u9fff]+|[a-zA-Z]+', question)
        keywords = [word for word in words if word not in stop_words
and len(word) > 1]

        return keywords[:5]  # 最多返回5个关键词

    def create_hybrid_context(
        self,
        question: str,
        retrieved_docs: List[Document],
        fallback_reason: str
    ) -> Dict[str, Any]:
```

```
        """
        创建混合上下文（文档 + 回退信息）

        Args:
            question: 用户问题
            retrieved_docs: 检索到的文档
            fallback_reason: 回退原因

        Returns:
            混合上下文字典
        """
        context = {
            'has_documents': bool(retrieved_docs),
            'document_count': len(retrieved_docs),
            'fallback_reason': fallback_reason,
            'document_content': '',
            'question_type': self._classify_question_type(question)
        }

        # 如果有文档，提取内容
        if retrieved_docs:
            doc_contents = []
            for i, doc in enumerate(retrieved_docs[:3]):  # 最多使用前 3 个
文档
                content = doc.page_content[:300]  # 限制长度
                source = doc.metadata.get('source', f'文档{i+1}')
                doc_contents.append(f"[{source}] {content}")

            context['document_content'] = '\n\n'.join(doc_contents)

        return context

    def _classify_question_type(self, question: str) -> str:
        """分类问题类型"""
        question_lower = question.lower()

        if any(word in question_lower for word in ['什么是', '定义', '概念
']):
            return 'definition'
        elif any(word in question_lower for word in ['如何', '怎么', '怎
样']):
            return 'how_to'
        elif '为什么' in question_lower:
            return 'why'
        elif any(word in question_lower for word in ['区别', '差异', '不
同']):
            return 'comparison'
```

```
        elif any(word in question_lower for word in ['优缺点', '优势', '
劣势']):
            return 'pros_cons'
        elif any(word in question_lower for word in ['应用', '用途', '作
用']):
            return 'application'
        else:
            return 'general'

    def get_fallback_strategy(self, question_type: str) -> Dict[str,
str]:
        """
        根据问题类型获取回退策略

        Args:
            question_type: 问题类型

        Returns:
            回退策略字典
        """
        strategies = {
            'definition': {
                'approach': 'provide_definition',
                'instruction': '请提供准确的定义和基本概念解释'
            },
            'how_to': {
                'approach': 'step_by_step',
                'instruction': '请提供详细的步骤和操作方法'
            },
            'why': {
                'approach': 'explain_reasons',
                'instruction': '请解释原因和背后的逻辑'
            },
            'comparison': {
                'approach': 'compare_contrast',
                'instruction': '请对比分析不同选项的特点'
            },
            'pros_cons': {
                'approach': 'analyze_advantages',
                'instruction': '请分析优势和劣势'
            },
            'application': {
                'approach': 'practical_examples',
                'instruction': '请提供实际应用场景和例子'
            },
            'general': {
                'approach': 'comprehensive_answer',
```

```
                'instruction': '请提供全面而准确的回答'
            }
        }

        return strategies.get(question_type, strategies['general'])


def get_fallback_manager() -> FallbackManager:
    """获取全局回退管理器实例"""
    global _fallback_manager
    if '_fallback_manager' not in globals():
        _fallback_manager = FallbackManager()
    return _fallback_manager
```

代码行数：278 行

13. src/core/qa_chain.py

功能描述：问答链核心，整合 RAG 检索和 LLM 生成

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""
问答链模块
实现基于 RAG 的智能问答
"""

from typing import List, Dict, Any, Optional
from langchain.chains import RetrievalQA
from langchain.schema import Document
from src.core.vector_store import VectorStoreManager
from src.core.memory_manager import MemoryManager, get_memory_manager
from src.core.fallback_manager import FallbackManager,
get_fallback_manager
from src.llm.qwen_client import QwenClientBase
from src.llm.prompt_template import PromptManager
from src.utils.logger import get_logger

logger = get_logger(__name__)


class QAChain:
    """问答链"""

    def __init__(
        self,
        vector_store: VectorStoreManager,
        llm_client: QwenClientBase,
        prompt_manager: Optional[PromptManager] = None,
        memory_manager: Optional[MemoryManager] = None,
        fallback_manager: Optional[FallbackManager] = None,
        top_k: int = 3
    ):
        """
        初始化问答链

        Args:
            vector_store: 向量存储管理器
            llm_client: LLM 客户端
            prompt_manager: Prompt 管理器
            memory_manager: 记忆管理器
            fallback_manager: 回退管理器
            top_k: 检索文档数量
        """
```

```
        self.vector_store = vector_store
        self.llm_client = llm_client
        self.prompt_manager = prompt_manager or PromptManager()
        self.memory_manager = memory_manager or get_memory_manager()
        self.fallback_manager = fallback_manager or
get_fallback_manager()
        self.top_k = top_k

        logger.info("问答链初始化完成（支持对话记忆 + 智能回退）")

    def answer(
        self,
        question: str,
        session_id: Optional[str] = None,
        return_sources: bool = True,
        return_related: bool = True
    ) -> Dict[str, Any]:
        """
        回答问题

        Args:
            question: 用户问题
            session_id: 会话 ID（用于记忆管理）
            return_sources: 是否返回来源文档
            return_related: 是否返回相关问题

        Returns:
            答案字典，包含 answer, sources, related 等字段
        """
        try:
            logger.info(f"处理问题: {question}")
            if session_id:
                logger.info(f"会话 ID: {session_id}")

            # 1. 获取对话记忆（如果有会话 ID）
            conversation_history = ""
            if session_id:
                try:
                    # 保存用户问题到记忆
                    self.memory_manager.add_message(session_id, 'user',
question)

                    # 获取相关的对话历史
                    memory_messages, conversation_history =
self.memory_manager.get_contextual_memory(
                        session_id, question
                    )
```

```
                    logger.info(f"获取到 {len(memory_messages)} 条历史记忆
")
                except Exception as e:
                    logger.warning(f"获取对话记忆失败: {str(e)}")
                    conversation_history = ""

            # 2. 检索相关文档
            relevant_docs = self.vector_store.similarity_search(
                query=question,
                k=self.top_k
            )

            # 3. 判断是否需要使用回退机制
            use_fallback, fallback_reason =
self.fallback_manager.should_use_fallback(
                question, relevant_docs
            )

            logger.info(f"回退判断: {use_fallback}, 原因:
{fallback_reason}")

            # 4. 根据回退判断选择不同的处理策略
            if use_fallback:
                answer = self._handle_fallback_answer(
                    question, relevant_docs, fallback_reason,
conversation_history
                )
                answer_mode = 'fallback'
            else:
                # 使用文档进行正常问答
                context = self._build_context(relevant_docs)

                # 生成答案（包含对话历史）
                if conversation_history:
                    prompt = self.prompt_manager.format_prompt(
                        'qa_with_memory',
                        context=context,
                        question=question,
                        conversation_history=conversation_history
                    )
                else:
                    prompt = self.prompt_manager.format_prompt(
                        'qa',
                        context=context,
                        question=question
                    )

                answer = self.llm_client.generate(prompt)
```

```
                answer_mode = 'document'

            # 5. 保存回答到记忆
            if session_id:
                try:
                    self.memory_manager.add_message(
                        session_id,
                        'assistant',
                        answer,
                        metadata={
                            'sources_count': len(relevant_docs),
                            'answer_mode': answer_mode,
                            'fallback_reason': fallback_reason if
use_fallback else None,
                            'has_context': bool(relevant_docs)
                        }
                    )
                except Exception as e:
                    logger.warning(f"保存回答到记忆失败: {str(e)}")

            # 6. 构建返回结果
            result = {
                'question': question,
                'answer': answer,
                'sources': [],
                'related': [],
                'session_id': session_id,
                'answer_mode': answer_mode,
                'fallback_reason': fallback_reason if use_fallback else
None
            }

            # 7. 添加来源信息
            if return_sources:
                result['sources'] = self._extract_sources(relevant_docs)

            # 8. 生成相关问题（仅在文档模式下）
            if return_related and not use_fallback and relevant_docs:
                context = self._build_context(relevant_docs)
                result['related'] = self._generate_related_questions(
                    question, context
                )

            logger.info(f"问题回答完成 (模式: {answer_mode})")
            return result

        except Exception as e:
            logger.error(f"问题回答失败: {str(e)}")
```

```
            return {
                'question': question,
                'answer': f'抱歉，处理您的问题时出现错误: {str(e)}',
                'sources': [],
                'related': []
            }

    def chat(
        self,
        messages: List[Dict[str, str]],
        return_sources: bool = True
    ) -> Dict[str, Any]:
        """
        多轮对话

        Args:
            messages: 对话消息列表
            return_sources: 是否返回来源文档

        Returns:
            回复字典
        """
        try:
            # 获取最后一条用户消息
            last_user_msg = None
            for msg in reversed(messages):
                if msg.get('role') == 'user':
                    last_user_msg = msg.get('content', '')
                    break

            if not last_user_msg:
                return {
                    'answer': '请输入您的问题',
                    'sources': []
                }

            # 检索相关文档
            relevant_docs = self.vector_store.similarity_search(
                query=last_user_msg,
                k=self.top_k
            )

            # 构建对话历史
            chat_history = self._build_chat_history(messages[:-1])

            # 构建上下文
            context = self._build_context(relevant_docs)
```

```
            # 生成回复
            prompt = self.prompt_manager.format_prompt(
                'chat',
                chat_history=chat_history,
                context=context,
                question=last_user_msg
            )

            answer = self.llm_client.generate(prompt)

            result = {
                'answer': answer,
                'sources': []
            }

            if return_sources:
                result['sources'] = self._extract_sources(relevant_docs)

            return result

        except Exception as e:
            logger.error(f"对话处理失败: {str(e)}")
            return {
                'answer': f'抱歉，处理对话时出现错误: {str(e)}',
                'sources': []
            }

    def _build_context(self, documents: List[Document]) -> str:
        """
        构建上下文字符串

        Args:
            documents: 文档列表

        Returns:
            上下文字符串
        """
        context_parts = []
        for i, doc in enumerate(documents, 1):
            source = doc.metadata.get('source_file', '未知来源')
            content = doc.page_content.strip()
            context_parts.append(f"[文档{i} - {source}]\n{content}")

        return "\n\n".join(context_parts)

    def _build_chat_history(self, messages: List[Dict[str, str]]) ->
str:
        """
```

```
        构建对话历史字符串

        Args:
            messages: 消息列表

        Returns:
            对话历史字符串
        """
        history_parts = []
        for msg in messages:
            role = msg.get('role', '')
            content = msg.get('content', '')

            if role == 'user':
                history_parts.append(f"用户: {content}")
            elif role == 'assistant':
                history_parts.append(f"助手: {content}")

        return "\n".join(history_parts) if history_parts else "无对话历史
"

    def _extract_sources(self, documents: List[Document]) ->
List[Dict[str, Any]]:
        """
        提取文档来源信息

        Args:
            documents: 文档列表

        Returns:
            来源信息列表
        """
        sources = []
        for doc in documents:
            sources.append({
                'source_file': doc.metadata.get('source_file', '未知'),
                'file_type': doc.metadata.get('file_type', ''),
                'chunk_id': doc.metadata.get('chunk_id', 0),
                'content_preview': doc.page_content[:100] + '...'
            })

        return sources

    def _generate_related_questions(
        self,
        question: str,
        context: str,
        num_questions: int = 3
```

```
) -> List[str]:
    """
    生成相关问题

    Args:
        question: 原问题
        context: 上下文
        num_questions: 生成数量

    Returns:
        相关问题列表
    """
    try:
        prompt = self.prompt_manager.format_prompt(
            'related',
            context=context[:500],  # 限制上下文长度
            question=question
        )

        response = self.llm_client.generate(prompt, max_tokens=200)

        # 解析生成的问题
        questions = [
            q.strip()
            for q in response.split('\n')
            if q.strip() and not q.strip().startswith('请')
        ]

        return questions[:num_questions]

    except Exception as e:
        logger.warning(f"生成相关问题失败: {str(e)}")
        return []

def answer_stream(
    self,
    question: str,
    session_id: Optional[str] = None,
    return_sources: bool = True,
    return_related: bool = True
):
    """
    流式问答

    Args:
        question: 用户问题
        session_id: 会话 ID（用于记忆管理）
```

```
            return_sources: 是否返回来源文档
            return_related: 是否返回相关问题

        Yields:
            dict: 流式数据包
        """
        try:
            logger.info(f"流式处理问题: {question}")
            if session_id:
                logger.info(f"会话 ID: {session_id}")

            # 1. 获取对话记忆
            conversation_history = ""
            if session_id:
                yield {
                    'type': 'thinking',
                    'data': {'status': 'memory', 'message': '正在获取对话
记忆...'}
                }

                try:
                    # 保存用户问题到记忆
                    self.memory_manager.add_message(session_id, 'user',
question)

                    # 获取相关的对话历史
                    memory_messages, conversation_history =
self.memory_manager.get_contextual_memory(
                        session_id, question
                    )
                    logger.info(f"获取到 {len(memory_messages)} 条历史记忆
")
                except Exception as e:
                    logger.warning(f"获取对话记忆失败: {str(e)}")
                    conversation_history = ""

            # 2. 立即返回"思考中"状态
            yield {
                'type': 'thinking',
                'data': {'status': 'searching', 'message': '正在搜索相关文
档...'}
            }

            # 3. 检索相关文档
            relevant_docs = self.vector_store.similarity_search(
                query=question,
                k=self.top_k
```

```
            )

            # 4. 判断是否需要使用回退机制
            use_fallback, fallback_reason =
self.fallback_manager.should_use_fallback(
                question, relevant_docs
            )

            yield {
                'type': 'thinking',
                'data': {
                    'status': 'analyzing',
                    'message': f'分析检索结果... (回退: {"是" if
use_fallback else "否"})'
                }
            }

            # 5. 根据回退判断处理文档来源
            if return_sources and relevant_docs:
                yield {
                    'type': 'sources',
                    'data': self._extract_sources(relevant_docs)
                }

            # 6. 构建 Prompt（根据回退情况）
            if use_fallback:
                # 使用回退模式的流式处理
                yield {
                    'type': 'thinking',
                    'data': {
                        'status': 'fallback',
                        'message': f'使用通用知识回答: {fallback_reason}'
                    }
                }

                # 构建回退 prompt
                if relevant_docs and len(relevant_docs) > 0:
                    # 混合模式
                    hybrid_context =
self.fallback_manager.create_hybrid_context(
                        question, relevant_docs, fallback_reason
                    )

                    if conversation_history:
                        prompt = f"""你是一个智能问答助手。用户提出了问题，系
统检索到了一些相关文档，但文档信息可能不够完整。请结合文档内容和你的通用知识来提供全面
的回答。
```

对话历史：
{conversation_history}

用户问题：{question}

检索到的文档内容：
{hybrid_context.get('document_content', '无相关文档')}

回退原因：{fallback_reason}

要求：
1．结合对话上下文理解问题
2．优先使用文档中的信息
3．如果文档信息不足，补充你的通用知识
4．明确区分哪些信息来自文档，哪些来自通用知识
5．保持回答的准确性和完整性

请给出你的回答："""
                    else:
                        prompt = f"""你是一个智能问答助手。用户提出了问题，系统检索到了一些相关文档，但文档信息可能不够完整。请结合文档内容和你的通用知识来提供全面的回答。

用户问题：{question}

检索到的文档内容：
{hybrid_context.get('document_content', '无相关文档')}

回退原因：{fallback_reason}

要求：
1．优先使用文档中的信息
2．如果文档信息不足，补充你的通用知识
3．明确区分哪些信息来自文档，哪些来自通用知识
4．保持回答的准确性和完整性
5．如果文档和通用知识有冲突，以文档为准并说明

请给出你的回答："""
                else:
                    # 纯回退模式
                    if conversation_history:
                        prompt = f"""你是一个具有记忆能力的 AI 助手。用户提出了问题，在文档库中没有找到足够的相关信息，请结合对话历史和你的通用知识来回答。

对话历史：

```
{conversation_history}

当前问题：{question}

回退原因：{fallback_reason}

要求：
1. 结合对话上下文理解问题
2. 基于你的通用知识提供回答
3. 保持与之前对话的连贯性
4. 如果问题与之前讨论相关，请明确指出
5. 回答要准确、有用且符合上下文

请给出你的回答："""
                    else:
                        prompt = f"""你是一个知识渊博的 AI 助手。用户向你提问，
但在专门的文档库中没有找到相关信息，请基于你的通用知识来回答。

用户问题：{question}

回退原因：{fallback_reason}

要求：
1. 基于你的知识提供准确、有用的回答
2. 如果不确定，请诚实说明
3. 回答要结构清晰、易于理解
4. 适当提供相关的背景信息
5. 如果是专业问题，建议用户查阅专业资料

请给出你的回答："""
            else:
                # 使用文档进行正常问答
                yield {
                    'type': 'thinking',
                    'data': {'status': 'document', 'message': '基于文档生
成回答...'}
                }

                context = self._build_context(relevant_docs)
                if conversation_history:
                    prompt = self.prompt_manager.format_prompt(
                        'qa_with_memory',
                        context=context,
                        question=question,
                        conversation_history=conversation_history
                    )
```

```
                else:
                    prompt = self.prompt_manager.format_prompt(
                        'qa',
                        context=context,
                        question=question
                    )

            # 6. 返回"生成中"状态
            yield {
                'type': 'thinking',
                'data': {'status': 'generating', 'message': '正在生成答
案...'}
            }

            # 7. 流式生成答案
            full_answer = ""
            for chunk in self.llm_client.generate_stream(prompt):
                full_answer += chunk
                yield {
                    'type': 'content',
                    'data': chunk
                }

            # 8. 添加回退标识（如果是回退模式）
            if use_fallback:
                full_answer = f"[基于通用知识回答] {full_answer}"

            # 9. 保存回答到记忆
            if session_id and full_answer:
                try:
                    self.memory_manager.add_message(
                        session_id,
                        'assistant',
                        full_answer,
                        metadata={
                            'sources_count': len(relevant_docs),
                            'answer_mode': 'fallback' if use_fallback
else 'document',
                            'fallback_reason': fallback_reason if
use_fallback else None,
                            'has_context': bool(relevant_docs),
                            'has_memory': bool(conversation_history)
                        }
                    )
                except Exception as e:
                    logger.warning(f"保存回答到记忆失败: {str(e)}")

            # 10. 生成完成后，返回相关问题（仅在文档模式下）
```

```
            if return_related and not use_fallback and relevant_docs:
                yield {
                    'type': 'thinking',
                    'data': {'status': 'related', 'message': '生成相关问
题...'}
                }

                context = self._build_context(relevant_docs)
                related_questions =
self._generate_related_questions(question, context)
                yield {
                    'type': 'related',
                    'data': related_questions
                }

            # 11. 发送完成标记
            yield {
                'type': 'done',
                'data': {
                    'question': question,
                    'full_answer': full_answer,
                    'sources_count': len(relevant_docs),
                    'session_id': session_id,
                    'answer_mode': 'fallback' if use_fallback else
'document',
                    'fallback_reason': fallback_reason if use_fallback
else None,
                    'has_memory': bool(conversation_history)
                }
            }

        except Exception as e:
            logger.error(f"流式问答失败: {str(e)}")
            yield {
                'type': 'error',
                'data': {'message': f'处理问题时出现错误: {str(e)}'}
            }

    def chat_stream(
        self,
        messages: List[Dict[str, str]],
        return_sources: bool = True
    ):
        """
        流式多轮对话

        Args:
            messages: 对话消息列表
```

```
            return_sources: 是否返回来源文档

        Yields:
            dict: 流式数据包
        """
        try:
            # 获取最后一条用户消息
            last_user_msg = None
            for msg in reversed(messages):
                if msg.get('role') == 'user':
                    last_user_msg = msg.get('content', '')
                    break

            if not last_user_msg:
                yield {
                    'type': 'error',
                    'data': {'message': '请输入您的问题'}
                }
                return

            # 思考状态
            yield {
                'type': 'thinking',
                'data': {'status': 'searching', 'message': '正在搜索相关文
档...'}
            }

            # 检索相关文档
            relevant_docs = self.vector_store.similarity_search(
                query=last_user_msg,
                k=self.top_k
            )

            # 返回来源
            if return_sources and relevant_docs:
                yield {
                    'type': 'sources',
                    'data': self._extract_sources(relevant_docs)
                }

            # 构建对话历史和上下文
            chat_history = self._build_chat_history(messages[:-1])
            context = self._build_context(relevant_docs)

            # 生成回复
            prompt = self.prompt_manager.format_prompt(
                'chat',
                chat_history=chat_history,
```

```
                context=context,
                question=last_user_msg
            )

            yield {
                'type': 'thinking',
                'data': {'status': 'generating', 'message': '正在生成回
复...'}
            }

            # 流式生成
            full_answer = ""
            for chunk in self.llm_client.generate_stream(prompt):
                full_answer += chunk
                yield {
                    'type': 'content',
                    'data': chunk
                }

            # 完成
            yield {
                'type': 'done',
                'data': {
                    'full_answer': full_answer,
                    'sources_count': len(relevant_docs)
                }
            }

        except Exception as e:
            logger.error(f"流式对话失败: {str(e)}")
            yield {
                'type': 'error',
                'data': {'message': f'处理对话时出现错误: {str(e)}'}
            }

    def _handle_fallback_answer(
        self,
        question: str,
        retrieved_docs: List[Document],
        fallback_reason: str,
        conversation_history: str = ""
    ) -> str:
        """
        处理回退模式的问答

        Args:
            question: 用户问题
            retrieved_docs: 检索到的文档（可能为空或不相关）
```

```
            fallback_reason: 回退原因
            conversation_history: 对话历史

        Returns:
            回答文本
        """
        try:
            logger.info(f"使用回退模式回答问题: {fallback_reason}")

            if retrieved_docs and len(retrieved_docs) > 0:
                # 混合模式：有文档但不够相关，结合文档和通用知识
                hybrid_context =
self.fallback_manager.create_hybrid_context(
                    question, retrieved_docs, fallback_reason
                )

                if conversation_history:
                    # 使用临时的混合模板（先用简单格式）
                    prompt = f"""你是一个智能问答助手。用户提出了问题，系统检索
到了一些相关文档，但文档信息可能不够完整。请结合文档内容和你的通用知识来提供全面的回
答。

对话历史：
{conversation_history}

用户问题：{question}

检索到的文档内容：
{hybrid_context.get('document_content', '无相关文档')}

回退原因：{fallback_reason}

要求：
1. 结合对话上下文理解问题
2. 优先使用文档中的信息
3. 如果文档信息不足，补充你的通用知识
4. 明确区分哪些信息来自文档，哪些来自通用知识
5. 保持回答的准确性和完整性

请给出你的回答："""
                else:
                    prompt = f"""你是一个智能问答助手。用户提出了问题，系统检索
到了一些相关文档，但文档信息可能不够完整。请结合文档内容和你的通用知识来提供全面的回
答。

用户问题：{question}
```

检索到的文档内容：
{hybrid_context.get('document_content', '无相关文档')}

回退原因：{fallback_reason}

要求：
1．优先使用文档中的信息
2．如果文档信息不足，补充你的通用知识
3．明确区分哪些信息来自文档，哪些来自通用知识
4．保持回答的准确性和完整性
5．如果文档和通用知识有冲突，以文档为准并说明

请给出你的回答："""
            else:
                # 纯回退模式：完全使用大模型通用知识
                if conversation_history:
                    prompt = f"""你是一个具有记忆能力的 AI 助手。用户提出了问题，在文档库中没有找到足够的相关信息，请结合对话历史和你的通用知识来回答。

对话历史：
{conversation_history}

当前问题：{question}

回退原因：{fallback_reason}

要求：
1．结合对话上下文理解问题
2．基于你的通用知识提供回答
3．保持与之前对话的连贯性
4．如果问题与之前讨论相关，请明确指出
5．回答要准确、有用且符合上下文

请给出你的回答："""
                else:
                    prompt = f"""你是一个知识渊博的 AI 助手。用户向你提问，但在专门的文档库中没有找到相关信息，请基于你的通用知识来回答。

用户问题：{question}

回退原因：{fallback_reason}

要求：
1．基于你的知识提供准确、有用的回答

```
2. 如果不确定，请诚实说明
3. 回答要结构清晰、易于理解
4. 适当提供相关的背景信息
5. 如果是专业问题，建议用户查阅专业资料

请给出你的回答："""

            # 生成回答
            answer = self.llm_client.generate(prompt)

            # 添加回退标识
            answer = f"[基于通用知识回答] {answer}"

            return answer

        except Exception as e:
            logger.error(f"回退模式回答失败: {str(e)}")
            return f"抱歉，处理您的问题时出现错误。回退原因: {fallback_reason}，错误: {str(e)}"
```

代码行数：821 行

4.2 第四部分：AI 模型集成

14. src/llm/prompt_template.py

功能描述：提示词模板管理，优化 LLM 输入格式

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""
Prompt 模板管理
"""

from typing import Dict, Any
from langchain.prompts import PromptTemplate


# 默认 QA 模板
DEFAULT_QA_TEMPLATE = """你是一个专业的问答助手。请基于以下提供的文档内容，准确回答用户的问题。

要求：
1. 如果文档中有明确答案，请直接引用并回答
2. 如果文档中没有完全匹配的信息，请基于相关内容进行合理推断
3. 如果完全无法从文档中找到相关信息，请诚实说明，并提供可能的建议
4. 回答要简洁明了，重点突出
5. 如果可能，引用文档来源

参考文档：
{context}

用户问题：{question}

请给出你的回答："""


# 精炼模板（用于迭代优化答案）
REFINE_TEMPLATE = """原始问题：{question}
已有答案：{existing_answer}

现在有新的文档内容可以帮助完善答案：
{context}

请基于新的文档内容，完善或修正之前的答案。如果新内容没有帮助，保持原答案不变。

完善后的回答："""
```

```
# 多轮对话模板
CHAT_TEMPLATE = """你是一个智能对话助手，可以基于提供的文档进行多轮对话。

对话历史：
{chat_history}

参考文档：
{context}

当前问题：{question}

请给出回答："""


# 相关问题推荐模板
RELATED_QUESTIONS_TEMPLATE = """基于以下文档内容和用户的问题，生成 3 个相关的推荐问题。

文档内容：
{context}

用户问题：{question}

请生成 3 个相关问题（每行一个问题，不要编号）："""


# 带记忆的 QA 模板
QA_WITH_MEMORY_TEMPLATE = """你是一个专业的问答助手，具有对话记忆能力。请基于以下信息回答用户的问题：

对话历史：
{conversation_history}

参考文档：
{context}

当前问题：{question}

要求：
1. 如果对话历史中有相关信息，请结合历史上下文回答
2. 优先使用参考文档中的信息
3. 保持回答的连贯性和一致性
4. 如果当前问题与之前的对话相关，请明确指出关联
5. 回答要简洁明了，重点突出
```

请给出你的回答："""

记忆摘要生成模板
MEMORY_SUMMARY_TEMPLATE = """请为以下对话内容生成一个简洁的摘要，突出关键信息和主要话题：

对话内容：
{conversation_text}

要求：
1. 提取主要讨论的话题和问题
2. 保留重要的结论和信息
3. 摘要长度控制在 100 字以内
4. 使用简洁明了的语言

摘要："""

回退模式问答模板（纯大模型回答）
FALLBACK_QA_TEMPLATE = """你是一个知识渊博的 AI 助手。用户向你提问，但在专门的文档库中没有找到相关信息，请基于你的通用知识来回答。

用户问题：{question}

回退原因：{fallback_reason}

要求：
1. 基于你的知识提供准确、有用的回答
2. 如果不确定，请诚实说明
3. 回答要结构清晰、易于理解
4. 适当提供相关的背景信息
5. 如果是专业问题，建议用户查阅专业资料

请给出你的回答："""

混合模式问答模板（文档+大模型）
HYBRID_QA_TEMPLATE = """你是一个智能问答助手。用户提出了问题，系统检索到了一些相关文档，但文档信息可能不够完整。请结合文档内容和你的通用知识来提供全面的回答。

用户问题：{question}

检索到的文档内容：
{document_content}

回退原因：{fallback_reason}

要求：
1. 优先使用文档中的信息
2. 如果文档信息不足，补充你的通用知识
3. 明确区分哪些信息来自文档，哪些来自通用知识
4. 保持回答的准确性和完整性
5. 如果文档和通用知识有冲突，以文档为准并说明

请给出你的回答："""

带记忆的回退问答模板
FALLBACK_QA_WITH_MEMORY_TEMPLATE = """你是一个具有记忆能力的 AI 助手。用户提出了问题，在文档库中没有找到足够的相关信息，请结合对话历史和你的通用知识来回答。

对话历史：
{conversation_history}

当前问题：{question}

回退原因：{fallback_reason}

要求：
1. 结合对话上下文理解问题
2. 基于你的通用知识提供回答
3. 保持与之前对话的连贯性
4. 如果问题与之前讨论相关，请明确指出
5. 回答要准确、有用且符合上下文

请给出你的回答："""


```
class PromptManager:
    """Prompt 管理器"""

    def __init__(self, custom_templates: Dict[str, str] = None):
        """
        初始化 Prompt 管理器

        Args:
            custom_templates: 自定义模板字典
        """
        self.templates = {
            'qa': DEFAULT_QA_TEMPLATE,
```

```
            'refine': REFINE_TEMPLATE,
            'chat': CHAT_TEMPLATE,
            'related': RELATED_QUESTIONS_TEMPLATE,
            'qa_with_memory': QA_WITH_MEMORY_TEMPLATE,
            'memory_summary': MEMORY_SUMMARY_TEMPLATE
        }

        # 覆盖自定义模板
        if custom_templates:
            self.templates.update(custom_templates)

    def get_template(self, template_name: str) -> str:
        """
        获取模板

        Args:
            template_name: 模板名称

        Returns:
            模板字符串
        """
        return self.templates.get(template_name, DEFAULT_QA_TEMPLATE)

    def get_prompt(self, template_name: str, **kwargs) ->
PromptTemplate:
        """
        获取 Prompt 对象

        Args:
            template_name: 模板名称
            **kwargs: 额外参数

        Returns:
            PromptTemplate 对象
        """
        template = self.get_template(template_name)

        if template_name == 'qa':
            input_variables = ['context', 'question']
        elif template_name == 'refine':
            input_variables = ['question', 'existing_answer', 'context']
        elif template_name == 'chat':
            input_variables = ['chat_history', 'context', 'question']
        elif template_name == 'related':
            input_variables = ['context', 'question']
        elif template_name == 'qa_with_memory':
            input_variables = ['conversation_history', 'context',
'question']
```

```
        elif template_name == 'memory_summary':
            input_variables = ['conversation_text']
        else:
            input_variables = ['context', 'question']

        return PromptTemplate(
            template=template,
            input_variables=input_variables,
            **kwargs
        )

    def format_prompt(self, template_name: str, **variables) -> str:
        """
        格式化 Prompt

        Args:
            template_name: 模板名称
            **variables: 变量字典

        Returns:
            格式化后的 prompt
        """
        template = self.get_template(template_name)
        return template.format(**variables)
```

代码行数：244 行

15. src/llm/qwen_client.py

功能描述：Qwen 大模型客户端，处理模型调用和响应

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""
Qwen 大模型客户端
支持本地部署和 API 调用两种方式
"""

from typing import Optional, List, Dict, Any
from abc import ABC, abstractmethod
from src.utils.logger import get_logger

logger = get_logger(__name__)


class QwenClientBase(ABC):
    """Qwen 客户端基类"""

    @abstractmethod
    def generate(
        self,
        prompt: str,
        max_tokens: int = 2048,
        temperature: float = 0.7,
        **kwargs
    ) -> str:
        """
        生成文本

        Args:
            prompt: 提示文本
            max_tokens: 最大生成 token 数
            temperature: 温度参数

        Returns:
            生成的文本
        """
        pass

    @abstractmethod
    def chat(
        self,
        messages: List[Dict[str, str]],
        max_tokens: int = 2048,
        temperature: float = 0.7,
        **kwargs
```

```
    ) -> str:
        """
        对话生成

        Args:
            messages: 对话消息列表
            max_tokens: 最大生成 token 数
            temperature: 温度参数

        Returns:
            生成的回复
        """
        pass

    def generate_stream(
        self,
        prompt: str,
        max_tokens: int = 2048,
        temperature: float = 0.7,
        **kwargs
    ):
        """
        流式生成文本（默认实现：回退到普通生成）

        Args:
            prompt: 提示文本
            max_tokens: 最大生成 token 数
            temperature: 温度参数

        Yields:
            str: 每次生成的文本块
        """
        # 默认实现：如果子类不支持流式，回退到普通生成
        result = self.generate(prompt, max_tokens, temperature,
**kwargs)
        yield result

    def chat_stream(
        self,
        messages: List[Dict[str, str]],
        max_tokens: int = 2048,
        temperature: float = 0.7,
        **kwargs
    ):
        """
        流式对话生成（默认实现：回退到普通对话）

        Args:
```

```
            messages: 对话消息列表
            max_tokens: 最大生成 token 数
            temperature: 温度参数

        Yields:
            str: 每次生成的文本块
        """
        # 默认实现：如果子类不支持流式，回退到普通对话
        result = self.chat(messages, max_tokens, temperature, **kwargs)
        yield result


class QwenLocalClient(QwenClientBase):
    """Qwen 本地部署客户端"""

    def __init__(
        self,
        model_path: str = "Qwen/Qwen-7B-Chat",
        device: str = "auto",
        load_in_8bit: bool = False,
        load_in_4bit: bool = False
    ):
        """
        初始化本地 Qwen 客户端

        Args:
            model_path: 模型路径
            device: 设备 (auto/cuda/cpu)
            load_in_8bit: 是否使用 8 位量化
            load_in_4bit: 是否使用 4 位量化
        """
        self.model_path = model_path
        self.device = device
        self.model = None
        self.tokenizer = None

        logger.info(f"初始化 Qwen 本地客户端: {model_path}")

        try:
            from transformers import AutoModelForCausalLM,
AutoTokenizer

            # 加载 tokenizer
            self.tokenizer = AutoTokenizer.from_pretrained(
                model_path,
                trust_remote_code=True
            )
```

```
            # 加载模型
            model_kwargs = {
                "trust_remote_code": True,
            }

            if device == "auto":
                model_kwargs["device_map"] = "auto"

            if load_in_8bit:
                model_kwargs["load_in_8bit"] = True
            elif load_in_4bit:
                model_kwargs["load_in_4bit"] = True

            self.model = AutoModelForCausalLM.from_pretrained(
                model_path,
                **model_kwargs
            ).eval()

            logger.info("Qwen 模型加载成功")

        except ImportError:
            logger.error("transformers 库未安装，请安装: pip install
transformers")
            raise
        except Exception as e:
            logger.error(f"Qwen 模型加载失败: {str(e)}")
            raise

    def generate(
        self,
        prompt: str,
        max_tokens: int = 2048,
        temperature: float = 0.7,
        **kwargs
    ) -> str:
        """生成文本"""
        try:
            response, _ = self.model.chat(
                self.tokenizer,
                prompt,
                history=None,
                max_new_tokens=max_tokens,
                temperature=temperature,
                **kwargs
            )
            return response

        except Exception as e:
```

```
            logger.error(f"文本生成失败: {str(e)}")
            raise

    def chat(
        self,
        messages: List[Dict[str, str]],
        max_tokens: int = 2048,
        temperature: float = 0.7,
        **kwargs
    ) -> str:
        """对话生成"""
        try:
            # 转换消息格式为 Qwen 格式
            history = []
            current_user_msg = ""

            for msg in messages:
                role = msg.get("role", "")
                content = msg.get("content", "")

                if role == "user":
                    current_user_msg = content
                elif role == "assistant" and current_user_msg:
                    history.append((current_user_msg, content))
                    current_user_msg = ""

            # 最后一条用户消息
            if messages and messages[-1].get("role") == "user":
                query = messages[-1].get("content", "")
            else:
                query = current_user_msg

            response, _ = self.model.chat(
                self.tokenizer,
                query,
                history=history if history else None,
                max_new_tokens=max_tokens,
                temperature=temperature,
                **kwargs
            )

            return response

        except Exception as e:
            logger.error(f"对话生成失败: {str(e)}")
            raise


class QwenAPIClient(QwenClientBase):
```

```
    """Qwen API 调用客户端（DashScope）"""

    def __init__(
        self,
        api_key: str = None,
        access_key_id: str = None,
        access_key_secret: str = None,
        model_name: str = "qwen-turbo"
    ):
        """
        初始化 Qwen API 客户端

        Args:
            api_key: DashScope API 密钥（方式 1）
            access_key_id: 阿里云 AccessKey ID（方式 2）
            access_key_secret: 阿里云 AccessKey Secret（方式 2）
            model_name: 模型名称
        """
        self.model_name = model_name

        logger.info(f"初始化 Qwen API 客户端: {model_name}")

        try:
            import dashscope

            # 优先使用 API Key，其次使用 AccessKey
            if api_key and api_key.strip() and api_key != "sk-xxxxx":
                dashscope.api_key = api_key
                logger.info("使用 API Key 认证")
            elif access_key_id and access_key_secret:
                # 使用 AccessKey 认证
                import os
                os.environ['DASHSCOPE_ACCESS_KEY_ID'] = access_key_id
                os.environ['DASHSCOPE_ACCESS_KEY_SECRET'] =
access_key_secret
                logger.info("使用 AccessKey 认证")
            else:
                raise ValueError("必须提供 api_key 或 (access_key_id +
access_key_secret)")

            self.dashscope = dashscope
            logger.info("Qwen API 客户端初始化成功")

        except ImportError:
            logger.error("dashscope 库未安装，请安装: pip install
dashscope")
            raise
```

```
def generate(
    self,
    prompt: str,
    max_tokens: int = 2048,
    temperature: float = 0.7,
    **kwargs
) -> str:
    """生成文本"""
    try:
        from dashscope import Generation

        response = Generation.call(
            model=self.model_name,
            prompt=prompt,
            max_tokens=max_tokens,
            temperature=temperature,
            **kwargs
        )

        if response.status_code == 200:
            return response.output.text
        else:
            raise Exception(f"API 调用失败: {response.message}")

    except Exception as e:
        logger.error(f"文本生成失败: {str(e)}")
        raise

def chat(
    self,
    messages: List[Dict[str, str]],
    max_tokens: int = 2048,
    temperature: float = 0.7,
    **kwargs
) -> str:
    """对话生成"""
    try:
        from dashscope import Generation

        response = Generation.call(
            model=self.model_name,
            messages=messages,
            max_tokens=max_tokens,
            temperature=temperature,
            result_format='message',
            **kwargs
        )
```

```
            if response.status_code == 200:
                return response.output.choices[0].message.content
            else:
                raise Exception(f"API 调用失败: {response.message}")

        except Exception as e:
            logger.error(f"对话生成失败: {str(e)}")
            raise

    def generate_stream(
        self,
        prompt: str,
        max_tokens: int = 2048,
        temperature: float = 0.7,
        **kwargs
    ):
        """
        流式生成文本

        Args:
            prompt: 输入提示
            max_tokens: 最大生成 token 数
            temperature: 温度参数
            **kwargs: 其他参数

        Yields:
            str: 每次生成的文本块
        """
        try:
            from dashscope import Generation

            responses = Generation.call(
                model=self.model_name,
                prompt=prompt,
                max_tokens=max_tokens,
                temperature=temperature,
                stream=True,  # 关键：启用流式
                **kwargs
            )

            for response in responses:
                if response.status_code == 200:
                    # 提取生成的文本片段
                    text = response.output.text
                    yield text
                else:
                    logger.error(f"流式生成错误: {response.message}")
                    yield f"[错误: {response.message}]"
```

```
                    break

        except Exception as e:
            logger.error(f"流式生成失败: {str(e)}")
            yield f"[生成错误: {str(e)}]"

    def chat_stream(
        self,
        messages: List[Dict[str, str]],
        max_tokens: int = 2048,
        temperature: float = 0.7,
        **kwargs
    ):
        """
        流式对话生成

        Args:
            messages: 对话消息列表
            max_tokens: 最大生成 token 数
            temperature: 温度参数
            **kwargs: 其他参数

        Yields:
            str: 每次生成的文本块
        """
        try:
            from dashscope import Generation

            responses = Generation.call(
                model=self.model_name,
                messages=messages,
                max_tokens=max_tokens,
                temperature=temperature,
                result_format='message',
                stream=True,  # 关键：启用流式
                **kwargs
            )

            for response in responses:
                if response.status_code == 200:
                    # 提取生成的文本片段
                    if hasattr(response.output, 'choices') and
response.output.choices:
                        delta = response.output.choices[0].message
                        if hasattr(delta, 'content') and delta.content:
                            yield delta.content
                else:
                    logger.error(f"流式对话错误: {response.message}")
```

```
                    yield f"[错误: {response.message}]"
                    break

        except Exception as e:
            logger.error(f"流式对话失败: {str(e)}")
            yield f"[生成错误: {str(e)}]"


def create_qwen_client(config: Dict[str, Any]) -> QwenClientBase:
    """
    根据配置创建 Qwen 客户端

    Args:
        config: 配置字典

    Returns:
        Qwen 客户端实例
    """
    llm_type = config.get('llm', {}).get('type', 'local')

    if llm_type == 'local':
        local_config = config.get('llm', {}).get('local', {})
        return QwenLocalClient(
            model_path=local_config.get('model_name', 'Qwen/Qwen-7B-
Chat'),
            device=local_config.get('device', 'auto'),
            load_in_8bit=local_config.get('load_in_8bit', False),
            load_in_4bit=local_config.get('load_in_4bit', False)
        )
    elif llm_type == 'api':
        api_config = config.get('llm', {}).get('api', {})
        return QwenAPIClient(
            api_key=api_config.get('api_key', ''),
            access_key_id=api_config.get('access_key_id', ''),
            access_key_secret=api_config.get('access_key_secret', ''),
            model_name=api_config.get('model_name', 'qwen-turbo')
        )
    else:
        raise ValueError(f"不支持的 LLM 类型: {llm_type}")
```

代码行数：465 行

16. src/llm/__init__.py

功能描述：LLM 模块初始化文件

```
# -*- coding: utf-8 -*-
"""
大模型模块
"""

from .qwen_client import (
    QwenClientBase,
    QwenLocalClient,
    QwenAPIClient,
    create_qwen_client
)
from .prompt_template import PromptManager

__all__ = [
    'QwenClientBase',
    'QwenLocalClient',
    'QwenAPIClient',
    'create_qwen_client',
    'PromptManager'
]
```

代码行数：24 行

4.2 第五部分：语音处理模块

17. src/voice/asr_handler.py

功能描述：语音识别处理器，将语音转换为文本

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""
ASR 语音识别处理器
支持多种 ASR 引擎：FunASR（离线）、阿里云 NLS（在线）
"""

import base64
import tempfile
import json
from pathlib import Path
from typing import Optional, Union
from src.utils.logger import get_logger

logger = get_logger(__name__)


class ASRHandler:
    """语音识别处理器"""

    def __init__(self, engine: str = "funasr", config: dict = None):
        """
        初始化 ASR

        Args:
            engine: 识别引擎
                - funasr: FunASR（推荐，离线，免费）
                - ali: 阿里云 NLS（在线，精度高，需 API 密钥）
            config: 配置字典
        """
        self.engine = engine
        self.config = config or {}
        self.model = None

        logger.info(f"初始化 ASR 引擎: {engine}")

        if engine == "funasr":
            self._init_funasr()
        elif engine == "ali":
            self._init_ali()
        else:
            raise ValueError(f"不支持的 ASR 引擎: {engine}")
```

```
        logger.info(f"✓ ASR 引擎初始化完成: {engine}")

    def _init_funasr(self):
        """初始化 FunASR（离线，免费）"""
        try:
            # 尝试导入 FunASR
            try:
                from funasr import AutoModel
            except ImportError:
                logger.error("FunASR 未安装，请运行: pip install funasr")
                raise ImportError(
                    "FunASR 未安装。请安装：\n"
                    "pip install funasr\n"
                    "或者切换到阿里云 ASR: engine='ali'"
                )

            # 加载模型（首次使用会自动下载）
            logger.info("正在加载 FunASR 模型（首次使用会自动下载，约
300MB）...")

            # 使用 paraformer-zh 模型（中文语音识别）
            model_name = self.config.get('model_name', 'paraformer-zh')

            self.model = AutoModel(
                model=model_name,
                disable_pbar=True,  # 禁用进度条
                device=self.config.get('device', 'cpu')  # cpu 或 cuda
            )

            logger.info("✓ FunASR 模型加载成功")

        except Exception as e:
            logger.error(f"FunASR 初始化失败: {e}")
            raise

    def _init_ali(self):
        """初始化阿里云 NLS（在线，需 API 密钥）"""
        try:
            # 检查 API 密钥
            app_key = self.config.get('app_key')
            access_key_id = self.config.get('access_key_id')
            access_key_secret = self.config.get('access_key_secret')

            if not all([app_key, access_key_id, access_key_secret]):
                raise ValueError(
                    "阿里云 ASR 需要配置 API 密钥：\n"
```

```
                    "- app_key\n"
                    "- access_key_id\n"
                    "- access_key_secret\n"
                    "请在 config.yaml 中配置，或切换到 FunASR:
engine='funasr'"
                )

            # 阿里云 NLS 需要实时初始化，这里只保存配置
            self.ali_config = {
                'app_key': app_key,
                'access_key_id': access_key_id,
                'access_key_secret': access_key_secret
            }

            logger.info("✓ 阿里云 NLS 配置已加载")

        except Exception as e:
            logger.error(f"阿里云 NLS 初始化失败: {e}")
            raise

    def recognize(
        self,
        audio_data: Optional[bytes] = None,
        audio_file: Optional[str] = None,
        format: str = 'wav'
    ) -> Optional[str]:
        """
        识别语音

        Args:
            audio_data: 音频字节数据（Base64 或原始字节）
            audio_file: 音频文件路径
            format: 音频格式（wav, mp3, pcm 等）

        Returns:
            识别的文本，失败返回 None
        """
        if not audio_data and not audio_file:
            logger.error("必须提供 audio_data 或 audio_file")
            return None

        try:
            if self.engine == "funasr":
                return self._recognize_funasr(audio_data, audio_file,
format)
            elif self.engine == "ali":
                return self._recognize_ali(audio_data, audio_file,
format)
```

```
    except Exception as e:
        logger.error(f"语音识别失败: {e}")
        return None

def _recognize_funasr(
    self,
    audio_data: Optional[bytes],
    audio_file: Optional[str],
    format: str
) -> Optional[str]:
    """FunASR 识别"""
    try:
        # 处理音频数据
        if audio_data:
            # 检查是否是 Base64 编码
            if isinstance(audio_data, str):
                # Base64 解码
                audio_data = base64.b64decode(audio_data)

            # 写入临时文件
            with tempfile.NamedTemporaryFile(
                suffix=f'.{format}',
                delete=False
            ) as f:
                f.write(audio_data)
                temp_file = f.name

            audio_file = temp_file

        # 识别
        logger.info(f"正在识别音频: {audio_file}")

        result = self.model.generate(
            input=audio_file,
            batch_size=1
        )

        # 解析结果
        if result and len(result) > 0:
            text = result[0].get('text', '')
            logger.info(f"✓ ASR 识别结果: {text}")

            # 删除临时文件
            if audio_data and Path(temp_file).exists():
                Path(temp_file).unlink()

            return text
        else:
```

```
                logger.warning("ASR 未识别到内容")
                return None

        except Exception as e:
            logger.error(f"FunASR 识别失败: {e}")
            return None

    def _recognize_ali(
        self,
        audio_data: Optional[bytes],
        audio_file: Optional[str],
        format: str
    ) -> Optional[str]:
        """阿里云 NLS 识别"""
        try:
            # TODO: 实现阿里云 NLS 识别
            # 这里需要集成阿里云 SDK
            logger.warning("阿里云 ASR 暂未实现，请使用 FunASR")
            return None

        except Exception as e:
            logger.error(f"阿里云 ASR 识别失败: {e}")
            return None

    def recognize_stream(self, audio_stream):
        """
        流式识别（实时识别）

        Args:
            audio_stream: 音频流

        Yields:
            识别的文本片段
        """
        # TODO: 实现流式识别
        logger.warning("流式识别暂未实现")
        yield ""


# 单例模式
_asr_instance: Optional[ASRHandler] = None


def get_asr_handler(engine: str = "funasr", config: dict = None) ->
ASRHandler:
    """获取 ASR 处理器实例（单例）"""
    global _asr_instance
```

```
    if _asr_instance is None:
        _asr_instance = ASRHandler(engine=engine, config=config)

    return _asr_instance
```

代码行数：233 行

18. src/voice/tts_handler.py

功能描述：语音合成处理器，将文本转换为语音

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""
TTS 语音合成处理器
支持多种 TTS 引擎：Edge TTS（免费）、阿里云 TTS（高质量）
"""

import asyncio
import hashlib
import time
from pathlib import Path
from typing import Optional, Tuple
from src.utils.logger import get_logger

logger = get_logger(__name__)


class TTSHandler:
    """语音合成处理器"""

    def __init__(self, engine: str = "edge", config: dict = None):
        """
        初始化 TTS

        Args:
            engine: 合成引擎
                - edge: Edge TTS（推荐，免费，音质好）
                - ali: 阿里云 TTS（付费，可定制）
            config: 配置字典
        """
        self.engine = engine
        self.config = config or {}

        # 音频缓存目录
        self.cache_dir = Path("data/audio_cache")
        self.cache_dir.mkdir(parents=True, exist_ok=True)

        # 声音角色映射
        self.voices = {
            'female': 'zh-CN-XiaoxiaoNeural',      # 晓晓-女声（清晰、自
然）
            'male': 'zh-CN-YunxiNeural',           # 云希-男声（温和）
            'female_warm': 'zh-CN-XiaoyiNeural',   # 晓伊-女声（温暖）
            'male_steady': 'zh-CN-YunjianNeural',  # 云健-男声（稳重）
```

```
        }

        # 情感风格（Edge TTS 支持）
        self.styles = {
            'cheerful': 'cheerful',     # 欢快
            'sad': 'sad',               # 悲伤
            'angry': 'angry',           # 愤怒
            'gentle': 'gentle',         # 温柔
            'serious': 'serious',       # 严肃
            'friendly': 'friendly',     # 友好
            'chat': 'chat',            # 聊天
        }

        logger.info(f"✅ TTS 引擎初始化完成: {engine}")

    async def synthesize(
        self,
        text: str,
        output_file: Optional[str] = None,
        voice: str = 'female',
        emotion: str = 'neutral',
        rate: float = 1.0,
        use_cache: bool = True
    ) -> Tuple[bool, Optional[str]]:
        """
        合成语音（异步）

        Args:
            text: 要合成的文本
            output_file: 输出文件路径（可选，默认自动生成）
            voice: 声音角色（female/male/female_warm/male_steady）
            emotion: 情感风格（neutral/cheerful/sad 等）
            rate: 语速（0.5-2.0，1.0 为正常速度）
            use_cache: 是否使用缓存

        Returns:
            (是否成功, 音频文件路径)
        """
        try:
            # 生成缓存 key
            cache_key = self._generate_cache_key(text, voice, emotion,
rate)
            cache_file = self.cache_dir / f"{cache_key}.mp3"

            # 检查缓存
            if use_cache and cache_file.exists():
```

```
                logger.info(f"✓ 使用缓存音频: {cache_file.name}")
                return (True, str(cache_file))

            # 如果没有指定输出文件，使用缓存文件
            if output_file is None:
                output_file = str(cache_file)

            # 根据引擎选择合成方法
            if self.engine == "edge":
                success = await self._synthesize_edge(
                    text, output_file, voice, emotion, rate
                )
            elif self.engine == "ali":
                success = await self._synthesize_ali(
                    text, output_file, voice, emotion, rate
                )
            else:
                logger.error(f"不支持的 TTS 引擎: {self.engine}")
                return (False, None)

            if success:
                logger.info(f"✓ TTS 合成成功: {Path(output_file).name}")
                return (True, output_file)
            else:
                return (False, None)

        except Exception as e:
            logger.error(f"TTS 合成失败: {e}")
            return (False, None)

    def synthesize_sync(self, *args, **kwargs) -> Tuple[bool,
Optional[str]]:
        """同步版本的合成（方便非异步环境调用）"""
        return asyncio.run(self.synthesize(*args, **kwargs))

    async def _synthesize_edge(
        self,
        text: str,
        output_file: str,
        voice: str,
        emotion: str,
        rate: float
    ) -> bool:
        """Edge TTS 合成（带重试机制）"""
        try:
            # 导入 edge-tts
            try:
                import edge_tts
```

```
            except ImportError:
                logger.error("edge-tts 未安装，请运行: pip install edge-
tts")
                raise ImportError(
                    "edge-tts 未安装。请安装：\n"
                    "pip install edge-tts"
                )

            # 获取声音 ID
            voice_id = self.voices.get(voice, self.voices['female'])

            # 构建语速参数
            rate_str = f"{int((rate - 1.0) * 100):+d}%"

            # 重试机制（最多 3 次）
            max_retries = 3
            for attempt in range(max_retries):
                try:
                    # 创建 Communicate 对象
                    communicate = edge_tts.Communicate(
                        text=text,
                        voice=voice_id,
                        rate=rate_str
                    )

                    # 保存音频
                    await communicate.save(output_file)

                    logger.info(f"Edge TTS 合成完成: {voice_id}, 语速:
{rate_str}")
                    return True

                except Exception as e:
                    if attempt < max_retries - 1:
                        logger.warning(f"Edge TTS 合成失败（尝试 {attempt
+ 1}/{max_retries}），重试中...")
                        await asyncio.sleep(1)  # 等待 1 秒后重试
                    else:
                        raise e

            return False

        except Exception as e:
            error_msg = str(e)
            logger.error(f"Edge TTS 合成失败: {error_msg}")

            # 提供详细的错误说明
```

```
            if "403" in error_msg or "Invalid response status" in
error_msg:
                logger.error("Edge TTS 403 错误，可能的原因：")
                logger.error("1. 微软服务器暂时限制访问")
                logger.error("2. edge-tts 版本过旧，请尝试：pip install
edge-tts --upgrade")
                logger.error("3. 网络连接问题")
                logger.error("解决方案：")
                logger.error("- 等待几分钟后重试")
                logger.error("- 更新 edge-tts: pip install edge-tts --
upgrade")
                logger.error("- 或切换到阿里云 TTS（需要 API 密钥）")

            return False

    async def _synthesize_ali(
        self,
        text: str,
        output_file: str,
        voice: str,
        emotion: str,
        rate: float
    ) -> bool:
        """阿里云 TTS 合成"""
        try:
            # TODO: 实现阿里云 TTS
            logger.warning("阿里云 TTS 暂未实现，请使用 Edge TTS")
            return False

        except Exception as e:
            logger.error(f"阿里云 TTS 合成失败: {e}")
            return False

    def _generate_cache_key(
        self,
        text: str,
        voice: str,
        emotion: str,
        rate: float
    ) -> str:
        """生成缓存 key"""
        # 使用 MD5 哈希
        content = f"{text}_{voice}_{emotion}_{rate}_{self.engine}"
        hash_obj = hashlib.md5(content.encode('utf-8'))
        return hash_obj.hexdigest()

    def clear_cache(self, older_than_days: int = 7):
```

```
    """
    清理缓存

    Args:
        older_than_days: 清理多少天前的缓存
    """
    try:
        current_time = time.time()
        count = 0

        for file in self.cache_dir.glob("*.mp3"):
            file_time = file.stat().st_mtime
            age_days = (current_time - file_time) / (24 * 3600)

            if age_days > older_than_days:
                file.unlink()
                count += 1

        logger.info(f"✓ 清理了 {count} 个旧缓存文件")

    except Exception as e:
        logger.error(f"清理缓存失败: {e}")

def get_cache_size(self) -> Tuple[int, str]:
    """
    获取缓存大小

    Returns:
        (文件数量, 总大小字符串)
    """
    try:
        files = list(self.cache_dir.glob("*.mp3"))
        count = len(files)

        total_size = sum(f.stat().st_size for f in files)

        # 格式化大小
        if total_size < 1024:
            size_str = f"{total_size} B"
        elif total_size < 1024 * 1024:
            size_str = f"{total_size / 1024:.2f} KB"
        else:
            size_str = f"{total_size / (1024 * 1024):.2f} MB"

        return (count, size_str)

    except Exception as e:
        logger.error(f"获取缓存大小失败: {e}")
```

```
            return (0, "0 B")


# 单例模式
_tts_instance: Optional[TTSHandler] = None


def get_tts_handler(engine: str = "edge", config: dict = None) ->
TTSHandler:
    """获取 TTS 处理器实例（单例）"""
    global _tts_instance

    if _tts_instance is None:
        _tts_instance = TTSHandler(engine=engine, config=config)

    return _tts_instance
```

代码行数：287 行

19. src/voice/voice_qa_handler.py

功能描述：语音问答处理器，完整的语音交互流程

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""
完整的语音问答处理器
整合 ASR、RAG 问答、情感分析、TTS 为一体
"""

import asyncio
import time
from typing import Optional, Dict, Any, AsyncGenerator
from pathlib import Path
from src.utils.logger import get_logger

logger = get_logger(__name__)


class VoiceQAHandler:
    """语音问答处理器"""

    def __init__(
        self,
        qa_chain,
        asr_handler=None,
        tts_handler=None,
        emotion_analyzer=None
    ):
        """
        初始化语音问答处理器

        Args:
            qa_chain: 问答链实例
            asr_handler: ASR 处理器
            tts_handler: TTS 处理器
            emotion_analyzer: 情感分析器
        """
        self.qa_chain = qa_chain
        self.asr = asr_handler
        self.tts = tts_handler
        self.emotion_analyzer = emotion_analyzer

        logger.info("语音问答处理器初始化完成")

    async def process_voice_question(
        self,
        audio_data: bytes,
```

```
        user_id: str,
        session_id: Optional[str] = None,
        audio_format: str = 'wav'
    ) -> Dict[str, Any]:
        """
        处理语音问题的完整流程

        Args:
            audio_data: 音频数据（bytes 或 base64）
            user_id: 用户 ID
            session_id: 会话 ID
            audio_format: 音频格式

        Returns:
            {
                'question_text': '识别的问题文本',
                'answer_text': '回答文本',
                'audio_url': '音频文件 URL',
                'audio_file': '音频文件路径',
                'emotion': '情感',
                'sources': [...],
                'duration': 处理时长
            }
        """
        start_time = time.time()

        try:
            logger.info(f"  [语音问答] 开始处理 (用户: {user_id}, 会话:
{session_id})")

            # 1. ASR 识别
            logger.info("步骤 1/4: ASR 语音识别...")
            if not self.asr:
                raise ValueError("ASR 处理器未初始化")

            question_text = self.asr.recognize(
                audio_data=audio_data,
                format=audio_format
            )

            if not question_text:
                raise ValueError("语音识别失败，未识别到内容")

            logger.info(f"✓ ASR 识别: {question_text}")

            # 2. RAG 问答处理（带记忆）
```

```
            logger.info("步骤 2/4: RAG 问答处理...")
            qa_result = self.qa_chain.answer(
                question=question_text,
                session_id=session_id,
                return_sources=True,
                return_related=False  # 语音模式下不需要相关问题
            )

            answer_text = qa_result.get('answer', '')
            sources = qa_result.get('sources', [])

            logger.info(f"✓ 问答完成: {len(answer_text)} 字")

            # 3. 情感分析
            logger.info("步骤 3/4: 情感分析...")
            emotion = self._analyze_emotion(answer_text)
            logger.info(f"✓ 情感: {emotion}")

            # 4. TTS 合成
            logger.info("步骤 4/4: TTS 语音合成...")
            if not self.tts:
                raise ValueError("TTS 处理器未初始化")

            # 根据情感调整语音参数
            voice, emotion_style, speaking_rate =
self._get_voice_params(emotion)

            success, audio_file = await self.tts.synthesize(
                text=answer_text,
                voice=voice,
                emotion=emotion_style,
                rate=speaking_rate,
                use_cache=True
            )

            if not success or not audio_file:
                raise ValueError("TTS 合成失败")

            logger.info(f"✓ TTS 合成: {Path(audio_file).name}")

            # 5. 构建返回结果
            duration = time.time() - start_time

            audio_filename = Path(audio_file).name
            audio_url = f"/api/voice/audio/{audio_filename}"

            result = {
```

```
            'question_text': question_text,
            'answer_text': answer_text,
            'audio_url': audio_url,
            'audio_file': audio_file,
            'emotion': emotion,
            'voice': voice,
            'sources': sources,
            'duration': round(duration, 2),
            'session_id': session_id
        }

        logger.info(f"✅ [语音问答] 处理完成 (耗时: {duration:.2f}s)")

        return result

    except Exception as e:
        logger.error(f"❌ [语音问答] 处理失败: {e}")
        raise

async def process_voice_question_stream(
    self,
    audio_data: bytes,
    user_id: str,
    session_id: Optional[str] = None,
    audio_format: str = 'wav'
) -> AsyncGenerator[Dict[str, Any], None]:
    """
    流式处理语音问题

    Yields:
        各个阶段的事件消息
    """
    try:
        logger.info(f"🎤 [流式语音问答] 开始处理 (用户: {user_id})")

        # 1. ASR 识别
        yield {
            'type': 'thinking',
            'data': {
                'status': 'asr',
                'message': '正在识别语音...'
            }
        }

        if not self.asr:
            raise ValueError("ASR 处理器未初始化")

        question_text = self.asr.recognize(
```

```
    audio_data=audio_data,
    format=audio_format
)

if not question_text:
    raise ValueError("语音识别失败")

# 返回识别结果
yield {
    'type': 'asr_result',
    'data': {
        'text': question_text,
        'confidence': 1.0
    }
}

logger.info(f"✓ ASR 识别: {question_text}")

# 2. 流式 RAG 问答
full_answer = ""
sources = []

for event in self.qa_chain.answer_stream(
    question=question_text,
    session_id=session_id,
    return_sources=True,
    return_related=False
):
    # 转发事件
    yield event

    # 收集完整答案和来源
    if event.get('type') == 'content':
        full_answer += event.get('data', '')
    elif event.get('type') == 'sources':
        sources = event.get('data', [])

    # 异步让出控制权
    await asyncio.sleep(0)

# 3. 情感分析
yield {
    'type': 'thinking',
    'data': {
        'status': 'emotion',
        'message': '正在分析情感...'
    }
}
```

```
            emotion = self._analyze_emotion(full_answer)

            yield {
                'type': 'emotion',
                'data': {
                    'emotion': emotion,
                    'text_preview': full_answer[:100]
                }
            }

            # 4. TTS 合成
            yield {
                'type': 'thinking',
                'data': {
                    'status': 'tts',
                    'message': '正在合成语音...'
                }
            }

            if not self.tts:
                logger.warning("TTS 处理器未初始化，跳过语音合成")
            else:
                # 根据情感调整语音参数
                voice, emotion_style, speaking_rate =
self._get_voice_params(emotion)

                success, audio_file = await self.tts.synthesize(
                    text=full_answer,
                    voice=voice,
                    emotion=emotion_style,
                    rate=speaking_rate,
                    use_cache=True
                )

                if success and audio_file:
                    audio_filename = Path(audio_file).name
                    audio_url = f"/api/voice/audio/{audio_filename}"

                    yield {
                        'type': 'voice_output',
                        'data': {
                            'audio_url': audio_url,
                            'audio_file': audio_file,
                            'text': full_answer,
                            'emotion': emotion,
                            'voice': voice
                        }
                    }
```

```
                    logger.info(f"✓ TTS 合成: {audio_filename}")

            # 5. 完成
            yield {
                'type': 'done',
                'data': {
                    'question': question_text,
                    'answer': full_answer,
                    'emotion': emotion,
                    'sources_count': len(sources)
                }
            }

            logger.info(f"✓ [流式语音问答] 处理完成")

        except Exception as e:
            logger.error(f"✗ [流式语音问答] 处理失败: {e}")
            yield {
                'type': 'error',
                'data': {'message': str(e)}
            }

    def _analyze_emotion(self, text: str) -> str:
        """
        分析文本情感

        Args:
            text: 文本内容

        Returns:
            情感标签
        """
        if self.emotion_analyzer:
            try:
                result = self.emotion_analyzer.analyze(text)
                return result.get('emotion', 'neutral')
            except Exception as e:
                logger.warning(f"情感分析失败: {e}")

        # 简单的关键词匹配（备用）
        return self._simple_emotion_analysis(text)

    def _simple_emotion_analysis(self, text: str) -> str:
        """简单的情感分析（关键词匹配）"""
        positive_keywords = ['很好', '不错', '优秀', '成功', '完成', '太棒了
', '很高兴', '开心', '喜欢']
```

```
        negative_keywords = ['抱歉', '遗憾', '失败', '错误', '问题', '困难',
'难过', '糟糕']
        question_keywords = ['吗', '呢', '？', '什么', '如何', '怎么']

        text_lower = text.lower()

        positive_count = sum(1 for kw in positive_keywords if kw in
text_lower)
        negative_count = sum(1 for kw in negative_keywords if kw in
text_lower)
        question_count = sum(1 for kw in question_keywords if kw in
text_lower)

        if positive_count > negative_count:
            return 'happy'
        elif negative_count > positive_count:
            return 'sad'
        elif question_count > 2:
            return 'curious'
        else:
            return 'neutral'

    def _get_voice_params(self, emotion: str) -> tuple:
        """
        根据情感获取语音参数

        Args:
            emotion: 情感

        Returns:
            (voice, emotion_style, speaking_rate)
        """
        # 情感到语音参数的映射
        emotion_mapping = {
            'happy': ('female', 'cheerful', 1.1),      # 欢快、稍快
            'sad': ('female', 'gentle', 0.9),          # 温柔、稍慢
            'angry': ('female', 'serious', 1.15),      # 严肃、较快
            'surprised': ('female', 'cheerful', 1.2),  # 欢快、快
            'neutral': ('female', 'friendly', 1.0),    # 友好、正常
            'curious': ('female', 'chat', 1.05),       # 聊天、稍快
            'thinking': ('female', 'chat', 0.95)       # 聊天、稍慢
        }

        return emotion_mapping.get(emotion, ('female', 'friendly', 1.0))


# 单例模式
```

```
_voice_qa_instance: Optional[VoiceQAHandler] = None


def get_voice_qa_handler(
    qa_chain,
    asr_handler=None,
    tts_handler=None,
    emotion_analyzer=None
) -> VoiceQAHandler:
    """获取语音问答处理器实例"""
    global _voice_qa_instance

    if _voice_qa_instance is None:
        _voice_qa_instance = VoiceQAHandler(
            qa_chain=qa_chain,
            asr_handler=asr_handler,
            tts_handler=tts_handler,
            emotion_analyzer=emotion_analyzer
        )

    return _voice_qa_instance
```

代码行数：391 行

20. src/voice/__init__.py

功能描述：语音模块初始化文件

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""
语音交互模块
包含 ASR（语音识别）、TTS（语音合成）和完整的语音问答流程
"""

from .asr_handler import ASRHandler, get_asr_handler
from .tts_handler import TTSHandler, get_tts_handler
from .voice_qa_handler import VoiceQAHandler, get_voice_qa_handler

__all__ = [
    'ASRHandler',
    'TTSHandler',
    'VoiceQAHandler',
    'get_asr_handler',
    'get_tts_handler',
    'get_voice_qa_handler'
]
```

代码行数：20 行

4.2 第六部分：情感分析模块

21. src/emotion/emotion_analyzer.py

功能描述：情感分析器，识别文本情感并生成控制参数

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""
情感分析器
支持多种情感分析方法，并提供数字人表情映射
"""

from typing import Dict, Any, Optional, List
from src.utils.logger import get_logger

logger = get_logger(__name__)


class EmotionAnalyzer:
    """情感分析器 - 数字人增强版"""

    def __init__(self, method: str = "keywords"):
        """
        初始化情感分析器

        Args:
            method: 分析方法
                - keywords: 关键词匹配（快速，轻量）
                - model: 深度学习模型（准确，需要模型）
        """
        self.method = method

        # 情感关键词库
        self.emotion_keywords = {
            'happy': [
                '开心', '高兴', '快乐', '愉快', '喜欢', '太好了', '很好', '不
错',
                '优秀', '成功', '完美', '棒', '赞', '哈哈', '呵呵', '笑', '
满意'
            ],
            'sad': [
                '难过', '伤心', '悲伤', '遗憾', '可惜', '抱歉', '对不起', '失
望',
                '沮丧', '痛苦', '哭', '失败', '糟糕', '不好', '担心', '忧虑'
            ],
            'angry': [
```

```
                '生气', '愤怒', '气愤', '恼火', '烦', '讨厌', '可恶', '混蛋',
                '该死', '垃圾', '糟糕透了', '受够了', '烦死了'
            ],
            'surprised': [
                '惊讶', '吃惊', '震惊', '意外', '没想到', '竟然', '居然', '天
啊',
                '哇', '不可思议', '太神奇了', '真的吗', '怎么可能'
            ],
            'fearful': [
                '害怕', '恐惧', '担心', '紧张', '焦虑', '不安', '忐忑', '慌',
                '吓', '可怕', '危险', '风险'
            ],
            'disgusted': [
                '恶心', '厌恶', '反感', '讨厌', '难受', '不舒服'
            ],
            'curious': [
                '什么', '为什么', '怎么', '如何', '哪里', '谁', '吗', '呢',
                '能否', '可以吗', '请问', '请教', '想知道', '好奇'
            ],
            'thinking': [
                '让我想想', '考虑', '思考', '分析', '研究', '理解', '根据',
                '可能', '大概', '估计', '应该'
            ]
        }

        # 初始化模型（如果需要）
        if method == "model":
            self._init_model()

        logger.info(f"✓ 情感分析器初始化完成: {method}")

    def _init_model(self):
        """初始化深度学习模型（可选）"""
        # TODO: 加载预训练的情感分析模型
        # 例如：使用 transformers 库加载情感分类模型
        pass

    def analyze(self, text: str) -> Dict[str, Any]:
        """
        分析文本情感

        Args:
            text: 文本内容

        Returns:
            {
```

```
                'emotion': '情感标签',
                'score': 0.85,  # 置信度
                'details': {...}  # 详细分数
            }
        """
        if self.method == "keywords":
            return self._analyze_by_keywords(text)
        elif self.method == "model":
            return self._analyze_by_model(text)
        else:
            return {'emotion': 'neutral', 'score': 0.5}

    def _analyze_by_keywords(self, text: str) -> Dict[str, Any]:
        """基于关键词的情感分析"""
        text_lower = text.lower()

        # 统计各情感的关键词数量
        emotion_scores = {}
        for emotion, keywords in self.emotion_keywords.items():
            count = sum(1 for kw in keywords if kw in text_lower)
            if count > 0:
                emotion_scores[emotion] = count

        # 如果没有匹配，返回中性
        if not emotion_scores:
            return {
                'emotion': 'neutral',
                'score': 1.0,
                'details': {'neutral': 1.0}
            }

        # 找出得分最高的情感
        primary_emotion = max(emotion_scores.items(), key=lambda x:
x[1])
        total_count = sum(emotion_scores.values())

        # 计算置信度
        confidence = primary_emotion[1] / total_count if total_count >
0 else 0.5

        # 归一化详细分数
        details = {
            emotion: count / total_count
            for emotion, count in emotion_scores.items()
        }

        return {
            'emotion': primary_emotion[0],
```

```
            'score': min(confidence, 1.0),
            'details': details
        }

    def _analyze_by_model(self, text: str) -> Dict[str, Any]:
        """基于深度学习模型的情感分析"""
        # TODO: 实现基于模型的分析
        logger.warning("模型分析暂未实现，使用关键词分析")
        return self._analyze_by_keywords(text)

    def analyze_for_digital_human(self, text: str) -> Dict[str, Any]:
        """
        为数字人分析情感（包含表情和语音参数）

        Args:
            text: 文本内容

        Returns:
            {
                'emotion': '情感标签',
                'score': 0.85,
                'expression_data': {
                    'expression': 'smile',
                    'intensity': 0.8,
                    'duration': 2.0,
                    'blend_shapes': {...}
                },
                'voice_params': {
                    'style': 'cheerful',
                    'rate': 1.1
                }
            }
        """
        # 基础情感分析
        emotion_result = self.analyze(text)
        emotion = emotion_result['emotion']
        score = emotion_result['score']

        # 获取表情数据
        expression_data = self._map_emotion_to_expression(emotion,
score)

        # 获取语音参数
        voice_params = self._map_emotion_to_voice(emotion, score)

        return {
            'emotion': emotion,
            'score': score,
```

```
            'expression_data': expression_data,
            'voice_params': voice_params,
            'details': emotion_result.get('details', {})
        }

    def _map_emotion_to_expression(
        self,
        emotion: str,
        score: float
    ) -> Dict[str, Any]:
        """
        将情感映射到数字人表情

        Returns:
            表情数据（Blend Shapes）
        """
        # 情感到表情的映射表（参考 Fay 和数字人集成路线图）
        expression_mapping = {
            'happy': {
                'expression': 'smile',
                'intensity': 0.8,
                'duration': 2.0,
                'blend_shapes': {
                    'mouthSmile_L': 0.8,
                    'mouthSmile_R': 0.8,
                    'eyeSquint_L': 0.3,
                    'eyeSquint_R': 0.3,
                    'cheekPuff': 0.2
                }
            },
            'sad': {
                'expression': 'sad',
                'intensity': 0.7,
                'duration': 2.5,
                'blend_shapes': {
                    'mouthFrown_L': 0.7,
                    'mouthFrown_R': 0.7,
                    'browDown_L': 0.5,
                    'browDown_R': 0.5,
                    'eyeClosed': 0.2
                }
            },
            'angry': {
                'expression': 'angry',
                'intensity': 0.9,
                'duration': 2.0,
                'blend_shapes': {
                    'browDown_L': 0.9,
                    'browDown_R': 0.9,
```

```
            'mouthPress_L': 0.7,
            'mouthPress_R': 0.7,
            'noseWrinkle_L': 0.5,
            'noseWrinkle_R': 0.5
        }
    },
    'surprised': {
        'expression': 'surprised',
        'intensity': 0.9,
        'duration': 1.5,
        'blend_shapes': {
            'eyeWide_L': 0.9,
            'eyeWide_R': 0.9,
            'mouthOpen': 0.6,
            'browUp_L': 0.7,
            'browUp_R': 0.7,
            'jawOpen': 0.3
        }
    },
    'fearful': {
        'expression': 'fearful',
        'intensity': 0.8,
        'duration': 2.0,
        'blend_shapes': {
            'eyeWide_L': 0.8,
            'eyeWide_R': 0.8,
            'browUp_L': 0.6,
            'browUp_R': 0.6,
            'mouthOpen': 0.3
        }
    },
    'curious': {
        'expression': 'thinking',
        'intensity': 0.6,
        'duration': 1.5,
        'blend_shapes': {
            'eyeSquint_L': 0.4,
            'browDown_L': 0.3,
            'browUp_R': 0.2,
            'mouthSide': 0.2
        }
    },
    'thinking': {
        'expression': 'thinking',
        'intensity': 0.5,
        'duration': 1.0,
        'blend_shapes': {
            'eyeSquint_L': 0.4,
            'eyeSquint_R': 0.3,
```

```
                    'browDown_L': 0.3,
                    'mouthSide': 0.2,
                    'eyeLookUp_L': 0.2,
                    'eyeLookUp_R': 0.2
                }
            },
            'neutral': {
                'expression': 'neutral',
                'intensity': 0.3,
                'duration': 1.0,
                'blend_shapes': {
                    'mouthSmile_L': 0.1,
                    'mouthSmile_R': 0.1
                }
            }
        }

        # 获取基础表情数据
        expression = expression_mapping.get(emotion,
expression_mapping['neutral'])

        # 根据置信度调整强度
        adjusted_expression = expression.copy()
        adjusted_expression['intensity'] = expression['intensity'] *
score

        # 调整 blend shapes 权重
        adjusted_blend_shapes = {
            key: value * score
            for key, value in expression['blend_shapes'].items()
        }
        adjusted_expression['blend_shapes'] = adjusted_blend_shapes

        return adjusted_expression

    def _map_emotion_to_voice(
        self,
        emotion: str,
        score: float
    ) -> Dict[str, Any]:
        """
        将情感映射到语音参数

        Returns:
            语音参数
        """
        voice_mapping = {
            'happy': {'style': 'cheerful', 'rate': 1.1},
            'sad': {'style': 'gentle', 'rate': 0.9},
```

```
            'angry': {'style': 'serious', 'rate': 1.15},
            'surprised': {'style': 'cheerful', 'rate': 1.2},
            'fearful': {'style': 'gentle', 'rate': 0.95},
            'curious': {'style': 'chat', 'rate': 1.05},
            'thinking': {'style': 'chat', 'rate': 0.95},
            'neutral': {'style': 'friendly', 'rate': 1.0}
        }

        params = voice_mapping.get(emotion, voice_mapping['neutral'])

        # 根据置信度微调
        if score < 0.5:
            # 置信度低，趋向中性
            params['rate'] = 1.0 + (params['rate'] - 1.0) * score

        return params

    def get_emotion_icon(self, emotion: str) -> str:
        """获取情感对应的 Emoji 图标"""
        icons = {
            'happy': ' ',
            'sad': ' ',
            'angry': ' ',
            'surprised': ' ',
            'fearful': ' ',
            'disgusted': ' ',
            'curious': ' ',
            'thinking': ' ',
            'neutral': ' '
        }
        return icons.get(emotion, ' ')


# 单例模式
_emotion_analyzer: Optional[EmotionAnalyzer] = None


def get_emotion_analyzer(method: str = "keywords") -> EmotionAnalyzer:
    """获取情感分析器实例（单例）"""
    global _emotion_analyzer

    if _emotion_analyzer is None:
        _emotion_analyzer = EmotionAnalyzer(method=method)

    return _emotion_analyzer
```

代码行数：367 行

22. src/emotion/__init__.py

功能描述：情感模块初始化文件

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""
情感分析模块
"""

from .emotion_analyzer import EmotionAnalyzer, get_emotion_analyzer

__all__ = ['EmotionAnalyzer', 'get_emotion_analyzer']
```

代码行数：10 行

4.2 第七部分：数字人控制

23. src/digital_human/websocket_server.py

功能描述：WebSocket 服务器，实现实时双向通信

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""
数字人 WebSocket 服务器
参考 Fay 的设计，结合 RAG 问答系统的优势
"""

import asyncio
import websockets
import json
import time
from typing import Set, Dict, Any, Callable, Optional
from src.utils.logger import get_logger

logger = get_logger(__name__)


class DigitalHumanServer:
    """数字人 WebSocket 服务器"""

    def __init__(
        self,
        host: str = "0.0.0.0",
        port: int = 10003,
        qa_handler: Optional[Callable] = None,
        enable_voice_output: bool = True
    ):
        """
        初始化服务器

        Args:
            host: 监听地址
            port: 监听端口（建议 10003，兼容 Fay）
            qa_handler: 问答处理器（异步生成器，用于流式输出）
            enable_voice_output: 是否启用语音输出（TTS）
        """
        self.host = host
        self.port = port
        self.qa_handler = qa_handler
        self.enable_voice_output = enable_voice_output

        # 连接管理（参考 Fay 的设计）
```

```
        self.lock = asyncio.Lock()
        self.clients: Set[websockets.WebSocketServerProtocol] = set()
        self.client_info: Dict[websockets.WebSocketServerProtocol, dict]
= {}
        self.cmd_queue: list = []  # 消息队列
        self.__running = True

        # TTS 处理器（延迟初始化）
        self.tts = None
        # 情感分析器（延迟初始化）
        self.emotion_analyzer = None
        # 表情控制器（延迟初始化）
        self.expression_controller = None
        # 唇形生成器（延迟初始化）
        self.lipsync_generator = None

        logger.info(f"数字人服务器初始化: {host}:{port}")

    async def start(self):
        """启动服务器"""
        try:
            async with websockets.serve(
                self.handle_client,
                self.host,
                self.port,
                ping_interval=20,
                ping_timeout=10
            ):
                logger.info(f"✓ 数字人 WebSocket 服务器已启动:
ws://{self.host}:{self.port}")
                logger.info(f"  兼容 Fay 协议，支持 RAG 问答、流式输出、对话记忆
")
                await asyncio.Future()  # 永久运行
        except Exception as e:
            logger.error(f"服务器启动失败: {e}")
            raise

    async def handle_client(
        self,
        websocket: websockets.WebSocketServerProtocol,
        path: str
    ):
        """
        处理客户端连接
        参考 Fay 的__handler 方法
        """
        client_id = id(websocket)
```

```
        remote_address = websocket.remote_address

        # 注册客户端
        async with self.lock:
            self.clients.add(websocket)
            self.client_info[websocket] = {
                'id': client_id,
                'address': remote_address,
                'username': 'User',
                'connected_at': time.time()
            }

        logger.info(f"  数字人客户端连接: {remote_address} (ID:
{client_id})")

        try:
            # 发送欢迎消息
            await self.send_message(websocket, {
                'type': 'welcome',
                'data': {
                    'server': 'Intelligent QA Digital Human',
                    'version': '2.0.0',
                    'features': ['RAG 检索', '对话记忆', '流式输出', '智能回
退', '情感分析'],
                    'client_id': client_id,
                    'timestamp': time.time()
                }
            })

            # 创建消息接收和发送任务（参考 Fay 的双任务模式）
            consumer_task = asyncio.create_task(
                self.consumer_handler(websocket, path)
            )
            producer_task = asyncio.create_task(
                self.producer_handler(websocket, path)
            )

            # 等待任何一个任务完成
            done, pending = await asyncio.wait(
                [consumer_task, producer_task],
                return_when=asyncio.FIRST_COMPLETED
            )

            # 取消未完成的任务
            for task in pending:
                task.cancel()
                try:
                    await task
```

```
                except asyncio.CancelledError:
                    pass

        except websockets.exceptions.ConnectionClosed:
            logger.info(f"✕ 客户端断开连接: {remote_address}")
        except Exception as e:
            logger.error(f"处理客户端连接时出错: {e}")

        finally:
            # 清理
            await self.remove_client(websocket)

    async def consumer_handler(
        self,
        websocket: websockets.WebSocketServerProtocol,
        path: str
    ):
        """接收消息处理器"""
        try:
            async for message in websocket:
                await asyncio.sleep(0.01)
                try:
                    data = json.loads(message)
                    await self.handle_message(websocket, data)
                except json.JSONDecodeError as e:
                    logger.error(f"JSON 解析错误: {e}")
                    await self.send_error(websocket, "Invalid JSON
format")
                except Exception as e:
                    logger.error(f"消息处理错误: {e}")
                    await self.send_error(websocket, str(e))
        except websockets.exceptions.ConnectionClosedError:
            pass
        except Exception as e:
            logger.error(f"接收处理器错误: {e}")

    async def producer_handler(
        self,
        websocket: websockets.WebSocketServerProtocol,
        path: str
    ):
        """发送消息处理器（参考 Fay 的 producer_handler）"""
        while self.__running:
            await asyncio.sleep(0.01)

            # 检查是否有待发送的消息
            if len(self.cmd_queue) > 0:
                async with self.lock:
```

```
                    if len(self.cmd_queue) > 0:
                        message = self.cmd_queue.pop(0)

                        # 判断是广播还是单播
                        username = message.get('Username')
                        if username is None:
                            # 广播到所有客户端
                            tasks = [
                                self.send_message_with_timeout(client,
message, timeout=3)
                                for client in self.clients
                            ]
                            await asyncio.gather(*tasks,
return_exceptions=True)
                        else:
                            # 单播给指定用户
                            target_clients = [
                                client for client in self.clients
                                if self.client_info.get(client,
{}).get('username') == username
                            ]
                            tasks = [
                                self.send_message_with_timeout(client,
message, timeout=3)
                                for client in target_clients
                            ]
                            await asyncio.gather(*tasks,
return_exceptions=True)

    async def send_message_with_timeout(
        self,
        client: websockets.WebSocketServerProtocol,
        message: dict,
        timeout: int = 3
    ):
        """带超时的消息发送"""
        try:
            await asyncio.wait_for(
                self.send_message(client, message),
                timeout=timeout
            )
        except asyncio.TimeoutError:
            username = self.client_info.get(client, {}).get('username',
'Unknown')
            logger.warning(f"发送消息超时: {username}")
        except Exception as e:
            logger.error(f"发送消息失败: {e}")
```

```
    async def handle_message(
        self,
        websocket: websockets.WebSocketServerProtocol,
        data: dict
    ):
        """处理收到的消息"""
        msg_type = data.get('type', '')
        msg_data = data.get('data', {})

        logger.debug(f"收到消息类型: {msg_type}")

        # 更新用户名（兼容 Fay 的 Username 字段）
        if 'Username' in data:
            async with self.lock:
                if websocket in self.client_info:
                    self.client_info[websocket]['username'] =
data['Username']
                    logger.debug(f"更新用户名: {data['Username']}")

        # 根据消息类型分发
        if msg_type == 'text_input':
            await self.handle_text_input(websocket, msg_data)

        elif msg_type == 'voice_input':
            await self.handle_voice_input(websocket, msg_data)

        elif msg_type == 'interrupt':
            await self.handle_interrupt(websocket)

        elif msg_type == 'ready':
            await self.handle_ready(websocket, msg_data)

        elif msg_type == 'ping':
            # 心跳响应
            await self.send_message(websocket, {
                'type': 'pong',
                'data': {'timestamp': time.time()}
            })

        else:
            logger.warning(f"未知消息类型: {msg_type}")

    async def handle_text_input(
        self,
        websocket: websockets.WebSocketServerProtocol,
        data: dict
    ):
        """
```

```
        处理文本输入

        这里是核心：将用户问题传给 RAG 系统，流式返回结果
        """
        question = data.get('text', '').strip()
        user_id = data.get('user_id') or
self.client_info[websocket].get('username', 'default')
        session_id = data.get('session_id')

        if not question:
            await self.send_error(websocket, "问题不能为空")
            return

        logger.info(f"  收到问题 (用户: {user_id}): {question}")

        # 发送"思考中"状态
        await self.send_message(websocket, {
            'type': 'thinking',
            'data': {
                'status': 'processing',
                'message': '正在思考您的问题...'
            }
        })

            # 调用问答处理器（流式处理）
        if self.qa_handler:
            try:
                full_answer = ""
                emotion = "neutral"

                async for event in self.qa_handler(question, user_id,
session_id):
                    # 转发每个事件到客户端
                    await self.send_message(websocket, event)

                    # 收集完整答案
                    if event.get('type') == 'content':
                        full_answer += event.get('data', '')
                    elif event.get('type') == 'emotion':
                        emotion = event.get('data', {}).get('emotion',
'neutral')

                # 问答完成后，如果启用了语音输出，自动生成 TTS
                if self.enable_voice_output and full_answer:
                    await self._generate_voice_output(
                        websocket,
                        full_answer,
                        emotion
```

```
                    )

            except Exception as e:
                logger.error(f"问答处理失败: {e}", exc_info=True)
                await self.send_error(websocket, f"问答处理失败:
{str(e)}")
        else:
            # 没有处理器，返回默认响应
            logger.warning("问答处理器未配置")
            await self.send_message(websocket, {
                'type': 'text_output',
                'data': {
                    'text': '问答处理器未配置，请检查服务器设置。',
                    'emotion': 'neutral'
                }
            })
            await self.send_message(websocket, {
                'type': 'done',
                'data': {}
            })

    async def handle_voice_input(
        self,
        websocket: websockets.WebSocketServerProtocol,
        data: dict
    ):
        """
        处理语音输入

        data: {
            'audio': 'base64 编码的音频数据' 或 '音频 URL',
            'format': 'wav/mp3/pcm',
            'user_id': '用户 ID',
            'session_id': '会话 ID'
        }
        """
        try:
            audio_data = data.get('audio')
            audio_format = data.get('format', 'wav')
            user_id = data.get('user_id') or
self.client_info[websocket].get('username', 'default')
            session_id = data.get('session_id')

            if not audio_data:
                await self.send_error(websocket, "音频数据不能为空")
                return

            logger.info(f"  收到语音输入 (用户: {user_id}, 格式:
```

```
{audio_format})")

            # 发送处理状态
            await self.send_message(websocket, {
                'type': 'thinking',
                'data': {
                    'status': 'asr',
                    'message': '正在识别语音...'
                }
            })

            # 导入 ASR 处理器
            from src.voice.asr_handler import get_asr_handler

            try:
                # 获取 ASR 实例
                asr = get_asr_handler(engine='funasr')

                # 识别语音
                text = asr.recognize(
                    audio_data=audio_data,
                    format=audio_format
                )

                if text:
                    logger.info(f"✓ ASR 识别结果: {text}")

                    # 发送识别结果
                    await self.send_message(websocket, {
                        'type': 'asr_result',
                        'data': {
                            'text': text,
                            'confidence': 1.0  # FunASR 暂不提供置信度
                        }
                    })

                    # 将识别的文本作为问题继续处理
                    await self.handle_text_input(websocket, {
                        'text': text,
                        'user_id': user_id,
                        'session_id': session_id
                    })
                else:
                    logger.warning("ASR 未识别到内容")
                    await self.send_error(websocket, "未识别到语音内容，请
重试")

            except ImportError as e:
```

```
                logger.error(f"ASR 模块未安装: {e}")
                await self.send_error(websocket,
                    "语音识别功能未安装，请先安装: pip install funasr")
            except Exception as e:
                logger.error(f"语音识别失败: {e}")
                await self.send_error(websocket, f"语音识别失败:
{str(e)}")

        except Exception as e:
            logger.error(f"处理语音输入失败: {e}")
            await self.send_error(websocket, str(e))

    async def handle_interrupt(
        self,
        websocket: websockets.WebSocketServerProtocol
    ):
        """处理打断指令"""
        user_id = self.client_info[websocket].get('username', 'Unknown')
        logger.info(f"收到打断指令 (用户: {user_id})")
        # TODO: 实现打断机制（停止当前生成）
        await self.send_message(websocket, {
            'type': 'interrupted',
            'data': {'message': '已中断当前回答'}
        })

    async def handle_ready(
        self,
        websocket: websockets.WebSocketServerProtocol,
        data: dict
    ):
        """处理客户端就绪通知"""
        user_id = self.client_info[websocket].get('username', 'Unknown')
        logger.info(f"客户端就绪 (用户: {user_id})")

        # 发送系统状态
        await self.send_message(websocket, {
            'type': 'system_status',
            'data': {
                'rag_enabled': True,
                'memory_enabled': True,
                'stream_enabled': True,
                'emotion_enabled': True,
                'voice_enabled': False,  # 第 2 阶段实现
                'lipsync_enabled': False  # 第 3 阶段实现
            }
        })
```

```
    async def send_message(
        self,
        websocket: websockets.WebSocketServerProtocol,
        message: dict
    ):
        """发送消息到客户端"""
        try:
            await websocket.send(json.dumps(message,
ensure_ascii=False))
        except websockets.exceptions.ConnectionClosed:
            logger.warning("连接已关闭，无法发送消息")
        except Exception as e:
            logger.error(f"发送消息失败: {e}")

    async def send_error(
        self,
        websocket: websockets.WebSocketServerProtocol,
        error: str
    ):
        """发送错误消息"""
        await self.send_message(websocket, {
            'type': 'error',
            'data': {'message': error}
        })

    async def _generate_voice_output(
        self,
        websocket: websockets.WebSocketServerProtocol,
        text: str,
        emotion: str = 'neutral'
    ):
        """
        生成完整的数字人输出（语音+表情+唇形）

        Args:
            websocket: WebSocket 连接
            text: 要合成的文本
            emotion: 情感
        """
        try:
            # 1. 情感分析（如果 emotion 是自动检测的）
            await self.send_message(websocket, {
                'type': 'thinking',
                'data': {
                    'status': 'emotion',
                    'message': '正在分析情感...'
                }
            })
```

```
            # 延迟导入情感分析器
            if self.emotion_analyzer is None:
                from src.emotion.emotion_analyzer import
get_emotion_analyzer
                self.emotion_analyzer = get_emotion_analyzer()

            # 获取完整的情感数据（包含表情和语音参数）
            emotion_data =
self.emotion_analyzer.analyze_for_digital_human(text)
            emotion = emotion_data['emotion']  # 可能更新 emotion

            # 2. 发送表情控制指令
            if self.expression_controller is None:
                from src.digital_human.expression_controller import
get_expression_controller
                self.expression_controller = get_expression_controller()

            expression_command =
self.expression_controller.create_expression_command(
                emotion=emotion,
                expression_data=emotion_data['expression_data']
            )

            await self.send_message(websocket, expression_command)
            logger.info(f"✓ 表情指令已发送: {emotion}")

            # 3. TTS 语音合成
            await self.send_message(websocket, {
                'type': 'thinking',
                'data': {
                    'status': 'tts',
                    'message': '正在生成语音...'
                }
            })

            # 延迟导入 TTS
            if self.tts is None:
                from src.voice.tts_handler import get_tts_handler
                self.tts = get_tts_handler(engine='edge')

            # 使用情感数据中的语音参数
            voice_params = emotion_data['voice_params']

            success, audio_file = await self.tts.synthesize(
                text=text,
                voice='female',  # 可以从配置读取
                emotion=voice_params['style'],
```

```
                rate=voice_params['rate'],
                use_cache=True
            )

            if not success or not audio_file:
                logger.warning("TTS 合成失败，仅返回文本")
                return

            logger.info(f"✓ TTS 合成成功")

            # 4. 生成唇形数据
            await self.send_message(websocket, {
                'type': 'thinking',
                'data': {
                    'status': 'lipsync',
                    'message': '正在生成唇形数据...'
                }
            })

            # 延迟导入唇形生成器
            if self.lipsync_generator is None:
                from src.digital_human.lipsync_generator import
get_lipsync_generator
                self.lipsync_generator = get_lipsync_generator()

            try:
                lipsync_data =
self.lipsync_generator.generate_with_emotion(
                    audio_file=audio_file,
                    text=text,
                    emotion=emotion
                )

                logger.info(f"✓ 唇形数据已生成:
{lipsync_data['frame_count']} 帧")
            except Exception as e:
                logger.warning(f"唇形生成失败: {e}, 将不包含唇形数据")
                lipsync_data = None

            # 5. 发送完整的语音输出（包含唇形）
            from pathlib import Path
            audio_filename = Path(audio_file).name
            audio_url = f"/api/voice/audio/{audio_filename}"

            output_data = {
                'audio_url': audio_url,
                'audio_file': audio_file,
                'text': text,
```

```
                'emotion': emotion,
                'voice': 'female',
                'expression': emotion_data['expression_data'],
                'emotion_score': emotion_data['score']
            }

            # 添加唇形数据
            if lipsync_data:
                output_data['lipsync'] = lipsync_data

            await self.send_message(websocket, {
                'type': 'voice_output',
                'data': output_data
            })

            logger.info(f"✓ 完整数字人输出已发送 (语音+表情+唇形)")

        except Exception as e:
            logger.error(f"生成数字人输出失败: {e}")
            # 不阻止流程，仅记录错误

    def _map_emotion_to_voice(self, emotion: str) -> tuple:
        """
        将情感映射到语音参数

        Returns:
            (voice, emotion_style, speaking_rate)
        """
        emotion_mapping = {
            'happy': ('female', 'cheerful', 1.1),
            'sad': ('female', 'gentle', 0.9),
            'angry': ('female', 'serious', 1.15),
            'surprised': ('female', 'cheerful', 1.2),
            'neutral': ('female', 'friendly', 1.0),
            'curious': ('female', 'chat', 1.05),
            'thinking': ('female', 'chat', 0.95)
        }

        return emotion_mapping.get(emotion, ('female', 'friendly', 1.0))

    async def remove_client(self, websocket:
websockets.WebSocketServerProtocol):
        """移除客户端连接"""
        async with self.lock:
            if websocket in self.clients:
                self.clients.remove(websocket)
            if websocket in self.client_info:
                user_info = self.client_info[websocket]
```

```
                logger.info(f"客户端已移除: {user_info.get('username',
'Unknown')}")
                del self.client_info[websocket]

    def add_cmd(self, content: dict):
        """
        添加待发送的消息到队列
        参考 Fay 的 add_cmd 方法
        """
        self.cmd_queue.append(content)

    async def broadcast(self, message: dict):
        """广播消息到所有客户端"""
        if self.clients:
            await asyncio.gather(
                *[self.send_message(client, message) for client in
self.clients],
                return_exceptions=True
            )

    def is_connected(self, username: Optional[str] = None) -> bool:
        """
        检查是否有客户端连接

        Args:
            username: 用户名（可选），如果指定则检查该用户是否在线

        Returns:
            是否连接
        """
        if username is None:
            return len(self.clients) > 0

        for client_info in self.client_info.values():
            if client_info.get('username') == username:
                return True
        return False

    def stop_server(self):
        """停止服务器"""
        logger.info("正在停止数字人 WebSocket 服务器...")
        self.__running = False


# 单例模式
_instance: Optional[DigitalHumanServer] = None
```

```
def get_instance() -> Optional[DigitalHumanServer]:
    """获取服务器实例"""
    return _instance


def set_instance(server: DigitalHumanServer):
    """设置服务器实例"""
    global _instance
    _instance = server
```

代码行数：690 行

24. src/digital_human/expression_controller.py

功能描述：表情控制器，生成数字人面部表情控制指令

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""
数字人表情控制器
将情感转换为数字人表情控制指令
"""

from typing import Dict, Any, List
import time
from src.utils.logger import get_logger

logger = get_logger(__name__)


class ExpressionController:
    """表情控制器"""

    def __init__(self):
        """初始化表情控制器"""

        # UE5 MetaHuman 标准 Blend Shapes 列表
        self.blend_shapes_list = [
            # 眉毛
            'browDown_L', 'browDown_R',
            'browUp_L', 'browUp_R',
            'browInnerUp', 'browOuterUp_L', 'browOuterUp_R',

            # 眼睛
            'eyeBlink_L', 'eyeBlink_R',
            'eyeSquint_L', 'eyeSquint_R',
            'eyeWide_L', 'eyeWide_R',
            'eyeClosed', 'eyeLookUp_L', 'eyeLookUp_R',
            'eyeLookDown_L', 'eyeLookDown_R',

            # 嘴巴
            'mouthSmile_L', 'mouthSmile_R',
            'mouthFrown_L', 'mouthFrown_R',
            'mouthOpen', 'mouthClose',
            'mouthPress_L', 'mouthPress_R',
            'mouthSide', 'mouthFunnel',
            'mouthPucker', 'mouthStretch_L', 'mouthStretch_R',

            # 脸颊
            'cheekPuff', 'cheekSquint_L', 'cheekSquint_R',
```

```
            # 鼻子
            'noseWrinkle_L', 'noseWrinkle_R',

            # 下巴
            'jawOpen', 'jawForward', 'jawLeft', 'jawRight'
        ]

        logger.info("表情控制器初始化完成")

    def create_expression_command(
        self,
        emotion: str,
        expression_data: Dict[str, Any]
    ) -> Dict[str, Any]:
        """
        创建表情控制指令

        Args:
            emotion: 情感标签
            expression_data: 表情数据（从 EmotionAnalyzer 获取）

        Returns:
            表情控制指令（发送给 UE5/Unity）
        """
        command = {
            'type': 'expression',
            'data': {
                'emotion': emotion,
                'expression': expression_data.get('expression',
'neutral'),
                'intensity': expression_data.get('intensity', 0.5),
                'duration': expression_data.get('duration', 1.0),
                'blend_shapes': expression_data.get('blend_shapes', {}),
                'timestamp': time.time()
            }
        }

        logger.debug(f"创建表情指令: {emotion} ->
{expression_data.get('expression')}")

        return command

    def interpolate_expressions(
        self,
        from_expression: Dict[str, Any],
        to_expression: Dict[str, Any],
        steps: int = 10
    ) -> List[Dict[str, Any]]:
```

```
        """
        在两个表情之间插值，生成平滑过渡

        Args:
            from_expression: 起始表情
            to_expression: 目标表情
            steps: 插值步数

        Returns:
            插值后的表情序列
        """
        result = []

        from_blend = from_expression.get('blend_shapes', {})
        to_blend = to_expression.get('blend_shapes', {})

        # 获取所有 blend shapes 的集合
        all_shapes = set(from_blend.keys()) | set(to_blend.keys())

        for i in range(steps + 1):
            t = i / steps  # 0.0 到 1.0

            # 插值 blend shapes
            interpolated_blend = {}
            for shape in all_shapes:
                from_val = from_blend.get(shape, 0.0)
                to_val = to_blend.get(shape, 0.0)
                interpolated_blend[shape] = from_val + (to_val -
from_val) * t

            # 插值其他参数
            from_intensity = from_expression.get('intensity', 0.5)
            to_intensity = to_expression.get('intensity', 0.5)

            result.append({
                'expression': to_expression.get('expression',
'neutral'),
                'intensity': from_intensity + (to_intensity -
from_intensity) * t,
                'blend_shapes': interpolated_blend,
                'step': i,
                'progress': t
            })

        return result

    def validate_blend_shapes(self, blend_shapes: Dict[str, float]) ->
Dict[str, float]:
```

```
        """
        验证和规范化 Blend Shapes 数据

        Args:
            blend_shapes: Blend shapes 字典

        Returns:
            验证后的 blend shapes（权重在 0-1 之间）
        """
        validated = {}

        for shape, weight in blend_shapes.items():
            # 检查是否是有效的 blend shape
            if shape in self.blend_shapes_list:
                # 限制权重在 0-1 之间
                validated[shape] = max(0.0, min(1.0, weight))
            else:
                logger.warning(f"未知的 blend shape: {shape}")

        return validated

    def get_neutral_expression(self) -> Dict[str, Any]:
        """获取中性表情"""
        return {
            'expression': 'neutral',
            'intensity': 0.3,
            'duration': 1.0,
            'blend_shapes': {
                'mouthSmile_L': 0.1,
                'mouthSmile_R': 0.1
            }
        }


# 单例模式
_expression_controller: Optional['ExpressionController'] = None


def get_expression_controller() -> ExpressionController:
    """获取表情控制器实例（单例）"""
    global _expression_controller

    if _expression_controller is None:
        _expression_controller = ExpressionController()

    return _expression_controller
```

代码行数：182 行

25. src/digital_human/lipsync_generator.py

功能描述：唇形同步生成器，生成语音同步的口型数据

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""
唇形同步生成器
根据音频生成 Viseme 口型数据，让数字人嘴型与语音同步
参考 Fay 的 OVR LipSync 实现
"""

from typing import List, Dict, Optional
import numpy as np
from pathlib import Path
from src.utils.logger import get_logger

logger = get_logger(__name__)


class LipSyncGenerator:
    """唇形同步生成器"""

    # Viseme 口型映射（国际音标到口型）
    VISEME_MAP = {
        'sil': 0,       # 静音
        'PP': 1,        # p, b, m (闭嘴音)
        'FF': 2,        # f, v (唇齿音)
        'TH': 3,        # th (齿间音)
        'DD': 4,        # t, d (齿龈音)
        'kk': 5,        # k, g (软腭音)
        'CH': 6,        # ch, j, sh (塞擦音)
        'SS': 7,        # s, z (齿龈擦音)
        'nn': 8,        # n, l (鼻音/边音)
        'RR': 9,        # r (卷舌音)
        'aa': 10,       # a (开口元音)
        'E': 11,        # e (半闭元音)
        'I': 12,        # i (闭元音)
        'O': 13,        # o (圆唇元音)
        'U': 14         # u (圆唇闭元音)
    }

    # 中文拼音到 Viseme 的映射
    PINYIN_TO_VISEME = {
        # 辅音
        'b': 'PP', 'p': 'PP', 'm': 'PP',
        'f': 'FF', 'v': 'FF',
```

```
        'd': 'DD', 't': 'DD',
        'n': 'nn', 'l': 'nn',
        'g': 'kk', 'k': 'kk', 'h': 'kk',
        'j': 'CH', 'q': 'CH', 'x': 'CH',
        'z': 'SS', 'c': 'SS', 's': 'SS',
        'zh': 'CH', 'ch': 'CH', 'sh': 'CH', 'r': 'RR',

        # 元音
        'a': 'aa', 'ai': 'aa', 'ao': 'aa', 'an': 'aa', 'ang': 'aa',
        'e': 'E', 'ei': 'E', 'en': 'E', 'eng': 'E', 'er': 'E',
        'i': 'I', 'ia': 'I', 'ie': 'I', 'iao': 'I', 'iu': 'I',
        'ian': 'I', 'in': 'I', 'ing': 'I', 'iang': 'I', 'iong': 'I',
        'o': 'O', 'ou': 'O', 'ong': 'O',
        'u': 'U', 'ua': 'U', 'uo': 'U', 'uai': 'U', 'ui': 'U',
        'uan': 'U', 'un': 'U', 'uang': 'U',
        'ü': 'U', 'üe': 'U', 'üan': 'U', 'ün': 'U'
    }

    def __init__(self):
        """初始化唇形生成器"""
        self.librosa = None
        self.soundfile = None

        logger.info("✓ 唇形同步生成器初始化完成")

    def _lazy_import_audio_libs(self):
        """延迟导入音频处理库"""
        if self.librosa is None:
            try:
                import librosa
                import soundfile
                self.librosa = librosa
                self.soundfile = soundfile
            except ImportError:
                logger.error("音频处理库未安装，请运行: pip install librosa
soundfile")
                raise ImportError(
                    "需要安装音频处理库：\n"
                    "pip install librosa soundfile"
                )

    def generate_from_audio(
        self,
        audio_file: str,
        text: Optional[str] = None,
        method: str = 'energy'
    ) -> List[Dict]:
        """
```

```
    从音频文件生成唇形数据

    Args:
        audio_file: 音频文件路径
        text: 对应文本（可选，用于改进精度）
        method: 生成方法
            - energy: 基于能量的简单方法（快速）
            - phoneme: 基于音素的高级方法（准确，需要文本）

    Returns:
        唇形数据列表
        [
            {'viseme': 'aa', 'timestamp': 0.0, 'weight': 0.8},
            {'viseme': 'E', 'timestamp': 0.1, 'weight': 0.9},
            ...
        ]
    """
    try:
        self._lazy_import_audio_libs()

        # 检查文件是否存在
        if not Path(audio_file).exists():
            logger.error(f"音频文件不存在: {audio_file}")
            return []

        logger.info(f"生成唇形数据: {audio_file} (方法: {method})")

        if method == 'energy':
            visemes = self._generate_by_energy(audio_file)
        elif method == 'phoneme' and text:
            visemes = self._generate_by_phoneme(audio_file, text)
        else:
            visemes = self._generate_by_energy(audio_file)

        # 平滑处理
        visemes = self.smooth_visemes(visemes, window_size=3)

        logger.info(f"✓ 生成唇形数据: {len(visemes)} 帧")

        return visemes

    except Exception as e:
        logger.error(f"唇形生成失败: {e}")
        return []

def _generate_by_energy(
    self,
```

```
    audio_file: str,
    frame_length: int = 1024,
    hop_length: int = 512
) -> List[Dict]:
    """
    基于音频能量的简单方法

    适合快速原型，效果一般但速度快
    """
    try:
        # 加载音频
        y, sr = self.librosa.load(audio_file, sr=16000)

        # 计算短时能量（RMS）
        energy = self.librosa.feature.rms(
            y=y,
            frame_length=frame_length,
            hop_length=hop_length
        )[0]

        # 计算 MFCC（频率特征）
        mfcc = self.librosa.feature.mfcc(
            y=y,
            sr=sr,
            n_mfcc=13,
            hop_length=hop_length
        )

        # 计算音高（pitch）
        pitches, magnitudes = self.librosa.piptrack(
            y=y,
            sr=sr,
            hop_length=hop_length
        )

        visemes = []
        times = self.librosa.frames_to_time(
            np.arange(len(energy)),
            sr=sr,
            hop_length=hop_length
        )

        for i, (t, e) in enumerate(zip(times, energy)):
            # 根据能量和频率特征估计口型
            if e < 0.01:
                viseme = 'sil'  # 静音
                weight = 0.0
            else:
```

```
                    # 使用 MFCC 和音高估计口型
                    mfcc_val = mfcc[0, i] if i < mfcc.shape[1] else 0
                    pitch_val = np.mean(pitches[:, i]) if i <
pitches.shape[1] else 0

                    # 简单规则：根据 MFCC 和音高判断口型
                    if pitch_val > 200:  # 高音
                        if mfcc_val > 10:
                            viseme = 'I'  # i 音
                        else:
                            viseme = 'E'  # e 音
                    elif pitch_val > 100:  # 中音
                        if mfcc_val > 5:
                            viseme = 'aa'  # a 音
                        else:
                            viseme = 'O'  # o 音
                    else:  # 低音或辅音
                        if e > 0.05:
                            viseme = 'aa'  # 开口
                        else:
                            viseme = 'PP'  # 闭口

                    weight = min(float(e * 10), 1.0)

                visemes.append({
                    'viseme': viseme,
                    'timestamp': float(t),
                    'weight': weight
                })

            return visemes

        except Exception as e:
            logger.error(f"基于能量的唇形生成失败: {e}")
            return []

    def _generate_by_phoneme(
        self,
        audio_file: str,
        text: str
    ) -> List[Dict]:
        """
        基于音素识别的高级方法

        需要文本内容，效果更好但更复杂
        """
        try:
```

```
            # TODO: 实现基于音素的精确唇形生成
            # 1. 使用 ASR 获取音素时间戳
            # 2. 将音素映射到 Viseme
            # 3. 生成精确的唇形数据

            # 现在先用中文分词简单实现
            logger.warning("基于音素的方法暂未完全实现，使用简化版本")

            # 加载音频获取时长
            y, sr = self.librosa.load(audio_file, sr=16000)
            duration = len(y) / sr

            # 简单地将文本平均分配到时间轴上
            if not text:
                return self._generate_by_energy(audio_file)

            # 使用 jieba 分词
            try:
                import jieba
                words = list(jieba.cut(text))
            except ImportError:
                logger.warning("jieba 未安装，使用字符级别")
                words = list(text)

            # 估算每个字的时长
            char_duration = duration / len(text) if len(text) > 0 else
0.1

            visemes = []
            current_time = 0.0

            for char in text:
                # 跳过标点符号
                if char in '，。！？；：、""''（）【】《》':
                    viseme = 'sil'
                    weight = 0.0
                else:
                    # 简单映射（可以改进）
                    viseme = self._char_to_viseme(char)
                    weight = 0.8

                visemes.append({
                    'viseme': viseme,
                    'timestamp': current_time,
                    'weight': weight
                })
```

```
                current_time += char_duration

            return visemes

        except Exception as e:
            logger.error(f"基于音素的唇形生成失败: {e}")
            # 降级到能量方法
            return self._generate_by_energy(audio_file)

    def _char_to_viseme(self, char: str) -> str:
        """
        简单地将中文字符映射到 Viseme

        这是一个简化实现，实际应用中需要更复杂的音素分析
        """
        # 元音字符（开口）
        if char in 'aāáǎà啊阿呀':
            return 'aa'
        elif char in 'eēéěè额诶':
            return 'E'
        elif char in 'iīíǐì衣一':
            return 'I'
        elif char in 'oōóǒò哦噢':
            return 'O'
        elif char in 'uūúǔù乌呜':
            return 'U'

        # 辅音开头（根据拼音首字母粗略判断）
        elif char in 'bpmf 波啵默佛':
            return 'PP'
        elif char in 'dtnl 的特呢勒':
            return 'DD'
        elif char in 'gkh 哥克喝':
            return 'kk'
        elif char in 'jqx 几期西':
            return 'CH'
        elif char in 'zcs 则词思':
            return 'SS'
        elif char in 'rw 日我':
            return 'RR'

        # 默认
        return 'aa'

    def smooth_visemes(
        self,
        visemes: List[Dict],
```

```
    window_size: int = 3
) -> List[Dict]:
    """
    平滑唇形数据，让过渡更自然

    Args:
        visemes: 原始唇形数据
        window_size: 平滑窗口大小

    Returns:
        平滑后的唇形数据
    """
    if len(visemes) < window_size:
        return visemes

    smoothed = []

    for i in range(len(visemes)):
        start = max(0, i - window_size // 2)
        end = min(len(visemes), i + window_size // 2 + 1)

        # 平均权重
        weights = [v['weight'] for v in visemes[start:end]]
        avg_weight = np.mean(weights)

        smoothed.append({
            'viseme': visemes[i]['viseme'],
            'timestamp': visemes[i]['timestamp'],
            'weight': float(avg_weight)
        })

    return smoothed

def generate_with_emotion(
    self,
    audio_file: str,
    text: Optional[str] = None,
    emotion: str = 'neutral'
) -> Dict[str, Any]:
    """
    生成带情感的唇形数据

    Args:
        audio_file: 音频文件
        text: 文本内容
        emotion: 情感

    Returns:
```

```
            完整的唇形数据包（包含 Viseme 和表情调整）
        """
        # 生成基础唇形数据
        visemes = self.generate_from_audio(audio_file, text)

        # 根据情感调整口型强度
        emotion_intensity_map = {
            'happy': 1.2,      # 开心时口型更夸张
            'sad': 0.8,        # 难过时口型收敛
            'angry': 1.3,      # 愤怒时口型强烈
            'surprised': 1.4,  # 惊讶时口型夸张
            'neutral': 1.0     # 中性
        }

        intensity_factor = emotion_intensity_map.get(emotion, 1.0)

        # 调整权重
        adjusted_visemes = []
        for v in visemes:
            adjusted_visemes.append({
                'viseme': v['viseme'],
                'timestamp': v['timestamp'],
                'weight': min(v['weight'] * intensity_factor, 1.0)
            })

        return {
            'visemes': adjusted_visemes,
            'emotion': emotion,
            'frame_count': len(adjusted_visemes),
            'duration': adjusted_visemes[-1]['timestamp'] if
adjusted_visemes else 0
        }

    def export_to_ue5_format(self, visemes: List[Dict]) -> List[Dict]:
        """
        导出为 UE5 格式的唇形数据

        Args:
            visemes: Viseme 数据列表

        Returns:
            UE5 格式的数据
        """
        ue5_data = []

        for v in visemes:
            ue5_data.append({
                'Time': v['timestamp'],
```

```
                'Viseme': self.VISEME_MAP.get(v['viseme'], 0),
                'VisemeName': v['viseme'],
                'Weight': v['weight']
            })

        return ue5_data

    def export_to_unity_format(self, visemes: List[Dict]) -> Dict[str,
Any]:
        """
        导出为 Unity 格式的唇形数据

        Args:
            visemes: Viseme 数据列表

        Returns:
            Unity 格式的数据
        """
        return {
            'clips': [{
                'time': v['timestamp'],
                'viseme': v['viseme'],
                'weight': v['weight']
            } for v in visemes],
            'duration': visemes[-1]['timestamp'] if visemes else 0
        }

    def get_viseme_blend_shapes(self, viseme: str, weight: float = 1.0)
-> Dict[str, float]:
        """
        将 Viseme 转换为 Blend Shapes 权重

        Args:
            viseme: Viseme ID
            weight: 权重

        Returns:
            Blend shapes 字典
        """
        # Viseme 到 Blend Shapes 的映射
        viseme_blend_map = {
            'sil': {
                'mouthClose': 1.0 * weight
            },
            'PP': {
                'mouthClose': 0.9 * weight,
                'mouthPress_L': 0.5 * weight,
                'mouthPress_R': 0.5 * weight
```

```
},
'FF': {
    'mouthFunnel': 0.3 * weight,
    'mouthPress_L': 0.4 * weight,
    'mouthPress_R': 0.4 * weight
},
'DD': {
    'mouthOpen': 0.3 * weight,
    'jawOpen': 0.2 * weight
},
'kk': {
    'mouthOpen': 0.2 * weight,
    'jawOpen': 0.1 * weight
},
'CH': {
    'mouthFunnel': 0.5 * weight,
    'mouthPucker': 0.3 * weight
},
'SS': {
    'mouthSmile_L': 0.3 * weight,
    'mouthSmile_R': 0.3 * weight,
    'mouthStretch_L': 0.2 * weight,
    'mouthStretch_R': 0.2 * weight
},
'aa': {
    'mouthOpen': 0.9 * weight,
    'jawOpen': 0.7 * weight
},
'E': {
    'mouthOpen': 0.5 * weight,
    'mouthSmile_L': 0.3 * weight,
    'mouthSmile_R': 0.3 * weight,
    'jawOpen': 0.3 * weight
},
'I': {
    'mouthSmile_L': 0.7 * weight,
    'mouthSmile_R': 0.7 * weight,
    'mouthStretch_L': 0.5 * weight,
    'mouthStretch_R': 0.5 * weight
},
'O': {
    'mouthFunnel': 0.8 * weight,
    'mouthPucker': 0.6 * weight,
    'jawOpen': 0.4 * weight
},
'U': {
    'mouthFunnel': 0.9 * weight,
    'mouthPucker': 0.8 * weight
}
```

```
        }

        return viseme_blend_map.get(viseme, {})


# 单例模式
_lipsync_generator: Optional[LipSyncGenerator] = None


def get_lipsync_generator() -> LipSyncGenerator:
    """获取唇形生成器实例（单例）"""
    global _lipsync_generator

    if _lipsync_generator is None:
        _lipsync_generator = LipSyncGenerator()

    return _lipsync_generator
```

代码行数：532 行

26. src/digital_human/__init__.py

功能描述：数字人模块初始化文件

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""
数字人集成模块
包含 WebSocket 服务器、表情控制、唇形同步
"""

from .websocket_server import DigitalHumanServer, get_instance,
set_instance
from .expression_controller import ExpressionController,
get_expression_controller
from .lipsync_generator import LipSyncGenerator, get_lipsync_generator

__all__ = [
    'DigitalHumanServer',
    'ExpressionController',
    'LipSyncGenerator',
    'get_instance',
    'set_instance',
    'get_expression_controller',
    'get_lipsync_generator'
]
```

代码行数：21 行

4.2 第八部分：API 接口层

27. src/api/app.py

功能描述：Flask 应用创建器，配置 Web 应用框架

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""
Flask 应用创建和配置
"""

from flask import Flask, jsonify
from flask_cors import CORS
from typing import Dict, Any

def create_app(config: Dict[str, Any]) -> Flask:
    """
    创建 Flask 应用

    Args:
        config: 配置字典

    Returns:
        Flask 应用实例
    """
    app = Flask(__name__)

    # 启用 CORS
    CORS(app)

    # 配置应用
    app.config['SECRET_KEY'] = 'your-secret-key-here'
    # 设置最大上传大小为 100MB（可根据需要调整）
    app.config['MAX_CONTENT_LENGTH'] = 100 * 1024 * 1024  # 100MB

    # 注册蓝图（路由）
    from src.api.routes import register_routes
    register_routes(app)

    # 错误处理
    @app.errorhandler(404)
    def not_found(error):
        return jsonify({'error': '接口不存在'}), 404

    @app.errorhandler(500)
    def internal_error(error):
        return jsonify({'error': '服务器内部错误'}), 500
```

```
    # 健康检查
    @app.route('/health')
    def health_check():
        return jsonify({
            'status': 'healthy',
            'app_name': config.get('app_name', 'QA System'),
            'version': config.get('version', '1.0.0')
        })

    return app
```

代码行数：57 行

28. src/api/routes.py

功能描述：API 路由定义，提供完整的 Web 服务接口

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""
API 路由定义
"""

from flask import Blueprint, request, jsonify, render_template,
Response, stream_with_context
from werkzeug.utils import secure_filename
from pathlib import Path
import json
import threading
import asyncio
from src.utils.logger import get_logger
from config.config_loader import load_config

logger = get_logger(__name__)

# 全局变量存储核心组件
_qa_system = None
_vector_store = None
_config = None
_session_manager = None
_digital_human_server = None  # 数字人 WebSocket 服务器

# 创建蓝图
main_bp = Blueprint('main', __name__)
api_bp = Blueprint('api', __name__, url_prefix='/api')

def register_routes(app):
    """注册所有路由"""
    app.register_blueprint(main_bp)
    app.register_blueprint(api_bp)

    # 在应用启动时自动启动数字人服务器
    # Flask 3.0+ 不再支持 before_first_request，改用延迟初始化
    def init_digital_human():
        """初始化数字人服务"""
        global _digital_human_server
        if _digital_human_server is not None:
            return  # 已经初始化过

        try:
            # 先初始化问答系统
```

```
            qa_system, _ = init_qa_system()

            # 启动数字人 WebSocket 服务器
            start_digital_human_server(qa_system)
        except Exception as e:
            logger.error(f"数字人服务初始化失败: {e}")
            # 不阻止主服务启动

    # 使用 before_request 配合全局标志实现首次请求初始化
    @app.before_request
    def lazy_init():
        """延迟初始化（仅在首次请求时执行）"""
        global _digital_human_server
        if _digital_human_server is None:
            init_digital_human()

def get_session_manager():
    """获取会话管理器实例"""
    global _session_manager
    if _session_manager is None:
        from src.core.session_manager import SessionManager
        _session_manager = SessionManager()
        logger.info("✓ 会话管理器初始化完成")
    return _session_manager


def start_digital_human_server(qa_system):
    """
    启动数字人 WebSocket 服务器

    Args:
        qa_system: 问答系统实例
    """
    global _digital_human_server

    if _digital_human_server is not None:
        logger.info("数字人服务器已经在运行")
        return

    try:
        from src.digital_human.websocket_server import
DigitalHumanServer, set_instance

        # 创建异步问答处理器
        async def qa_handler(question: str, user_id: str, session_id:
str = None):
            """
            异步问答处理器
```

```
            将 qa_system.answer_stream 转换为异步生成器

            Yields:
                消息字典
            """
            try:
                logger.info(f"[数字人] 处理问题: {question} (用户:
{user_id}, 会话: {session_id})")

                # 调用流式问答链
                for event in qa_system.answer_stream(
                    question=question,
                    session_id=session_id,
                    return_sources=True,
                    return_related=True
                ):
                    # 直接 yield 事件，格式已经正确
                    yield event

                    # 异步让出控制权
                    await asyncio.sleep(0)

            except Exception as e:
                logger.error(f"[数字人] 问答处理错误: {e}", exc_info=True)
                yield {
                    'type': 'error',
                    'data': {'message': str(e)}
                }

        # 读取配置
        config = load_config()
        fay_config = config.get('fay_integration', {})
        ws_config = fay_config.get('websocket', {})

        # 创建服务器
        _digital_human_server = DigitalHumanServer(
            host=ws_config.get('host', '0.0.0.0'),
            port=ws_config.get('port', 10003),
            qa_handler=qa_handler,
            enable_voice_output=ws_config.get('enable_voice_output',
True)
        )

        # 设置为全局实例（方便其他地方访问）
        set_instance(_digital_human_server)

        # 在后台线程启动
```

```
        def run_server():
            """在新的事件循环中运行服务器"""
            asyncio.run(_digital_human_server.start())

        thread = threading.Thread(target=run_server, daemon=True)
        thread.start()

        logger.info("=" * 60)
        logger.info("✓ 数字人 WebSocket 服务器已在后台启动")
        logger.info("   监听地址: ws://0.0.0.0:10003")
        logger.info("   兼容 Fay 协议，支持流式 RAG 问答")
        logger.info("=" * 60)

    except Exception as e:
        logger.error("=" * 60)
        logger.error(f"✗ 数字人服务器启动失败: {e}")
        logger.error("=" * 60)
        raise


def init_qa_system():
    """初始化问答系统组件（延迟加载，带锁机制）"""
    global _qa_system, _vector_store, _config

    # 如果已经初始化，直接返回
    if _qa_system is not None and _vector_store is not None:
        return _qa_system, _vector_store

    try:
        logger.info("=" * 60)
        logger.info("正在初始化问答系统（首次启动，请稍候...）")
        logger.info("=" * 60)

        # 加载配置
        _config = load_config()

        # 初始化向量存储
        from src.core.vector_store import VectorStoreManager

        logger.info("正在加载 Embedding 模型...")
        logger.info("首次使用会自动下载模型文件（约 1.3GB），需要 5-15 分钟")
        logger.info("下载完成后会缓存在本地，后续启动很快")

        _vector_store = VectorStoreManager(
            embedding_model_name=_config['embedding']['model_name'],
            persist_directory=_config['vectordb']['persist_directory'],
            device=_config['embedding'].get('device', 'cpu')
```

```
        )

        logger.info("✓ Embedding 模型加载成功")

        # 尝试加载已存在的向量库
        try:
            _vector_store.load_vectorstore()
            logger.info("✓ 成功加载已存在的向量库")
        except Exception as e:
            logger.warning(f"未找到已存在的向量库，将在上传文档后创建:
{str(e)}")

        # 初始化 LLM 客户端
        from src.llm.qwen_client import create_qwen_client
        from src.llm.prompt_template import PromptManager
        from src.core.qa_chain import QAChain

        logger.info("正在初始化 Qwen 大模型客户端...")
        llm_client = create_qwen_client(_config)
        prompt_manager = PromptManager()

        _qa_system = QAChain(
            vector_store=_vector_store,
            llm_client=llm_client,
            prompt_manager=prompt_manager,
            top_k=_config['vectordb']['retrieval']['top_k']
        )

        logger.info("=" * 60)
        logger.info("✓ 问答系统初始化完成！")
        logger.info("=" * 60)
        return _qa_system, _vector_store

    except Exception as e:
        logger.error("=" * 60)
        logger.error(f"✗ 问答系统初始化失败：{str(e)}")
        logger.error("=" * 60)
        logger.error("可能原因：")
        logger.error("1. 模型下载中断（网络问题）")
        logger.error("2. 依赖包未正确安装")
        logger.error("3. 显存/内存不足")
        logger.error("")
        logger.error("解决方案：")
        logger.error("1. 检查网络连接，等待模型下载完成")
        logger.error("2. 运行 fix_dependencies.bat 修复依赖")
        logger.error("3. 查看详细日志：logs/qa_system.log")
        raise
```

```
# ==================== 前端页面路由 ====================

@main_bp.route('/')
def index():
    """首页 - 对话式界面"""
    try:
        with open('templates/index_with_sessions.html', 'r',
encoding='utf-8') as f:
            return f.read()
    except Exception as e:
        # 如果文件不存在，返回简单版本
        logger.warning(f"无法加载模板文件: {e}")
        return """
    <!DOCTYPE html>
    <html lang="zh-CN">
    <head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-
scale=1.0">
        <title>智能问答系统 - 对话助手</title>
        <style>
            * {
                margin: 0;
                padding: 0;
                box-sizing: border-box;
            }

            body {
                font-family: 'Microsoft YaHei', 'Segoe UI', sans-serif;
                background: linear-gradient(135deg, #667eea 0%, #764ba2
100%);
                height: 100vh;
                overflow: hidden;
            }

            /* 主容器 */
            .chat-container {
                display: flex;
                flex-direction: column;
                height: 100vh;
                max-width: 1200px;
                margin: 0 auto;
                background: white;
                box-shadow: 0 0 50px rgba(0,0,0,0.3);
            }
```

```
            /* 顶部栏 */
            .header {
                background: linear-gradient(135deg, #667eea 0%, #764ba2
100%);
                color: white;
                padding: 20px 30px;
                display: flex;
                justify-content: space-between;
                align-items: center;
                box-shadow: 0 2px 10px rgba(0,0,0,0.1);
            }

            .header-left {
                display: flex;
                align-items: center;
                gap: 15px;
            }

            .header-title {
                font-size: 1.5em;
                font-weight: bold;
            }

            .header-subtitle {
                font-size: 0.9em;
                opacity: 0.9;
            }

            .header-buttons {
                display: flex;
                gap: 10px;
            }

            .header-btn {
                background: rgba(255,255,255,0.2);
                border: 1px solid rgba(255,255,255,0.3);
                color: white;
                padding: 8px 16px;
                border-radius: 20px;
                cursor: pointer;
                font-size: 0.9em;
                transition: all 0.3s;
            }

            .header-btn:hover {
                background: rgba(255,255,255,0.3);
            }

            /* 聊天区域 */
```

```
.chat-messages {
    flex: 1;
    overflow-y: auto;
    padding: 30px;
    background: #f8f9fa;
}

.chat-messages::-webkit-scrollbar {
    width: 8px;
}

.chat-messages::-webkit-scrollbar-track {
    background: #f1f1f1;
}

.chat-messages::-webkit-scrollbar-thumb {
    background: #888;
    border-radius: 4px;
}

.chat-messages::-webkit-scrollbar-thumb:hover {
    background: #555;
}

/* 欢迎消息 */
.welcome-message {
    text-align: center;
    padding: 50px 20px;
    color: #666;
}

.welcome-message h2 {
    color: #667eea;
    margin-bottom: 20px;
    font-size: 2em;
}

.welcome-message p {
    line-height: 1.8;
    margin-bottom: 10px;
}

.quick-actions {
    display: flex;
    gap: 15px;
    justify-content: center;
    margin-top: 30px;
    flex-wrap: wrap;
}
```

```css
.quick-action-btn {
    background: white;
    border: 2px solid #667eea;
    color: #667eea;
    padding: 12px 24px;
    border-radius: 25px;
    cursor: pointer;
    font-size: 0.95em;
    transition: all 0.3s;
}

.quick-action-btn:hover {
    background: #667eea;
    color: white;
    transform: translateY(-2px);
}

/* 消息气泡 */
.message {
    margin-bottom: 20px;
    display: flex;
    animation: fadeIn 0.3s ease-in;
}

@keyframes fadeIn {
    from { opacity: 0; transform: translateY(10px); }
    to { opacity: 1; transform: translateY(0); }
}

.message.user {
    justify-content: flex-end;
}

.message.assistant {
    justify-content: flex-start;
}

.message.system {
    justify-content: center;
}

.message-content {
    max-width: 70%;
    padding: 15px 20px;
    border-radius: 20px;
    line-height: 1.6;
    word-wrap: break-word;
}
```

```
            .message.user .message-content {
                background: linear-gradient(135deg, #667eea 0%, #764ba2
100%);
                color: white;
                border-bottom-right-radius: 5px;
            }

            .message.assistant .message-content {
                background: white;
                color: #333;
                box-shadow: 0 2px 5px rgba(0,0,0,0.1);
                border-bottom-left-radius: 5px;
            }

            .message.system .message-content {
                background: #e3f2fd;
                color: #1976d2;
                font-size: 0.9em;
                padding: 10px 20px;
                max-width: 90%;
                text-align: center;
            }

            .message-time {
                font-size: 0.75em;
                opacity: 0.7;
                margin-top: 5px;
            }

            .message-sources {
                margin-top: 10px;
                padding-top: 10px;
                border-top: 1px solid #eee;
                font-size: 0.85em;
                color: #666;
            }

            .message-sources-title {
                font-weight: bold;
                margin-bottom: 5px;
            }

            .source-item {
                margin: 3px 0;
                padding-left: 10px;
            }

            /* 输入区域 */
```

```
            .chat-input-area {
                padding: 20px 30px;
                background: white;
                border-top: 1px solid #e0e0e0;
            }

            .chat-input-container {
                display: flex;
                gap: 10px;
                align-items: flex-end;
            }

            .chat-input {
                flex: 1;
                border: 2px solid #e0e0e0;
                border-radius: 25px;
                padding: 12px 20px;
                font-size: 15px;
                resize: none;
                max-height: 120px;
                font-family: inherit;
                transition: border-color 0.3s;
            }

            .chat-input:focus {
                outline: none;
                border-color: #667eea;
            }

            .input-buttons {
                display: flex;
                gap: 10px;
            }

            .btn-send {
                background: linear-gradient(135deg, #667eea 0%, #764ba2
100%);
                color: white;
                border: none;
                padding: 12px 30px;
                border-radius: 25px;
                cursor: pointer;
                font-size: 15px;
                font-weight: bold;
                transition: all 0.3s;
            }

            .btn-send:hover:not(:disabled) {
                transform: translateY(-2px);
```

```
    box-shadow: 0 5px 15px rgba(102, 126, 234, 0.4);
}

.btn-send:disabled {
    opacity: 0.6;
    cursor: not-allowed;
}

.btn-upload {
    background: white;
    border: 2px solid #667eea;
    color: #667eea;
    padding: 12px 20px;
    border-radius: 25px;
    cursor: pointer;
    font-size: 15px;
    transition: all 0.3s;
}

.btn-upload:hover {
    background: #f0f4ff;
}

/* 输入提示 */
.input-hint {
    font-size: 0.85em;
    color: #999;
    margin-top: 8px;
    text-align: center;
}

/* 加载动画 */
.loading-dots {
    display: inline-flex;
    gap: 4px;
    align-items: center;
}

.loading-dots span {
    width: 8px;
    height: 8px;
    border-radius: 50%;
    background: #667eea;
    animation: bounce 1.4s infinite ease-in-out both;
}

.loading-dots span:nth-child(1) {
    animation-delay: -0.32s;
}
```

```
.loading-dots span:nth-child(2) {
    animation-delay: -0.16s;
}

@keyframes bounce {
    0%, 80%, 100% {
        transform: scale(0);
        opacity: 0.5;
    }
    40% {
        transform: scale(1);
        opacity: 1;
    }
}

/* 弹窗样式 */
.modal {
    display: none;
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    background: rgba(0,0,0,0.5);
    z-index: 1000;
    justify-content: center;
    align-items: center;
    animation: fadeIn 0.3s;
}

.modal.active {
    display: flex;
}

.modal-content {
    background: white;
    padding: 30px;
    border-radius: 20px;
    max-width: 500px;
    width: 90%;
    box-shadow: 0 20px 60px rgba(0,0,0,0.3);
    animation: slideUp 0.3s;
}

@keyframes slideUp {
    from { transform: translateY(50px); opacity: 0; }
    to { transform: translateY(0); opacity: 1; }
}
```

```
.modal-header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 20px;
}

.modal-title {
    font-size: 1.5em;
    color: #667eea;
    font-weight: bold;
}

.modal-close {
    background: none;
    border: none;
    font-size: 1.5em;
    cursor: pointer;
    color: #999;
    width: 30px;
    height: 30px;
    display: flex;
    align-items: center;
    justify-content: center;
    border-radius: 50%;
    transition: all 0.3s;
}

.modal-close:hover {
    background: #f0f0f0;
    color: #333;
}

.modal-body {
    margin-bottom: 20px;
}

.upload-area-modal {
    border: 3px dashed #667eea;
    border-radius: 15px;
    padding: 40px;
    text-align: center;
    background: #f8f9fa;
    transition: all 0.3s;
    cursor: pointer;
}

.upload-area-modal:hover {
```

```
            border-color: #764ba2;
            background: #f0f4ff;
        }

        .upload-icon {
            font-size: 3em;
            margin-bottom: 15px;
        }

        #fileInput {
            display: none;
        }

        .progress-bar {
            width: 100%;
            height: 10px;
            background: #e0e0e0;
            border-radius: 5px;
            overflow: hidden;
            margin: 20px 0;
            display: none;
        }

        .progress-fill {
            height: 100%;
            background: linear-gradient(135deg, #667eea 0%, #764ba2
100%);
            width: 0%;
            transition: width 0.3s;
        }

        .upload-status {
            text-align: center;
            color: #666;
            margin-top: 15px;
            display: none;
        }

        /* 通知弹窗 */
        .notification {
            position: fixed;
            top: 20px;
            right: 20px;
            background: white;
            padding: 20px 25px;
            border-radius: 10px;
            box-shadow: 0 5px 20px rgba(0,0,0,0.2);
            display: none;
            z-index: 2000;
```

```
    animation: slideInRight 0.3s;
    max-width: 400px;
}

@keyframes slideInRight {
    from { transform: translateX(400px); opacity: 0; }
    to { transform: translateX(0); opacity: 1; }
}

.notification.active {
    display: block;
}

.notification.success {
    border-left: 4px solid #4caf50;
}

.notification.error {
    border-left: 4px solid #f44336;
}

.notification.info {
    border-left: 4px solid #2196f3;
}

.notification-header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 10px;
}

.notification-title {
    font-weight: bold;
    font-size: 1.1em;
}

.notification.success .notification-title {
    color: #4caf50;
}

.notification.error .notification-title {
    color: #f44336;
}

.notification.info .notification-title {
    color: #2196f3;
}
```

```
        .notification-close {
            background: none;
            border: none;
            font-size: 1.2em;
            cursor: pointer;
            color: #999;
        }

        .notification-body {
            color: #666;
            line-height: 1.5;
        }

        /* 文档列表模态框 */
        .doc-list {
            max-height: 400px;
            overflow-y: auto;
        }

        .doc-item {
            background: #f8f9fa;
            padding: 15px;
            border-radius: 10px;
            margin-bottom: 10px;
            display: flex;
            align-items: center;
            gap: 10px;
        }

        .doc-icon {
            font-size: 1.5em;
        }

        .doc-info {
            flex: 1;
        }

        .doc-name {
            font-weight: bold;
            color: #333;
            margin-bottom: 3px;
        }

        .doc-size {
            font-size: 0.85em;
            color: #999;
        }
    </style>
</head>
```

```
    <body>
        <!-- 主聊天容器 -->
        <div class="chat-container">
            <!-- 顶部栏 -->
            <div class="header">
                <div class="header-left">
                    <div>
                        <div class="header-title">  智能问答助手</div>
                        <div class="header-subtitle">基于 Qwen 大模型
</div>
            </div>
                    </div>
                <div class="header-buttons">
                    <button class="header-btn"
onclick="showDocuments()">  文档库</button>
                    <button class="header-btn"
onclick="showUploadModal()">  上传</button>
                    <button class="header-btn" onclick="clearChat()">
清空</button>
                </div>
                    </div>

            <!-- 聊天消息区域 -->
            <div class="chat-messages" id="chatMessages">
                <div class="welcome-message" id="welcomeMessage">
                    <h2>  你好！我是你的智能问答助手</h2>
                    <p>我可以帮你基于已上传的文档回答问题</p>
                    <p>请先上传文档，然后开始提问吧！</p>
                    <div class="quick-actions">
                        <button class="quick-action-btn"
onclick="showUploadModal()">  上传文档</button>
                        <button class="quick-action-btn"
onclick="showDocuments()">  查看文档库</button>
                </div>
                </div>
            </div>

            <!-- 输入区域 -->
            <div class="chat-input-area">
                <div class="chat-input-container">
                    <textarea
                        id="messageInput"
                        class="chat-input"
                        placeholder="输入你的问题..."
                        rows="1"
                        onkeydown="handleKeyDown(event)"
                    ></textarea>
```

```
                    <div class="input-buttons">
                        <button class="btn-upload"
onclick="showUploadModal()"> </button>
                        <button class="btn-send" id="sendBtn"
onclick="sendMessage()">发送</button>
                    </div>
                </div>
                <div class="input-hint">按 Enter 发送，Shift + Enter 换行
</div>
                </div>
            </div>

        <!-- 上传文档模态框 -->
        <div class="modal" id="uploadModal">
            <div class="modal-content">
                <div class="modal-header">
                    <div class="modal-title">  上传文档</div>
                    <button class="modal-close"
onclick="closeModal('uploadModal')">×</button>
                </div>
                <div class="modal-body">
                    <div class="upload-area-modal" id="uploadArea"
onclick="document.getElementById('fileInput').click()">
                        <div class="upload-icon"> </div>
                        <div style="margin: 10px 0;">拖拽文件到此处或点击选
择</div>
                        <div style="font-size: 0.9em; color: #999;">支持:
TXT, PDF, Word, Markdown, CSV</div>
                    </div>
                    <input type="file" id="fileInput"
accept=".txt,.pdf,.docx,.md,.csv" multiple>
                    <div class="progress-bar" id="progressBar">
                        <div class="progress-fill"
id="progressFill"></div>
                </div>
                    <div class="upload-status" id="uploadStatus"></div>
                </div>
            </div>
            </div>

        <!-- 文档列表模态框 -->
        <div class="modal" id="documentsModal">
            <div class="modal-content">
                <div class="modal-header">
                    <div class="modal-title">  文档库</div>
                    <button class="modal-close"
onclick="closeModal('documentsModal')">×</button>
                </div>
```

```
                <div class="modal-body">
                    <div class="doc-list" id="docList">
                        <div style="text-align: center; color: #999;
padding: 20px;">加载中...</div>
                </div>
                </div>
                </div>
            </div>

        <!-- 通知弹窗 -->
        <div class="notification" id="notification">
            <div class="notification-header">
                <div class="notification-title" id="notificationTitle">
通知</div>
                <button class="notification-close"
onclick="closeNotification()">×</button>
            </div>
            <div class="notification-body" id="notificationBody"></div>
        </div>

        <script>
            // 全局变量
            let isWaiting = false;
            let chatHistory = [];

            // 初始化
            window.addEventListener('load', () => {
                loadChatHistory();
                setupDragDrop();
                autoResizeTextarea();
            });

            // 自动调整文本框高度
            function autoResizeTextarea() {
                const textarea =
document.getElementById('messageInput');
                textarea.addEventListener('input', function() {
                    this.style.height = 'auto';
                    this.style.height = Math.min(this.scrollHeight, 120)
+ 'px';
                });
            }

            // 处理键盘事件
            function handleKeyDown(event) {
                if (event.key === 'Enter' && !event.shiftKey) {
                    event.preventDefault();
                    sendMessage();
```

```
            }
        }

        // 发送消息
        async function sendMessage() {
            const input = document.getElementById('messageInput');
            const message = input.value.trim();

            if (!message || isWaiting) return;

            // 添加用户消息
            addMessage('user', message);
            input.value = '';
            input.style.height = 'auto';

            // 隐藏欢迎消息
            document.getElementById('welcomeMessage').style.display
= 'none';

            // 禁用发送按钮
            isWaiting = true;
            const sendBtn = document.getElementById('sendBtn');
            sendBtn.disabled = true;
            sendBtn.textContent = '思考中...';

            // 添加加载消息
            const loadingId = addMessage('assistant', '<div
class="loading-dots"><span></span><span></span><span></span></div>',
true);

            try {
                const response = await fetch('/api/question', {
                    method: 'POST',
                    headers: {
                        'Content-Type': 'application/json'
                    },
                    body: JSON.stringify({ question: message })
                });

                const data = await response.json();

                // 移除加载消息
                document.getElementById(loadingId).remove();

                // 添加回答
                let answerHtml = data.answer || '抱歉，我无法回答这个问
题';
```

```
                    // 添加来源信息
                    if (data.sources && data.sources.length > 0) {
                        answerHtml += '<div class="message-sources">';
                        answerHtml += '<div class="message-sources-
title">  参考来源：</div>';
                        data.sources.forEach((source, index) => {
                            answerHtml += `<div class="source-
item">${index + 1}. ${source.source_file}</div>`;
                        });
                        answerHtml += '</div>';
                    }

                    addMessage('assistant', answerHtml);

                } catch (error) {
                    // 移除加载消息
                    document.getElementById(loadingId).remove();
                    addMessage('assistant', '抱歉，处理您的问题时出现了错误：
' + error.message);
                    showNotification('error', '错误', '处理消息时出现问题');
                }

                // 恢复发送按钮
                isWaiting = false;
                sendBtn.disabled = false;
                sendBtn.textContent = '发送';
            }

            // 添加消息到聊天窗口
            function addMessage(type, content, isTemp = false) {
                const messagesDiv =
document.getElementById('chatMessages');
                const messageDiv = document.createElement('div');
                const messageId = 'msg-' + Date.now() + '-' +
Math.random();
                messageDiv.id = messageId;
                messageDiv.className = 'message ' + type;

                const time = new Date().toLocaleTimeString('zh-CN',
{ hour: '2-digit', minute: '2-digit' });

                messageDiv.innerHTML = `
                    <div class="message-content">
                        ${content}
                        ${!isTemp ? '<div class="message-time">' + time
+ '</div>' : ''}
                    </div>
                `;
```

```
                messagesDiv.appendChild(messageDiv);
                messagesDiv.scrollTop = messagesDiv.scrollHeight;

                // 保存到历史记录（非临时消息）
                if (!isTemp) {
                    chatHistory.push({ type, content, time });
                    saveChatHistory();
                }

                return messageId;
            }

            // 添加系统消息
            function addSystemMessage(content) {
                const messagesDiv =
document.getElementById('chatMessages');
                const messageDiv = document.createElement('div');
                messageDiv.className = 'message system';
                messageDiv.innerHTML = `<div class="message-
content">${content}</div>`;
                messagesDiv.appendChild(messageDiv);
                messagesDiv.scrollTop = messagesDiv.scrollHeight;
            }

            // 清空聊天
            function clearChat() {
                if (confirm('确定要清空所有聊天记录吗？')) {
                    chatHistory = [];
                    saveChatHistory();
                    document.getElementById('chatMessages').innerHTML =
`
                        <div class="welcome-message"
id="welcomeMessage">
                            <h2>  你好！我是你的智能问答助手</h2>
                            <p>我可以帮你基于已上传的文档回答问题</p>
                            <p>请先上传文档，然后开始提问吧！</p>
                            <div class="quick-actions">
                                <button class="quick-action-btn"
onclick="showUploadModal()">  上传文档</button>
                                <button class="quick-action-btn"
onclick="showDocuments()">  查看文档库</button>
                            </div>
                        </div>
                    `;
                    showNotification('info', '提示', '聊天记录已清空');
                }
            }
```

```
            // 保存聊天历史
            function saveChatHistory() {
                localStorage.setItem('chatHistory',
JSON.stringify(chatHistory));
            }

            // 加载聊天历史
            function loadChatHistory() {
                const saved = localStorage.getItem('chatHistory');
                if (saved) {
                    try {
                        chatHistory = JSON.parse(saved);
                        if (chatHistory.length > 0) {

document.getElementById('welcomeMessage').style.display = 'none';
                            chatHistory.forEach(msg => {
                                const messagesDiv =
document.getElementById('chatMessages');
                                const messageDiv =
document.createElement('div');
                                messageDiv.className = 'message ' +
msg.type;
                                messageDiv.innerHTML = `
                                    <div class="message-content">
                                        ${msg.content}
                                        <div class="message-
time">${msg.time}</div>
                                    </div>
                                `;
                                messagesDiv.appendChild(messageDiv);
                            });

document.getElementById('chatMessages').scrollTop =

document.getElementById('chatMessages').scrollHeight;
                        }
                    } catch (e) {
                        console.error('加载聊天历史失败:', e);
                    }
                }
            }

            // 显示/隐藏模态框
            function showModal(modalId) {

document.getElementById(modalId).classList.add('active');
            }
```

```
            function closeModal(modalId) {
document.getElementById(modalId).classList.remove('active');
            }

            // 显示上传模态框
            function showUploadModal() {
                showModal('uploadModal');
                document.getElementById('progressBar').style.display =
'none';
                document.getElementById('uploadStatus').style.display =
'none';
            }

            // 显示文档列表
            async function showDocuments() {
                showModal('documentsModal');
                const docList = document.getElementById('docList');
                docList.innerHTML = '<div style="text-align: center;
color: #999; padding: 20px;">加载中...</div>';

                try {
                    const response = await fetch('/api/documents');
                    const data = await response.json();
                    const docs = data.documents || [];

                    if (docs.length === 0) {
                        docList.innerHTML = '<div style="text-align:
center; color: #999; padding: 20px;">暂无文档</div>';
                        return;
                    }

                    docList.innerHTML = docs.map(doc => {
                        const sizeMB = (doc.size / 1024 /
1024).toFixed(2);
                        return `
                            <div class="doc-item">
                                <div class="doc-icon"> </div>
                                <div class="doc-info">
                                    <div class="doc-
name">${doc.filename}</div>
                                    <div class="doc-size">${sizeMB}
MB</div>
                                </div>
                            </div>
                        `;
                    }).join('');
                } catch (error) {
                    docList.innerHTML = '<div style="text-align: center;
```

```
color: #f44336; padding: 20px;">加载失败</div>';
                }
            }

            // 文件上传处理
            const fileInput = document.getElementById('fileInput');
            fileInput.addEventListener('change', (e) => {
                handleFiles(e.target.files);
            });

            async function handleFiles(files) {
                if (files.length === 0) return;

                const progressBar =
document.getElementById('progressBar');
                const progressFill =
document.getElementById('progressFill');
                const uploadStatus =
document.getElementById('uploadStatus');

                progressBar.style.display = 'block';
                uploadStatus.style.display = 'block';

                for (let i = 0; i < files.length; i++) {
                    const file = files[i];
                    uploadStatus.textContent = `正在上传: ${file.name}
(${i + 1}/${files.length})`;

                    const formData = new FormData();
                    formData.append('file', file);

                    try {
                        progressFill.style.width = '50%';

                        const response = await fetch('/api/upload', {
                            method: 'POST',
                            body: formData
                        });

                        const data = await response.json();
                        progressFill.style.width = '100%';

                        if (response.ok) {
                            showNotification('success', '上传成功',
                                `文档 "${file.name}" 已成功上传并处理，生成
了 ${data.chunks_count || 0} 个文档块`);
                            addSystemMessage(`  文档 "${file.name}" 已添
加到知识库`);
```

```
                        } else {
                            showNotification('error', '上传失败',
data.error || '未知错误');
                        }

                        await new Promise(resolve => setTimeout(resolve,
500));

                    } catch (error) {
                        showNotification('error', '上传失败',
error.message);
                    }
                }

                uploadStatus.textContent = '上传完成! ';
                setTimeout(() => {
                    closeModal('uploadModal');
                    progressBar.style.display = 'none';
                    uploadStatus.style.display = 'none';
                    progressFill.style.width = '0%';
                    fileInput.value = '';
                }, 1500);
            }

            // 拖拽上传
            function setupDragDrop() {
                const uploadArea =
document.getElementById('uploadArea');

                ['dragenter', 'dragover', 'dragleave',
'drop'].forEach(eventName => {
                    uploadArea.addEventListener(eventName,
preventDefaults, false);
                });

                function preventDefaults(e) {
                    e.preventDefault();
                    e.stopPropagation();
                }

                ['dragenter', 'dragover'].forEach(eventName => {
                    uploadArea.addEventListener(eventName, () => {
                        uploadArea.style.borderColor = '#764ba2';
                        uploadArea.style.background = '#f0f4ff';
                    }, false);
                });

                ['dragleave', 'drop'].forEach(eventName => {
                    uploadArea.addEventListener(eventName, () => {
```

```
                        uploadArea.style.borderColor = '#667eea';
                        uploadArea.style.background = '#f8f9fa';
                    }, false);
                });

                uploadArea.addEventListener('drop', (e) => {
                    const files = e.dataTransfer.files;
                    handleFiles(files);
                }, false);
            }

            // 通知系统
            function showNotification(type, title, message) {
                const notification =
document.getElementById('notification');
                const notificationTitle =
document.getElementById('notificationTitle');
                const notificationBody =
document.getElementById('notificationBody');

                notification.className = 'notification ' + type + '
active';
                notificationTitle.textContent = title;
                notificationBody.textContent = message;

                setTimeout(() => {
                    closeNotification();
                }, 5000);
            }

            function closeNotification() {

document.getElementById('notification').classList.remove('active');
            }

            // 点击模态框外部关闭
            window.addEventListener('click', (e) => {
                if (e.target.classList.contains('modal')) {
                    e.target.classList.remove('active');
                }
            });
        </script>
    </body>
    </html>
    """


# ==================== API 路由 ====================
```

```
@api_bp.route('/upload', methods=['POST'])
def upload_document():
    """上传文档接口"""
    try:
        if 'file' not in request.files:
            return jsonify({'error': '没有文件'}), 400

        file = request.files['file']
        if file.filename == '':
            return jsonify({'error': '文件名为空'}), 400

        # 保存文件
        filename = secure_filename(file.filename)

        # 验证文件格式
        import os
        file_ext = os.path.splitext(filename)[1].lower()
        supported_formats = ['.txt', '.pdf', '.docx', '.md', '.csv']

        if file_ext not in supported_formats:
            return jsonify({
                'error': f'不支持的文件格式: {file_ext}',
                'supported_formats': supported_formats
            }), 400

        upload_folder = Path('data/documents')
        upload_folder.mkdir(parents=True, exist_ok=True)

        filepath = upload_folder / filename

        # 检查文件是否为空
        file.seek(0, 2)  # 移到文件末尾
        file_size = file.tell()
        file.seek(0)  # 回到开头

        if file_size == 0:
            return jsonify({'error': '文件为空，请选择有效文件'}), 400

        # 保存文件
        file.save(str(filepath))

        logger.info("=" * 60)
        logger.info(f"  文档上传成功: {filename}")
        logger.info(f"   文件大小: {file_size / 1024:.2f} KB")
        logger.info(f"   保存路径: {filepath}")
        logger.info("=" * 60)
```

```
        # 处理文档并添加到向量数据库
        try:
            logger.info(f"开始处理文档: {filename}")
            qa_system, vector_store = init_qa_system()

            from src.core.document_loader import DocumentLoader
            from src.core.text_splitter import SmartTextSplitter

            # 加载文档
            logger.info(f"正在加载文档...")
            loader = DocumentLoader()
            documents = loader.load_file(str(filepath))
            logger.info(f"✓ 文档加载成功，共 {len(documents)} 个文档对象")

            # 分割文本
            logger.info(f"正在分割文本...")
            config = load_config()
            splitter = SmartTextSplitter(

chunk_size=config['document']['text_splitter']['chunk_size'],

chunk_overlap=config['document']['text_splitter']['chunk_overlap'],

separators=config['document']['text_splitter'].get('separators')
            )
            chunks = splitter.split_documents(documents)
            logger.info(f"✓ 文本分割完成，生成 {len(chunks)} 个文档块")

            # 添加到向量库
            logger.info(f"正在添加到向量数据库...")
            vector_store.add_documents(chunks)
            logger.info(f"✓ 向量化完成")

            logger.info("=" * 60)
            logger.info(f"✅ 文档处理完成: {filename}")
            logger.info(f"   文档块数量: {len(chunks)}")
            logger.info("=" * 60)

            return jsonify({
                'message': '文档上传并处理成功',
                'filename': filename,
                'status': 'processed',
                'chunks_count': len(chunks)
            })

        except UnicodeDecodeError as e:
            logger.error("=" * 60)
```

```
        logger.error(f"✗ 文件编码错误: {filename}")
        logger.error(f"   错误信息: {str(e)}")
        logger.error("=" * 60)
        # 删除已上传的文件
        if filepath.exists():
            import os
            os.remove(filepath)
        return jsonify({
            'error': '文件编码错误',
            'detail': '请确保文件使用 UTF-8 编码，或尝试转换编码后再上传',
            'technical_info': str(e)
        }), 400

    except ValueError as e:
        logger.error("=" * 60)
        logger.error(f"✗ 文件格式错误: {filename}")
        logger.error(f"   错误信息: {str(e)}")
        logger.error("=" * 60)
        if filepath.exists():
            import os
            os.remove(filepath)
        return jsonify({
            'error': '文件格式错误',
            'detail': str(e)
        }), 400

    except MemoryError:
        logger.error("=" * 60)
        logger.error(f"✗ 内存不足，文件太大: {filename}")
        logger.error("=" * 60)
        if filepath.exists():
            import os
            os.remove(filepath)
        return jsonify({
            'error': '文件太大，处理失败',
            'detail': '请尝试上传较小的文件，或将大文件分割后上传'
        }), 413

    except Exception as e:
        logger.error("=" * 60)
        logger.error(f"✗ 文档处理失败: {filename}")
        logger.error(f"   错误信息: {str(e)}", exc_info=True)
        logger.error("=" * 60)
        # 保留文件，但标记为未处理
        return jsonify({
            'message': '文档已上传，但处理失败',
            'filename': filename,
```

```
                'status': 'upload_only',
                'error': str(e),
                'suggestion': '请检查文件格式是否正确，或查看系统日志了解详情'
            }), 200  # 返回 200 但标注处理失败

    except Exception as e:
        logger.error("=" * 60)
        logger.error(f"❌ 文档上传失败")
        logger.error(f"   错误信息: {str(e)}")
        logger.error("=" * 60)
        return jsonify({'error': str(e)}), 500


@api_bp.route('/question', methods=['POST'])
def ask_question():
    """问答接口"""
    try:
        data = request.get_json()
        question = data.get('question', '')
        session_id = data.get('session_id', '')

        if not question:
            return jsonify({'error': '问题不能为空'}), 400

        logger.info("=" * 60)
        logger.info(f"  收到用户问题: {question}")
        if session_id:
            logger.info(f"   会话 ID: {session_id}")
        logger.info("=" * 60)

        # 初始化会话管理器
        session_manager = get_session_manager()

        # 如果提供了 session_id，保存用户消息
        if session_id:
            session_manager.add_message(session_id, 'user', question)

        # 初始化并调用问答系统
        try:
            qa_system, vector_store = init_qa_system()

            logger.info("正在检索相关文档...")
            # 调用问答链（支持会话记忆）
            result = qa_system.answer(
                question=question,
                session_id=session_id if session_id else None,
                return_sources=True,
```

```
                return_related=True
            )

            logger.info(f"✓ 检索完成，找到 {len(result.get('sources',
[]))} 个相关文档块")
            logger.info("正在生成回答...")
            logger.info("=" * 60)
            logger.info(f"✓ 回答生成完成")
            if result.get('sources'):
                logger.info(f"   参考来源: {[s.get('source_file', '') for
s in result['sources'][:3]]}")
            logger.info("=" * 60)

            # 如果提供了 session_id，保存助手回答
            if session_id:
                session_manager.add_message(
                    session_id,
                    'assistant',
                    result.get('answer', ''),
                    sources=result.get('sources', [])
                )

            return jsonify(result)

        except Exception as e:
            logger.error("=" * 60)
            logger.error(f"✗ 问答系统调用失败")
            logger.error(f"   问题: {question}")
            logger.error(f"   错误: {str(e)}")
            logger.error("=" * 60)

            error_answer = f'抱歉，处理您的问题时出现错误: {str(e)}'

            # 保存错误消息
            if session_id:
                session_manager.add_message(session_id, 'assistant',
error_answer)

            return jsonify({
                'question': question,
                'answer': error_answer,
                'sources': [],
                'related': [],
                'error': str(e)
            })

    except Exception as e:
        logger.error("=" * 60)
```

```
        logger.error(f"✗ 问答处理失败: {str(e)}")
        logger.error("=" * 60)
        return jsonify({'error': str(e)}), 500


@api_bp.route('/question/stream', methods=['POST'])
def ask_question_stream():
    """流式问答接口（Server-Sent Events）"""
    try:
        data = request.get_json()
        question = data.get('question', '').strip()
        session_id = data.get('session_id', '')

        if not question:
            return jsonify({'error': '问题不能为空'}), 400

        logger.info("=" * 60)
        logger.info(f"  收到流式问题: {question}")
        if session_id:
            logger.info(f"   会话 ID: {session_id}")
        logger.info("=" * 60)

        # 初始化会话管理器
        session_manager = get_session_manager()

        # 如果提供了 session_id，保存用户消息
        if session_id:
            session_manager.add_message(session_id, 'user', question)

        def generate():
            """SSE 生成器"""
            try:
                # 初始化问答系统
                qa_system, vector_store = init_qa_system()

                full_answer = ""
                sources = []

                # 流式问答（支持会话记忆）
                for event in qa_system.answer_stream(
                    question=question,
                    session_id=session_id if session_id else None,
                    return_sources=True,
                    return_related=True
                ):
                    # 格式化为 SSE 格式
                    event_type = event.get('type')
                    event_data = event.get('data')
```

```
                    # 收集完整答案和来源
                    if event_type == 'content':
                        full_answer += event_data
                    elif event_type == 'sources':
                        sources = event_data

                    # 发送事件
                    yield f"event: {event_type}\n"
                    yield f"data: {json.dumps(event_data,
ensure_ascii=False)}\n\n"

                # 如果提供了 session_id，保存助手回答
                if session_id and full_answer:
                    session_manager.add_message(
                        session_id,
                        'assistant',
                        full_answer,
                        sources=sources
                    )

                logger.info("✓ 流式问答完成")

            except Exception as e:
                logger.error(f"流式问答错误: {str(e)}")
                yield f"event: error\n"
                yield f"data: {json.dumps({'message': str(e)},
ensure_ascii=False)}\n\n"

        return Response(
            stream_with_context(generate()),
            mimetype='text/event-stream',
            headers={
                'Cache-Control': 'no-cache',
                'Connection': 'keep-alive',
                'X-Accel-Buffering': 'no'  # 禁用 nginx 缓冲
            }
        )

    except Exception as e:
        logger.error(f"流式问答接口错误: {str(e)}")
        return jsonify({'error': str(e)}), 500


@api_bp.route('/chat/stream', methods=['POST'])
def chat_stream():
    """流式多轮对话接口"""
    try:
```

```
        data = request.get_json()
        messages = data.get('messages', [])
        session_id = data.get('session_id', '')

        if not messages:
            return jsonify({'error': '消息列表不能为空'}), 400

        logger.info(f"  收到流式对话请求，消息数: {len(messages)}")

        def generate():
            """SSE 生成器"""
            try:
                # 初始化问答系统
                qa_system, vector_store = init_qa_system()

                full_answer = ""
                sources = []

                # 流式对话
                for event in qa_system.chat_stream(
                    messages=messages,
                    return_sources=True
                ):
                    # 格式化为 SSE 格式
                    event_type = event.get('type')
                    event_data = event.get('data')

                    # 收集完整答案和来源
                    if event_type == 'content':
                        full_answer += event_data
                    elif event_type == 'sources':
                        sources = event_data

                    # 发送事件
                    yield f"event: {event_type}\n"
                    yield f"data: {json.dumps(event_data,
ensure_ascii=False)}\n\n"

                # 保存到会话（如果需要）
                if session_id and full_answer:
                    session_manager = get_session_manager()
                    # 获取最后一条用户消息
                    last_user_msg = ""
                    for msg in reversed(messages):
                        if msg.get('role') == 'user':
                            last_user_msg = msg.get('content', '')
                            break
```

```
                    if last_user_msg:
                        session_manager.add_message(session_id, 'user',
last_user_msg)
                        session_manager.add_message(session_id,
'assistant', full_answer, sources=sources)

                logger.info("✅ 流式对话完成")

            except Exception as e:
                logger.error(f"流式对话错误: {str(e)}")
                yield f"event: error\n"
                yield f"data: {json.dumps({'message': str(e)},
ensure_ascii=False)}\n\n"

        return Response(
            stream_with_context(generate()),
            mimetype='text/event-stream',
            headers={
                'Cache-Control': 'no-cache',
                'Connection': 'keep-alive',
                'X-Accel-Buffering': 'no'
            }
        )

    except Exception as e:
        logger.error(f"流式对话接口错误: {str(e)}")
        return jsonify({'error': str(e)}), 500


@api_bp.route('/documents', methods=['GET'])
def list_documents():
    """获取文档列表"""
    try:
        upload_folder = Path('data/documents')
        if not upload_folder.exists():
            return jsonify({'documents': []})

        documents = []
        for file_path in upload_folder.iterdir():
            if file_path.is_file():
                documents.append({
                    'filename': file_path.name,
                    'size': file_path.stat().st_size,
                    'created_at': file_path.stat().st_ctime
                })

        return jsonify({'documents': documents})

    except Exception as e:
```

```
        logger.error(f"获取文档列表失败: {str(e)}")
        return jsonify({'error': str(e)}), 500


# ==================== 会话管理 API ====================

@api_bp.route('/sessions', methods=['GET'])
def list_sessions():
    """获取会话列表"""
    try:
        session_manager = get_session_manager()
        sessions = session_manager.list_sessions()
        logger.info(f"获取会话列表: {len(sessions)} 个会话")
        return jsonify({'sessions': sessions})
    except Exception as e:
        logger.error(f"获取会话列表失败: {str(e)}")
        return jsonify({'error': str(e)}), 500


@api_bp.route('/sessions', methods=['POST'])
def create_session():
    """创建新会话"""
    try:
        data = request.get_json() or {}
        title = data.get('title', None)

        session_manager = get_session_manager()
        session = session_manager.create_session(title)

        logger.info(f"创建新会话: {session['id']}")
        return jsonify({'session': session})
    except Exception as e:
        logger.error(f"创建会话失败: {str(e)}")
        return jsonify({'error': str(e)}), 500


@api_bp.route('/sessions/<session_id>', methods=['GET'])
def get_session(session_id):
    """获取会话详情"""
    try:
        session_manager = get_session_manager()
        session = session_manager.get_session(session_id)

        if not session:
            return jsonify({'error': '会话不存在'}), 404

        logger.info(f"获取会话详情: {session_id}")
        return jsonify({'session': session})
```

```
    except Exception as e:
        logger.error(f"获取会话详情失败: {str(e)}")
        return jsonify({'error': str(e)}), 500


@api_bp.route('/sessions/<session_id>', methods=['DELETE'])
def delete_session(session_id):
    """删除会话"""
    try:
        session_manager = get_session_manager()
        success = session_manager.delete_session(session_id)

        if not success:
            return jsonify({'error': '会话不存在'}), 404

        logger.info(f"删除会话: {session_id}")
        return jsonify({'message': '会话已删除'})
    except Exception as e:
        logger.error(f"删除会话失败: {str(e)}")
        return jsonify({'error': str(e)}), 500


@api_bp.route('/sessions/<session_id>/title', methods=['PUT'])
def update_session_title(session_id):
    """更新会话标题"""
    try:
        data = request.get_json()
        title = data.get('title', '')

        if not title:
            return jsonify({'error': '标题不能为空'}), 400

        session_manager = get_session_manager()
        success = session_manager.update_session_title(session_id,
title)

        if not success:
            return jsonify({'error': '会话不存在'}), 404

        logger.info(f"更新会话标题: {session_id} -> {title}")
        return jsonify({'message': '标题已更新', 'title': title})
    except Exception as e:
        logger.error(f"更新会话标题失败: {str(e)}")
        return jsonify({'error': str(e)}), 500
```

```
# ==================== 记忆管理API ====================

@api_bp.route('/memory/sessions', methods=['POST'])
def create_memory_session():
    """创建新的记忆会话"""
    try:
        data = request.get_json() or {}
        user_id = data.get('user_id', 'default_user')
        title = data.get('title', None)

        from src.core.memory_manager import get_memory_manager
        memory_manager = get_memory_manager()

        session_id = memory_manager.create_session(user_id, title)
        session_info = memory_manager.get_session_info(session_id)

        logger.info(f"创建记忆会话: {session_id}")
        return jsonify({
            'session_id': session_id,
            'session_info': session_info
        })

    except Exception as e:
        logger.error(f"创建记忆会话失败: {str(e)}")
        return jsonify({'error': str(e)}), 500


@api_bp.route('/memory/sessions/<session_id>/history', methods=['GET'])
def get_memory_history(session_id):
    """获取会话的对话历史"""
    try:
        limit = request.args.get('limit', type=int)
        include_summary = request.args.get('include_summary',
'true').lower() == 'true'

        from src.core.memory_manager import get_memory_manager
        memory_manager = get_memory_manager()

        history = memory_manager.get_conversation_history(
            session_id,
            limit=limit,
            include_summary=include_summary
        )

        return jsonify({
            'session_id': session_id,
            'history': history,
            'count': len(history)
        })
```

```
    except Exception as e:
        logger.error(f"获取对话历史失败: {str(e)}")
        return jsonify({'error': str(e)}), 500


@api_bp.route('/memory/sessions/<session_id>/context', methods=['GET'])
def get_memory_context(session_id):
    """获取会话的上下文记忆"""
    try:
        question = request.args.get('question', '')
        max_tokens = request.args.get('max_tokens', type=int)

        from src.core.memory_manager import get_memory_manager
        memory_manager = get_memory_manager()

        messages, history_text = memory_manager.get_contextual_memory(
            session_id,
            question,
            max_tokens=max_tokens
        )

        return jsonify({
            'session_id': session_id,
            'messages': messages,
            'history_text': history_text,
            'message_count': len(messages)
        })

    except Exception as e:
        logger.error(f"获取上下文记忆失败: {str(e)}")
        return jsonify({'error': str(e)}), 500


@api_bp.route('/memory/sessions/<session_id>/info', methods=['GET'])
def get_memory_session_info(session_id):
    """获取会话信息"""
    try:
        from src.core.memory_manager import get_memory_manager
        memory_manager = get_memory_manager()

        session_info = memory_manager.get_session_info(session_id)

        if not session_info:
            return jsonify({'error': '会话不存在'}), 404

        return jsonify({
            'session_info': session_info
        })
```

```
    except Exception as e:
        logger.error(f"获取会话信息失败: {str(e)}")
        return jsonify({'error': str(e)}), 500


@api_bp.route('/memory/users/<user_id>/sessions', methods=['GET'])
def list_user_memory_sessions(user_id):
    """获取用户的记忆会话列表"""
    try:
        limit = request.args.get('limit', 50, type=int)
        active_only = request.args.get('active_only', 'true').lower()
== 'true'

        from src.core.memory_manager import get_memory_manager
        memory_manager = get_memory_manager()

        sessions = memory_manager.list_user_sessions(
            user_id=user_id,
            limit=limit,
            active_only=active_only
        )

        return jsonify({
            'user_id': user_id,
            'sessions': sessions,
            'count': len(sessions)
        })

    except Exception as e:
        logger.error(f"获取用户会话列表失败: {str(e)}")
        return jsonify({'error': str(e)}), 500


@api_bp.route('/memory/sessions/<session_id>/title', methods=['PUT'])
def update_memory_session_title(session_id):
    """更新记忆会话标题"""
    try:
        data = request.get_json()
        title = data.get('title', '')

        if not title:
            return jsonify({'error': '标题不能为空'}), 400

        from src.core.memory_manager import get_memory_manager
        memory_manager = get_memory_manager()

        success = memory_manager.update_session_title(session_id, title)
```

```
        if not success:
            return jsonify({'error': '会话不存在'}), 404

        logger.info(f"更新记忆会话标题: {session_id} -> {title}")
        return jsonify({'message': '标题已更新', 'title': title})

    except Exception as e:
        logger.error(f"更新记忆会话标题失败: {str(e)}")
        return jsonify({'error': str(e)}), 500


@api_bp.route('/memory/sessions/<session_id>', methods=['DELETE'])
def delete_memory_session(session_id):
    """删除记忆会话"""
    try:
        from src.core.memory_manager import get_memory_manager
        memory_manager = get_memory_manager()

        success = memory_manager.delete_session(session_id)

        if not success:
            return jsonify({'error': '会话不存在'}), 404

        logger.info(f"删除记忆会话: {session_id}")
        return jsonify({'message': '会话已删除'})

    except Exception as e:
        logger.error(f"删除记忆会话失败: {str(e)}")
        return jsonify({'error': str(e)}), 500


@api_bp.route('/memory/cleanup', methods=['POST'])
def cleanup_old_memory_sessions():
    """清理旧的记忆会话"""
    try:
        data = request.get_json() or {}
        days_old = data.get('days_old', 30)

        from src.core.memory_manager import get_memory_manager
        memory_manager = get_memory_manager()

        cleaned_count = memory_manager.cleanup_old_sessions(days_old)

        logger.info(f"清理了 {cleaned_count} 个旧记忆会话")
        return jsonify({
            'message': f'已清理 {cleaned_count} 个旧会话',
            'cleaned_count': cleaned_count
        })
```

```
    except Exception as e:
        logger.error(f"清理旧会话失败: {str(e)}")
        return jsonify({'error': str(e)}), 500


@api_bp.route('/voice/tts', methods=['POST'])
def text_to_speech():
    """文字转语音接口"""
    try:
        data = request.get_json()
        text = data.get('text', '').strip()
        voice = data.get('voice', 'female')
        emotion = data.get('emotion', 'neutral')
        rate = data.get('rate', 1.0)

        if not text:
            return jsonify({'error': '文本不能为空'}), 400

        logger.info(f"  TTS 请求: {text[:50]}... (voice: {voice}, emotion:
{emotion})")

        # 导入 TTS 处理器
        from src.voice.tts_handler import get_tts_handler

        try:
            # 获取 TTS 实例
            tts = get_tts_handler(engine='edge')

            # 合成语音
            success, audio_file = tts.synthesize_sync(
                text=text,
                voice=voice,
                emotion=emotion,
                rate=rate,
                use_cache=True
            )

            if success and audio_file:
                logger.info(f"✓ TTS 合成成功: {audio_file}")

                # 返回音频文件 URL
                from pathlib import Path
                filename = Path(audio_file).name
                audio_url = f"/api/voice/audio/{filename}"

                # 获取缓存统计
                cache_count, cache_size = tts.get_cache_size()
```

```
                return jsonify({
                    'success': True,
                    'audio_url': audio_url,
                    'audio_file': audio_file,
                    'cache_info': {
                        'count': cache_count,
                        'size': cache_size
                    }
                })
            else:
                return jsonify({'error': 'TTS 合成失败'}), 500

        except ImportError as e:
            logger.error(f"TTS 模块未安装: {e}")
            return jsonify({
                'error': 'TTS 功能未安装',
                'detail': '请先安装: pip install edge-tts'
            }), 500
        except Exception as e:
            logger.error(f"TTS 合成失败: {e}")
            return jsonify({'error': str(e)}), 500

    except Exception as e:
        logger.error(f"TTS 接口错误: {str(e)}")
        return jsonify({'error': str(e)}), 500


@api_bp.route('/voice/audio/<filename>')
def get_audio_file(filename):
    """获取音频文件"""
    try:
        from flask import send_file
        from pathlib import Path

        audio_dir = Path("data/audio_cache")
        audio_file = audio_dir / filename

        if not audio_file.exists():
            return jsonify({'error': '音频文件不存在'}), 404

        return send_file(
            audio_file,
            mimetype='audio/mpeg',
            as_attachment=False,
            download_name=filename
        )
```

```
    except Exception as e:
        logger.error(f"获取音频文件失败: {str(e)}")
        return jsonify({'error': str(e)}), 500


@api_bp.route('/voice/cache/clear', methods=['POST'])
def clear_audio_cache():
    """清理音频缓存"""
    try:
        data = request.get_json() or {}
        older_than_days = data.get('older_than_days', 7)

        from src.voice.tts_handler import get_tts_handler

        tts = get_tts_handler()
        tts.clear_cache(older_than_days=older_than_days)

        cache_count, cache_size = tts.get_cache_size()

        return jsonify({
            'message': f'已清理超过 {older_than_days} 天的缓存',
            'remaining': {
                'count': cache_count,
                'size': cache_size
            }
        })

    except Exception as e:
        logger.error(f"清理缓存失败: {str(e)}")
        return jsonify({'error': str(e)}), 500


@api_bp.route('/docs')
def api_docs():
    """API 文档"""
    return """
    <!DOCTYPE html>
    <html lang="zh-CN">
    <head>
        <meta charset="UTF-8">
        <title>API 文档</title>
        <style>
            body { font-family: 'Microsoft YaHei', sans-serif; max-
width: 1000px; margin: 0 auto; padding: 40px; }
            h1 { color: #667eea; }
            .endpoint { background: #f5f5f5; padding: 20px; margin:
20px 0; border-radius: 10px; }
            .method { display: inline-block; padding: 5px 10px; border-
```

```
radius: 5px; color: white; font-weight: bold; }
            .post { background: #4caf50; }
            .get { background: #2196f3; }
            code { background: #e0e0e0; padding: 2px 6px; border-radius:
3px; }
        </style>
    </head>
    <body>
        <h1>智能问答系统 API 文档</h1>

        <div class="endpoint">
            <h2><span class="method post">POST</span> /api/upload</h2>
            <p><strong>功能：</strong>上传文档</p>
            <p><strong>参数：</strong></p>
            <ul>
                <li><code>file</code>: 文件对象（multipart/form-data）
</li>
            </ul>
            <p><strong>返回：</strong></p>
            <pre>{"message": "文档上传成功", "filename": "example.pdf",
"status": "pending_processing"}</pre>
        </div>

        <div class="endpoint">
            <h2><span class="method post">POST</span>
/api/question</h2>
            <p><strong>功能：</strong>提问并获取答案</p>
            <p><strong>参数：</strong></p>
            <ul>
                <li><code>question</code>: 问题文本（JSON）</li>
            </ul>
            <p><strong>返回：</strong></p>
            <pre>{"question": "...", "answer": "...", "sources": [...],
"related": [...]}</pre>
        </div>

        <div class="endpoint">
            <h2><span class="method get">GET</span> /api/documents</h2>
            <p><strong>功能：</strong>获取已上传文档列表</p>
            <p><strong>返回：</strong></p>
            <pre>{"documents": [{"filename": "...", "size": 1024,
"created_at": 1234567890}]}</pre>
        </div>

        <div class="endpoint">
            <h2><span class="method get">GET</span> /health</h2>
            <p><strong>功能：</strong>健康检查</p>
```

```
            <p><strong>返回：</strong></p>
            <pre>{"status": "healthy", "app_name": "...", "version":
"..."}</pre>
        </div>
    </body>
    </html>
    """
```

代码行数：2161 行

29. src/api/__init__.py

功能描述：API 模块初始化文件

```
# -*- coding: utf-8 -*-
"""
API 模块
"""

from .app import create_app

__all__ = ['create_app']
```

代码行数：12 行

4.2 第九部分：第三方集成

30. src/integration/fay_bridge.py

功能描述：Fay 数字人桥接器，实现与 Fay 系统的兼容

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""
Fay 数字人桥接模块
实现与 Fay 数字人系统的 WebSocket 通信
"""

import asyncio
import websockets
import json
from typing import Dict, Any, Optional
from src.core.qa_chain import QAChain
from src.utils.logger import get_logger

logger = get_logger(__name__)


class FayBridge:
    """Fay 数字人桥接器"""

    def __init__(
        self,
        qa_chain: QAChain,
        host: str = "localhost",
        port: int = 10003,
        fay_server_url: Optional[str] = None
    ):
        """
        初始化 Fay 桥接器

        Args:
            qa_chain: 问答链实例
            host: 服务监听地址
            port: 服务监听端口
            fay_server_url: Fay 服务器 WebSocket 地址
        """
        self.qa_chain = qa_chain
        self.host = host
        self.port = port
        self.fay_server_url = fay_server_url or "ws://localhost:10002"

        # 连接管理
```

```
        self.connections = set()
        self.fay_connection = None

        logger.info(f"Fay 桥接器初始化: {host}:{port}")

    async def handle_client(self, websocket, path):
        """
        处理客户端连接

        Args:
            websocket: WebSocket 连接
            path: 连接路径
        """
        # 注册连接
        self.connections.add(websocket)
        client_id = id(websocket)
        logger.info(f"新客户端连接: {client_id}")

        try:
            async for message in websocket:
                try:
                    data = json.loads(message)
                    logger.info(f"收到消息: {data}")

                    # 处理不同类型的消息
                    msg_type = data.get('type', '')

                    if msg_type == 'question':
                        # 处理问题
                        response = await self.handle_question(data)
                    elif msg_type == 'chat':
                        # 处理对话
                        response = await self.handle_chat(data)
                    elif msg_type == 'ping':
                        # 心跳
                        response = {'type': 'pong', 'timestamp':
data.get('timestamp')}
                    else:
                        response = {'type': 'error', 'message': f'未知消
息类型: {msg_type}'}

                    # 发送响应
                    await websocket.send(json.dumps(response,
ensure_ascii=False))

                except json.JSONDecodeError as e:
                    logger.error(f"JSON 解析错误: {str(e)}")
```

```
                    await websocket.send(json.dumps({
                        'type': 'error',
                        'message': 'JSON 格式错误'
                    }))
                except Exception as e:
                    logger.error(f"消息处理错误: {str(e)}")
                    await websocket.send(json.dumps({
                        'type': 'error',
                        'message': str(e)
                    }))

        except websockets.exceptions.ConnectionClosed:
            logger.info(f"客户端断开连接: {client_id}")
        finally:
            # 取消注册
            self.connections.remove(websocket)

    async def handle_question(self, data: Dict[str, Any]) -> Dict[str,
Any]:
        """
        处理问题

        Args:
            data: 请求数据

        Returns:
            响应数据
        """
        question = data.get('text', '')

        if not question:
            return {
                'type': 'error',
                'message': '问题不能为空'
            }

        # 调用问答链
        result = self.qa_chain.answer(
            question=question,
            return_sources=True,
            return_related=True
        )

        # 分析情感（简单实现）
        emotion = self.analyze_emotion(result['answer'])

        return {
            'type': 'answer',
```

```
            'question': question,
            'answer': result['answer'],
            'sources': result.get('sources', []),
            'related': result.get('related', []),
            'emotion': emotion,
            'timestamp': data.get('timestamp')
        }

    async def handle_chat(self, data: Dict[str, Any]) -> Dict[str, Any]:
        """
        处理多轮对话

        Args:
            data: 请求数据

        Returns:
            响应数据
        """
        messages = data.get('messages', [])

        if not messages:
            return {
                'type': 'error',
                'message': '消息列表不能为空'
            }

        # 调用问答链
        result = self.qa_chain.chat(
            messages=messages,
            return_sources=True
        )

        # 分析情感
        emotion = self.analyze_emotion(result['answer'])

        return {
            'type': 'chat_response',
            'answer': result['answer'],
            'sources': result.get('sources', []),
            'emotion': emotion,
            'timestamp': data.get('timestamp')
        }

    def analyze_emotion(self, text: str) -> str:
        """
        分析文本情感

        Args:
```

```
            text: 文本内容

        Returns:
            情感标签 (happy/sad/neutral/angry/surprised)
        """
        # 简单的关键词匹配
        # 实际应用中可以接入更专业的情感分析模型

        positive_keywords = ['很好', '不错', '优秀', '成功', '完成', '太棒了
', '很高兴']
        negative_keywords = ['抱歉', '遗憾', '失败', '错误', '问题', '困难']

        text_lower = text.lower()

        positive_count = sum(1 for kw in positive_keywords if kw in
text_lower)
        negative_count = sum(1 for kw in negative_keywords if kw in
text_lower)

        if positive_count > negative_count:
            return 'happy'
        elif negative_count > positive_count:
            return 'sad'
        else:
            return 'neutral'

    async def connect_to_fay(self):
        """连接到 Fay 服务器"""
        try:
            logger.info(f"连接到 Fay 服务器: {self.fay_server_url}")
            self.fay_connection = await
websockets.connect(self.fay_server_url)
            logger.info("已连接到 Fay 服务器")

            # 保持连接
            async for message in self.fay_connection:
                logger.info(f"收到 Fay 消息: {message}")
                # 处理 Fay 的消息

        except Exception as e:
            logger.error(f"连接 Fay 服务器失败: {str(e)}")

    async def start_server(self):
        """启动 WebSocket 服务器"""
        logger.info(f"启动 WebSocket 服务器: {self.host}:{self.port}")

        async with websockets.serve(self.handle_client, self.host,
```

```
self.port):
            logger.info("WebSocket 服务器已启动")
            await asyncio.Future()  # 永久运行

    def run(self):
        """运行服务"""
        try:
            asyncio.run(self.start_server())
        except KeyboardInterrupt:
            logger.info("服务被中断")
        except Exception as e:
            logger.error(f"服务运行错误: {str(e)}")


# 独立运行脚本
if __name__ == "__main__":
    from config.config_loader import load_config
    from src.core.vector_store import VectorStoreManager
    from src.llm.qwen_client import create_qwen_client
    from src.llm.prompt_template import PromptManager
    from src.core.qa_chain import QAChain

    # 加载配置
    config = load_config()

    # 初始化组件
    vector_store = VectorStoreManager(
        embedding_model_name=config['embedding']['model_name'],
        persist_directory=config['vectordb']['persist_directory'],
        device=config['embedding']['device']
    )

    llm_client = create_qwen_client(config)
    prompt_manager = PromptManager()

    qa_chain = QAChain(
        vector_store=vector_store,
        llm_client=llm_client,
        prompt_manager=prompt_manager,
        top_k=config['vectordb']['retrieval']['top_k']
    )

    # 创建并运行桥接器
    fay_config = config.get('fay_integration', {})
    bridge = FayBridge(
        qa_chain=qa_chain,
        host=fay_config.get('websocket', {}).get('host', 'localhost'),
        port=fay_config.get('websocket', {}).get('port', 10003)
```

```
    )

    bridge.run()
```

代码行数：278 行

31. src/integration/__init__.py

功能描述：集成模块初始化文件

```
# -*- coding: utf-8 -*-
"""
集成模块
"""

from .fay_bridge import FayBridge

__all__ = ['FayBridge']
```

代码行数：12 行

4.2 第十部分：模块初始化

32. src/__init__.py

功能描述：源代码根模块初始化文件

```
# -*- coding: utf-8 -*-
"""
智能问答系统
"""

__version__ = "1.0.0"
__author__ = "Your Name"
```

代码行数：11 行

33. src/core/__init__.py

功能描述：核心模块初始化文件，导出核心类和函数

```
# -*- coding: utf-8 -*-
"""
核心功能模块
"""

from .document_loader import DocumentLoader
from .text_splitter import SmartTextSplitter
from .vector_store import VectorStoreManager
from .qa_chain import QAChain

__all__ = [
    'DocumentLoader',
    'SmartTextSplitter',
    'VectorStoreManager',
    'QAChain'
]
```

代码行数：20 行

5. 代码统计信息

源代码统计信息：

总文件数：38 个
总代码行数：9,508 行
开发语言：Python (85%), HTML (9%), JavaScript (3%), 其他 (3%)

模块分布：
- 系统入口与配置：3 个文件，176 行
- 核心工具模块：3 个文件，180 行
- 数据处理核心：7 个文件，2,315 行
- AI 模型集成：3 个文件，690 行
- 语音处理模块：4 个文件，887 行
- 情感分析模块：2 个文件，369 行
- 数字人控制：4 个文件，1,393 行
- API 接口层：3 个文件，2,116 行
- 第三方集成：2 个文件，275 行
- 模块初始化：7 个文件，87 行

6. 技术特点说明

系统技术特点：

1. 先进的 RAG 架构：基于向量检索的智能问答
2. 多模态融合：文本、语音、表情、唇形一体化
3. 实时通信：WebSocket 双向通信，低延迟交互
4. 模块化设计：高内聚低耦合，易于维护扩展
5. 异步架构：支持高并发，性能优异
6. 完全免费：使用开源组件，无 API 费用
7. 标准兼容：支持 UE5/Unity 数字人引擎对接
8. 智能优化：缓存机制、流式处理、自动回退